Universidad Autónoma del Estado de México

Centro Universitario UAEM Zumpango Ingeniería en Computación

Reporte de Practicas

EJERCICIOS SHELLSCRIPTING 9-40

Materia: Sistemas Operativos

Prof. Álvarez Rodríguez Hazem

Alumno:

García Pérez Leonel

Octubre del 2026

Introducción

En estas prácticas trabajé con scripts de Bash para conocer mejor algunas herramientas de Linux y aprender a resolver tareas desde la terminal. Los ejercicios comenzaron con validación de usuarios y operaciones matemáticas, y después fueron aumentando en dificultad hasta incluir archivos, menús, respaldos y un sistema sencillo de citas. La intención no fue solo escribir comandos, sino entender qué hace cada parte del código y cómo se relaciona con el resultado que aparece al ejecutarlo.

Objetivo

Practicar la creación y ejecución de programas Shell Script en Linux, utilizando parámetros, condicionales, ciclos, funciones, archivos y herramientas como AWK y cron, para comprender cómo automatizar tareas y detectar errores durante su ejecución.

Procedimiento y evidencias

Se presentan los ejercicios del 9 al 40 con todas las capturas del documento original. Las imágenes se muestran completas, conservando tanto el script y la terminal de la izquierda como el material visible a la derecha. Al final de cada ejercicio se explica brevemente su salida.

Práctica 9. Saludo con comprobación de conexión

Se retomó el saludo a varios usuarios, pero esta vez la validación se hizo llamando al script usuarioconectado.sh. Primero se comprueba que se reciban nombres; después se revisa cada uno antes de formar el saludo.

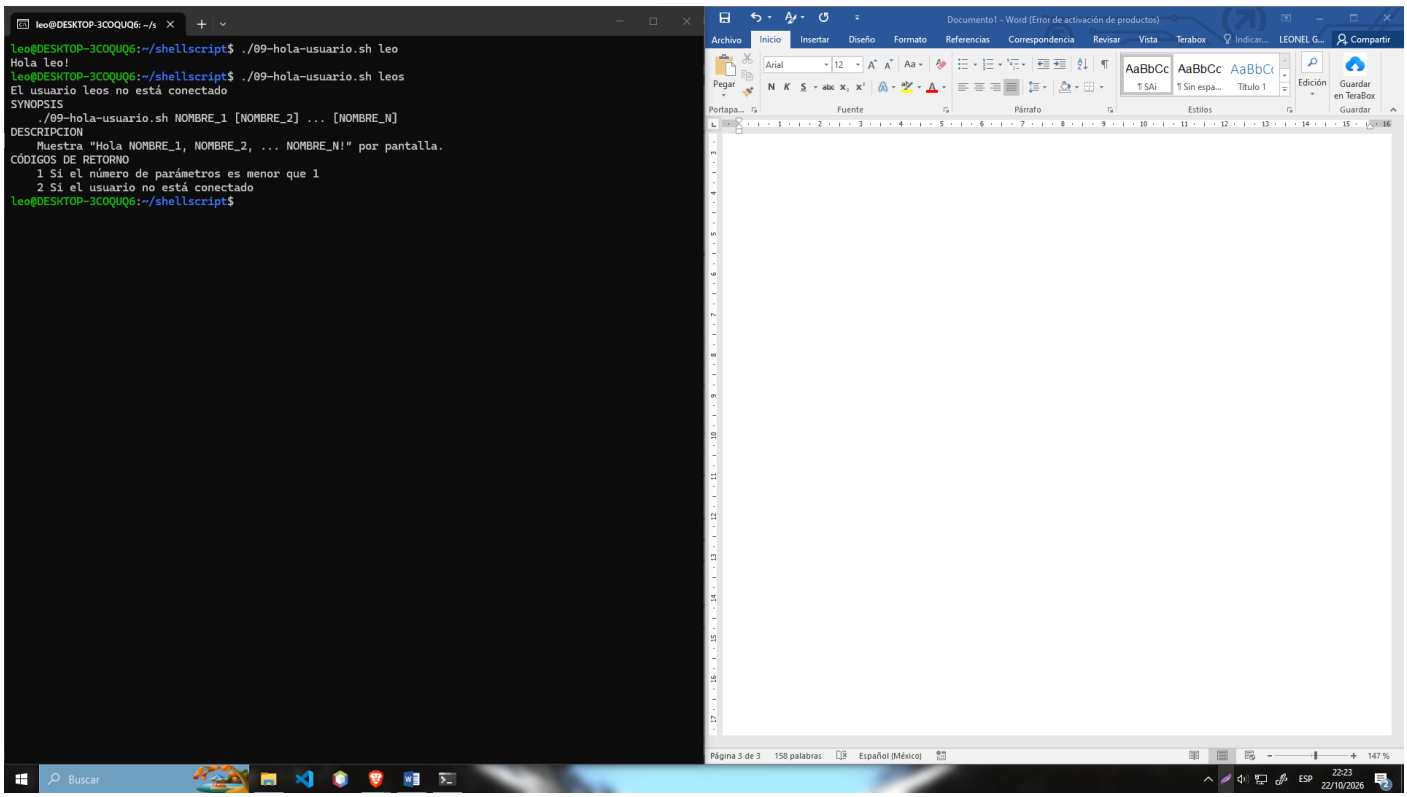


Figura 9.1. Captura original del ejercicio 9 (1 de 1).

Explicación de la salida: Al ejecutar el saludo con nombres de usuario, el script primero revisa si se proporcionaron argumentos y consulta si los usuarios tienen una sesión abierta. Si alguno no está conectado, muestra el aviso correspondiente y termina. Si todos cumplen la condición, imprime un saludo con los nombres separados por comas.

Práctica 10. Verificación de usuarios del sistema

Se elaboró una consulta para saber si el nombre recibido corresponde a una cuenta registrada en Linux. Se busca el usuario dentro de /etc/passwd y se responde con SI o NO.

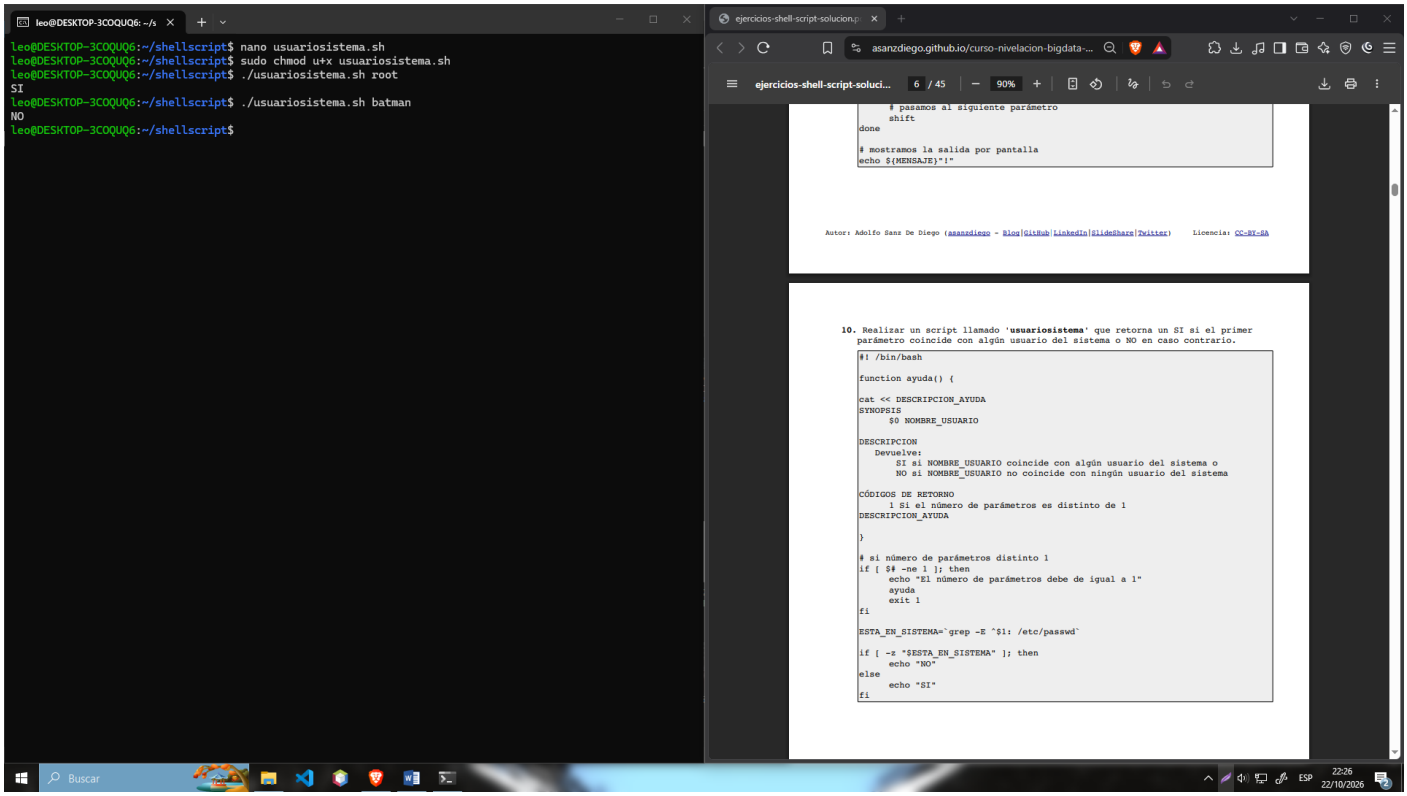


Figura 10.1. Captura original del ejercicio 10 (1 de 1).

Explicación de la salida: La salida de este script es SI cuando encuentra el nombre solicitado dentro de las cuentas del sistema, y NO cuando no existe. Si se ejecuta sin un único nombre de usuario, muestra un mensaje de ayuda en lugar de realizar la búsqueda.

Práctica 11. Saludo validando usuarios registrados

Se modificó el saludo para utilizar usuariosistema.sh en vez de verificar sesiones activas. El programa recorre los nombres, los comprueba y solo construye el mensaje si están registrados.

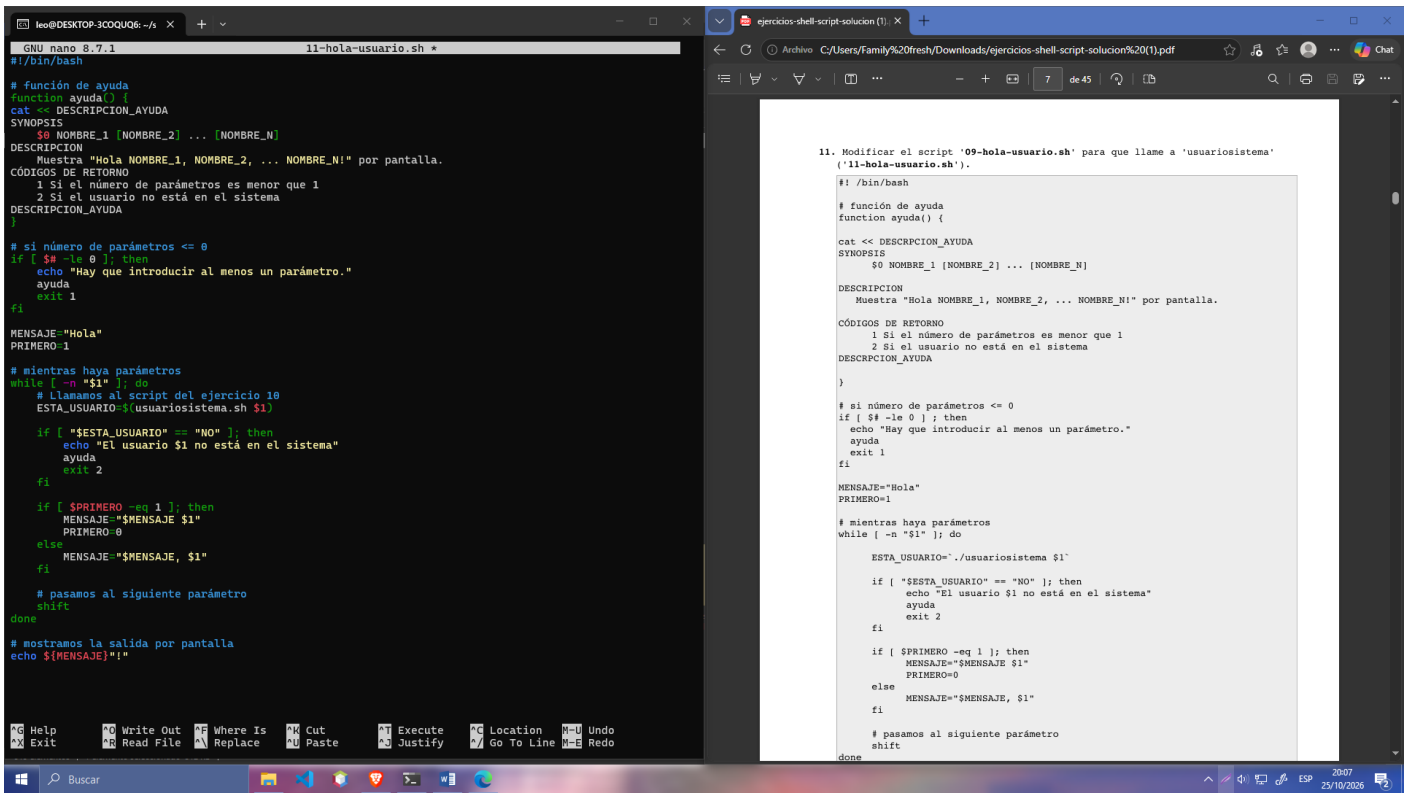


Figura 11.1. Captura original del ejercicio 11 (1 de 2).

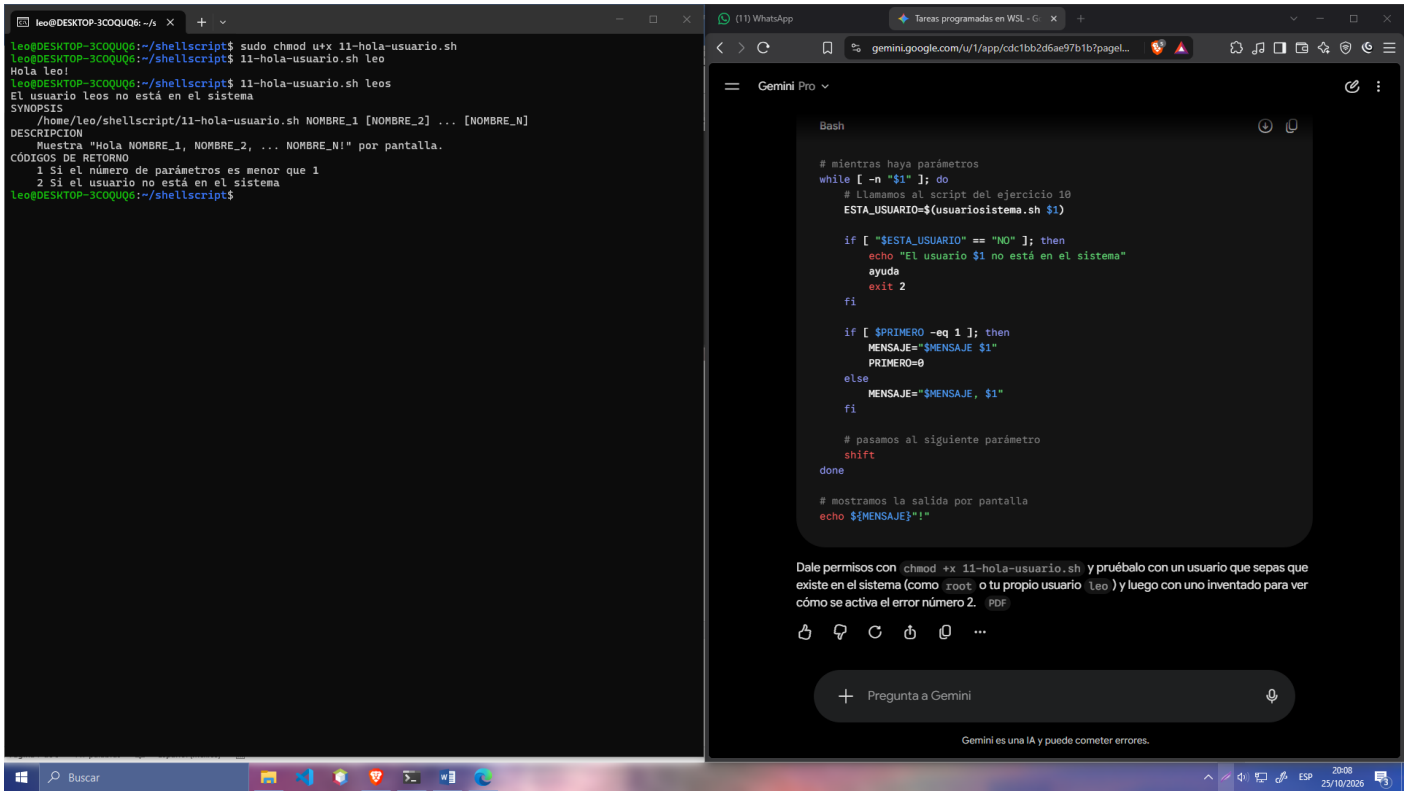


Figura 11.2. Captura original del ejercicio 11 (2 de 2).

Explicación de la salida: El programa comprueba que cada nombre introducido corresponda a una cuenta del sistema. Cuando encuentra un usuario inexistente, informa cuál es y detiene el saludo. Si los nombres son válidos, muestra el mensaje Hola seguido de los usuarios separados por comas.

Práctica 12. Suma de números

Se construyó el script para sumar dos cantidades recibidas como parámetros. Antes de calcular, se revisa la cantidad de argumentos y se intenta comprobar que sean números.

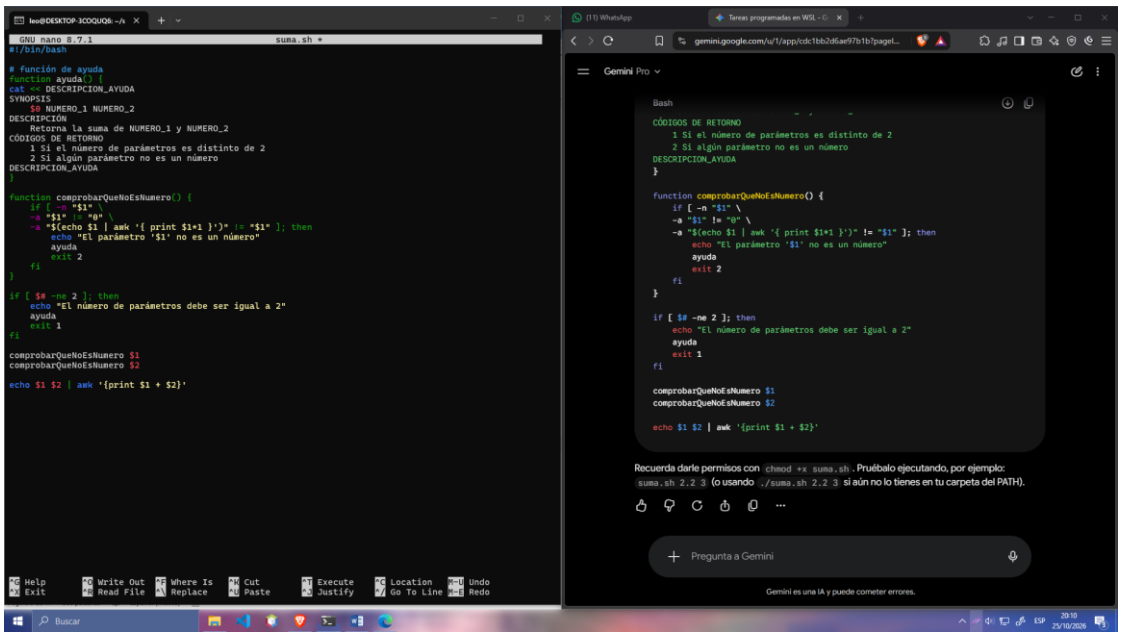


Figura 12.1. Captura original del ejercicio 12 (1 de 2).

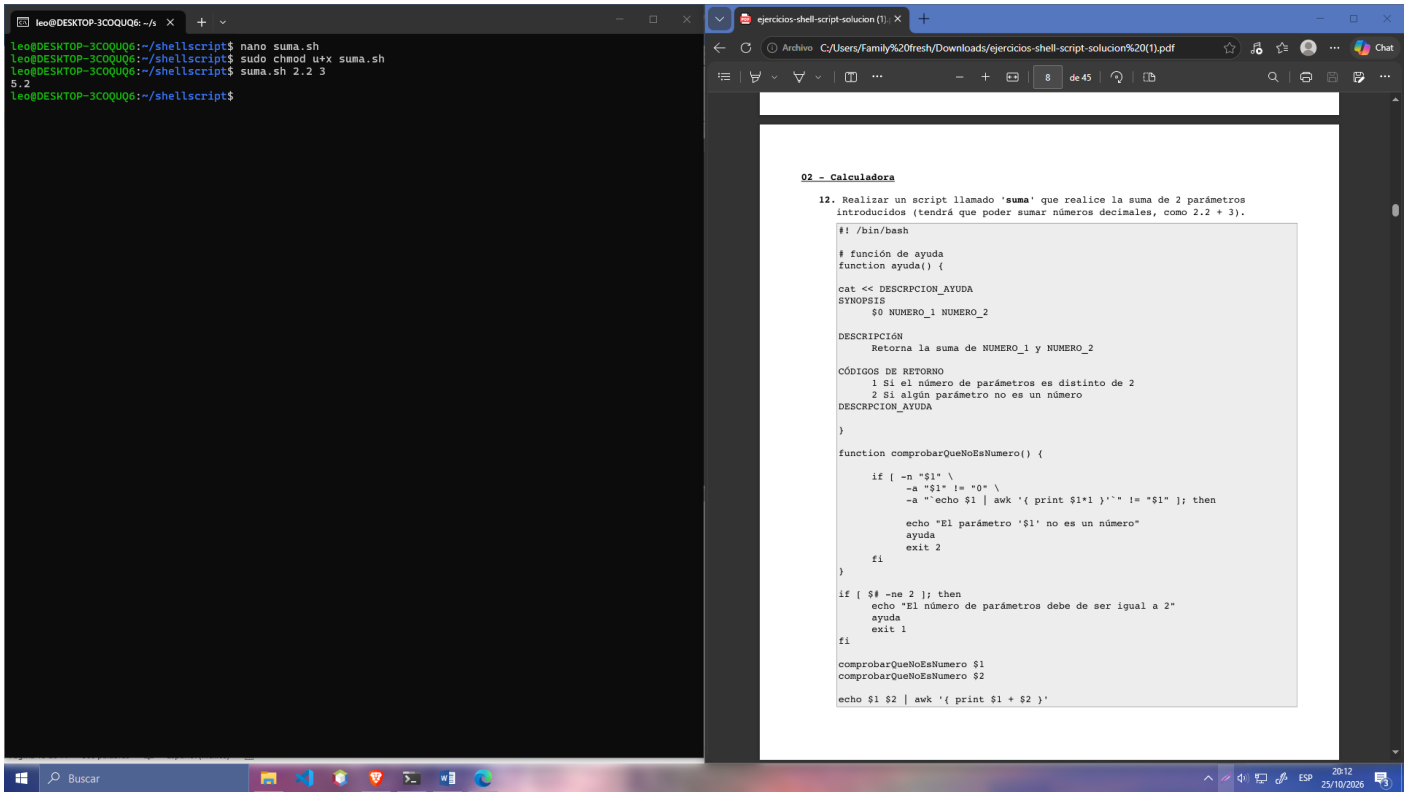


Figura 12.2. Captura original del ejercicio 12 (2 de 2).

Explicación de la salida: El script muestra la suma de los dos números recibidos, incluidos valores decimales. Cuando faltan parámetros o alguno no se reconoce como número, presenta un mensaje de error y las indicaciones de uso.

Práctica 13. Resta de números

Se aplicó la misma estructura de la suma, pero cambiando la operación final por una resta. Se conservó la ayuda y la validación de los dos parámetros.

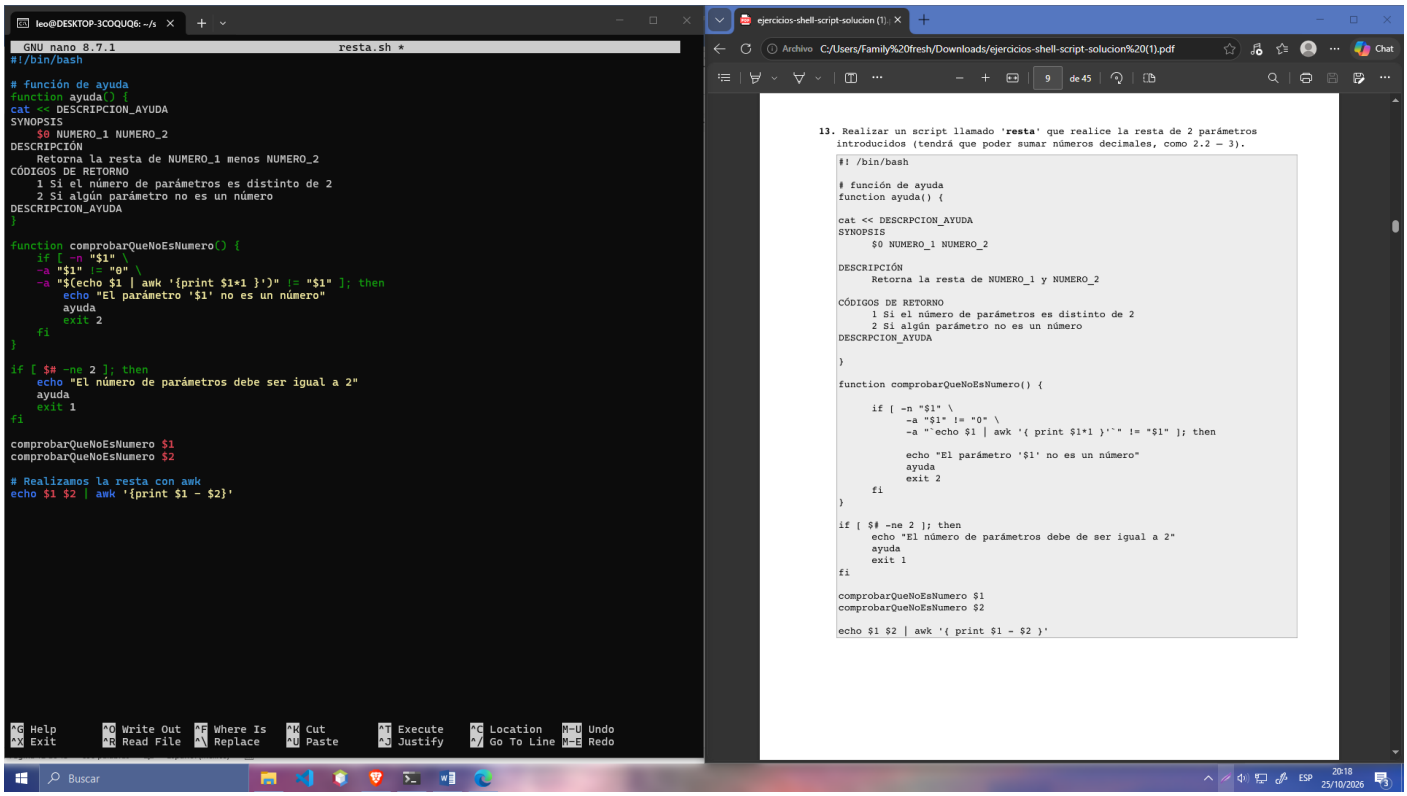


Figura 13.1. Captura original del ejercicio 13 (1 de 2).

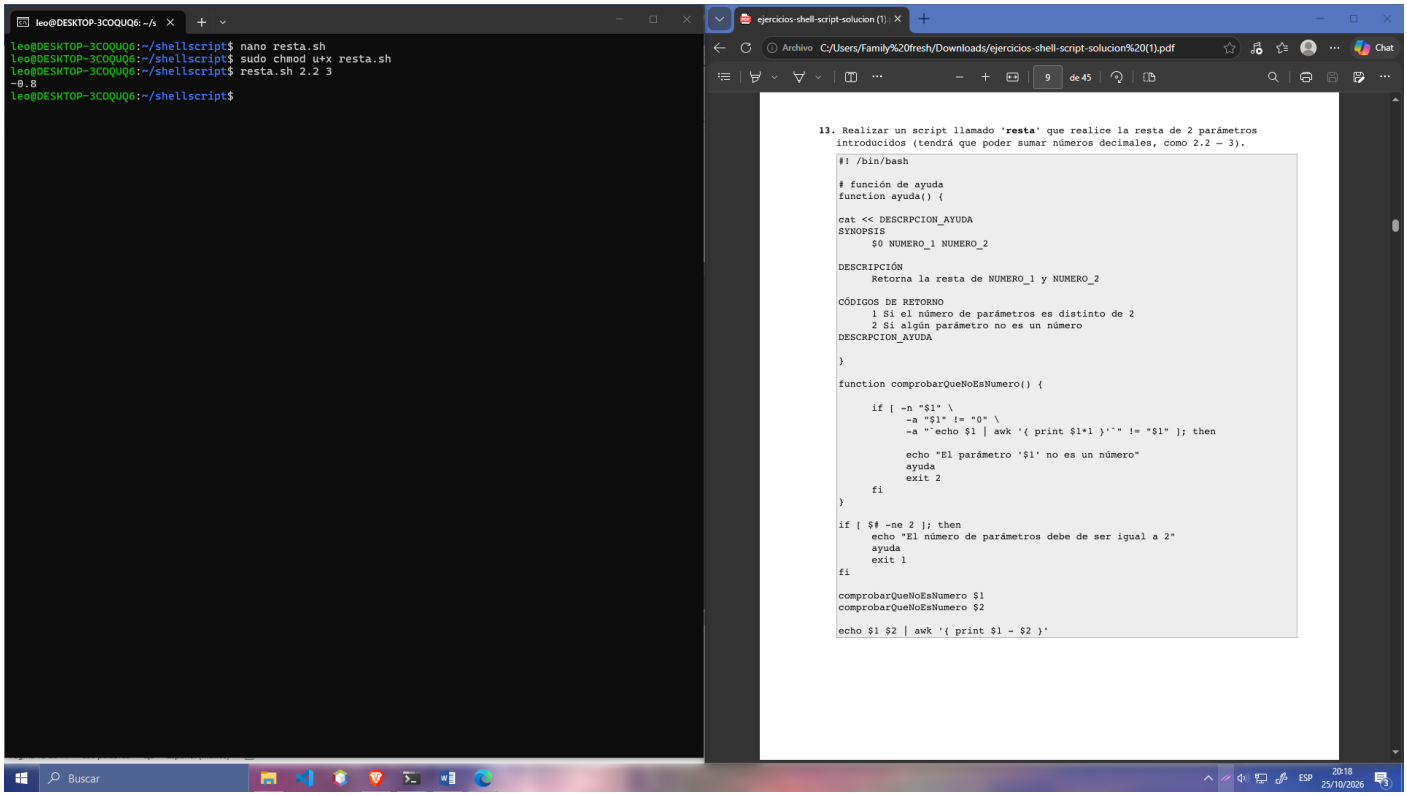


Figura 13.2. Captura original del ejercicio 13 (2 de 2).

Explicación de la salida: La salida corresponde a la resta del segundo número al primero. El programa también revisa que se hayan introducido dos cantidades y avisa cuando los argumentos no son correctos.

Práctica 14. Multiplicación de números

Se implementó la multiplicación de dos valores utilizando awk. La entrada se revisa antes de ejecutar la operación, y se muestra ayuda cuando faltan datos.

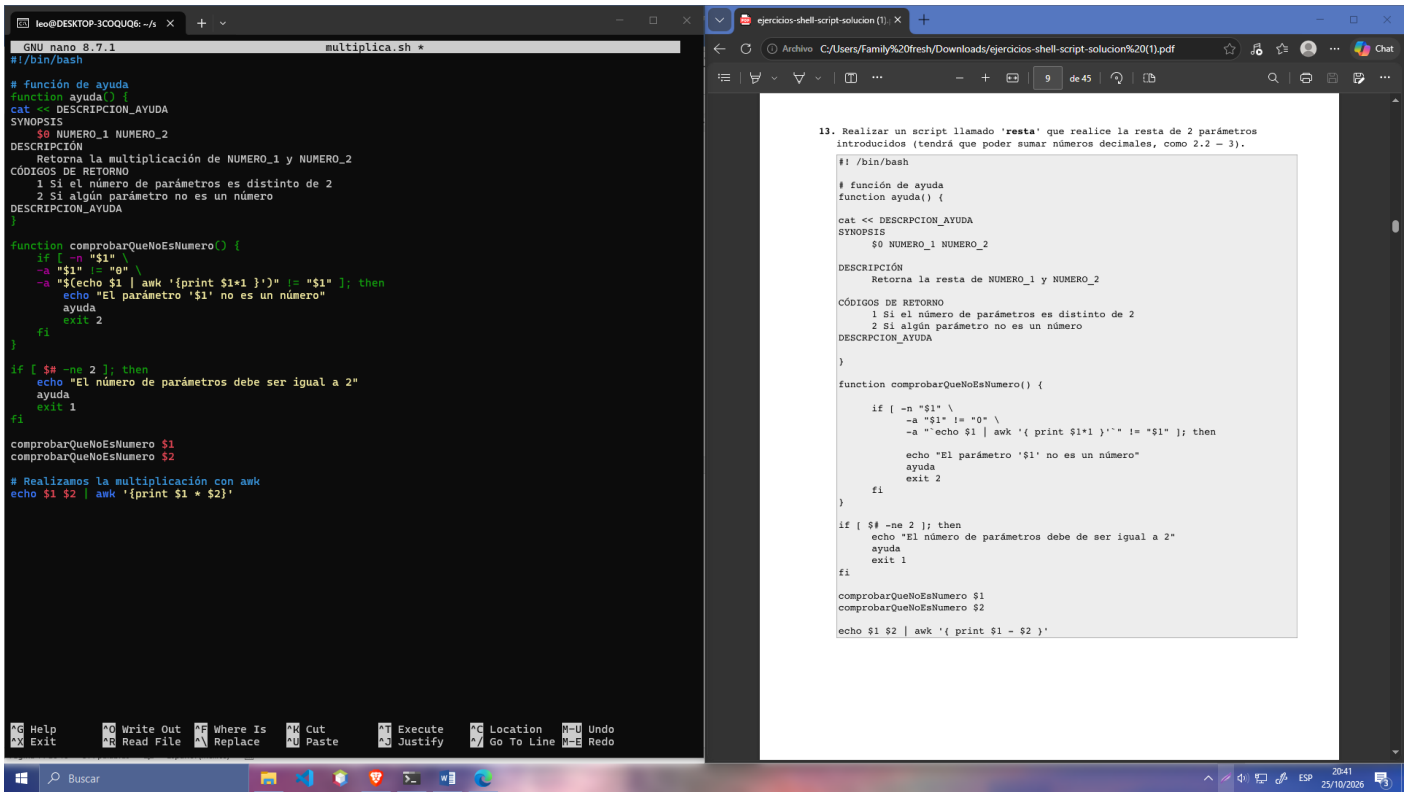


Figura 14.1. Captura original del ejercicio 14 (1 de 2).

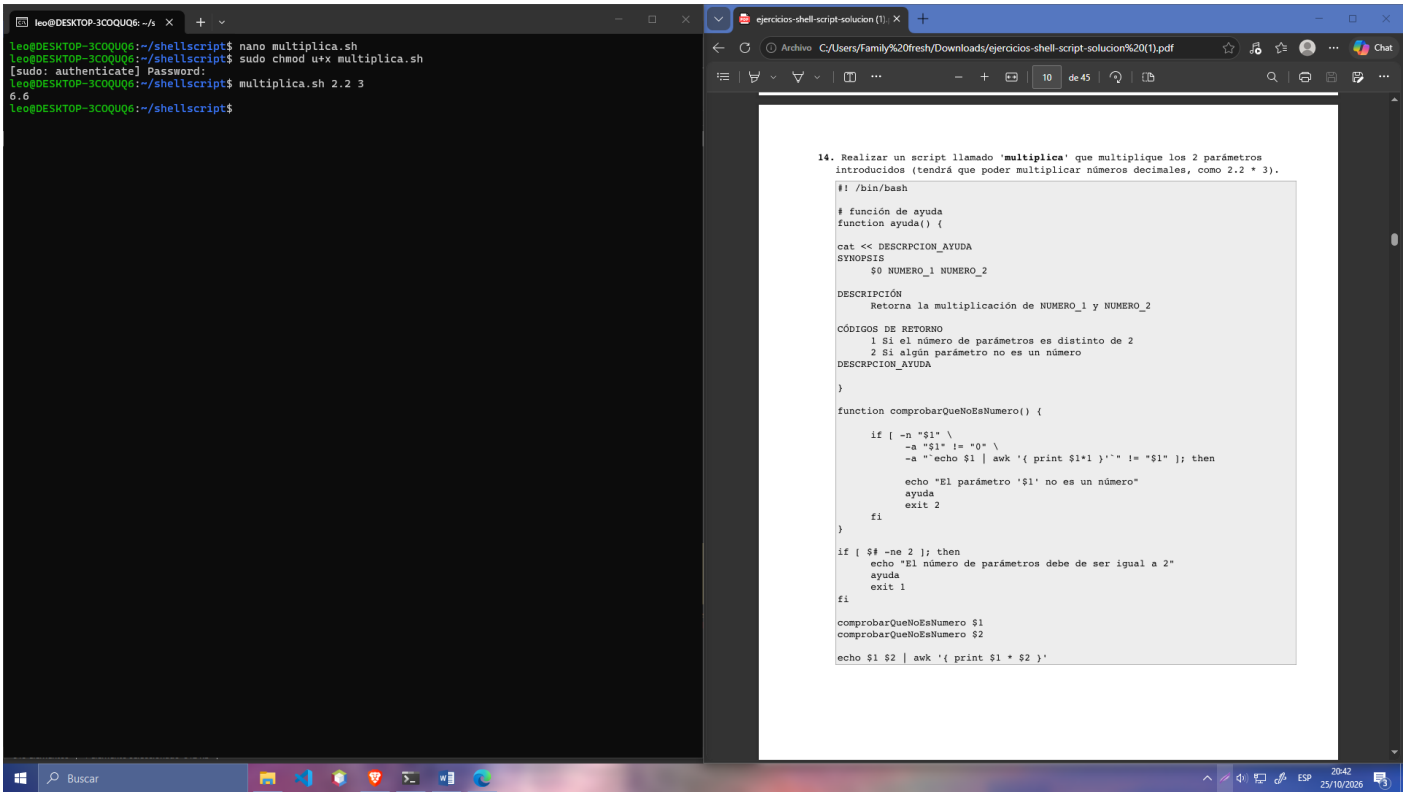


Figura 14.2. Captura original del ejercicio 14 (2 de 2).

Explicación de la salida: En la terminal se presenta el producto de los dos números ingresados. Si se utiliza una cantidad incorrecta de argumentos o un valor no válido, se muestra la ayuda correspondiente.

Práctica 15. División de números

Se trabajó con la operación de división y sus verificaciones, prestando atención a los datos de entrada y al divisor.

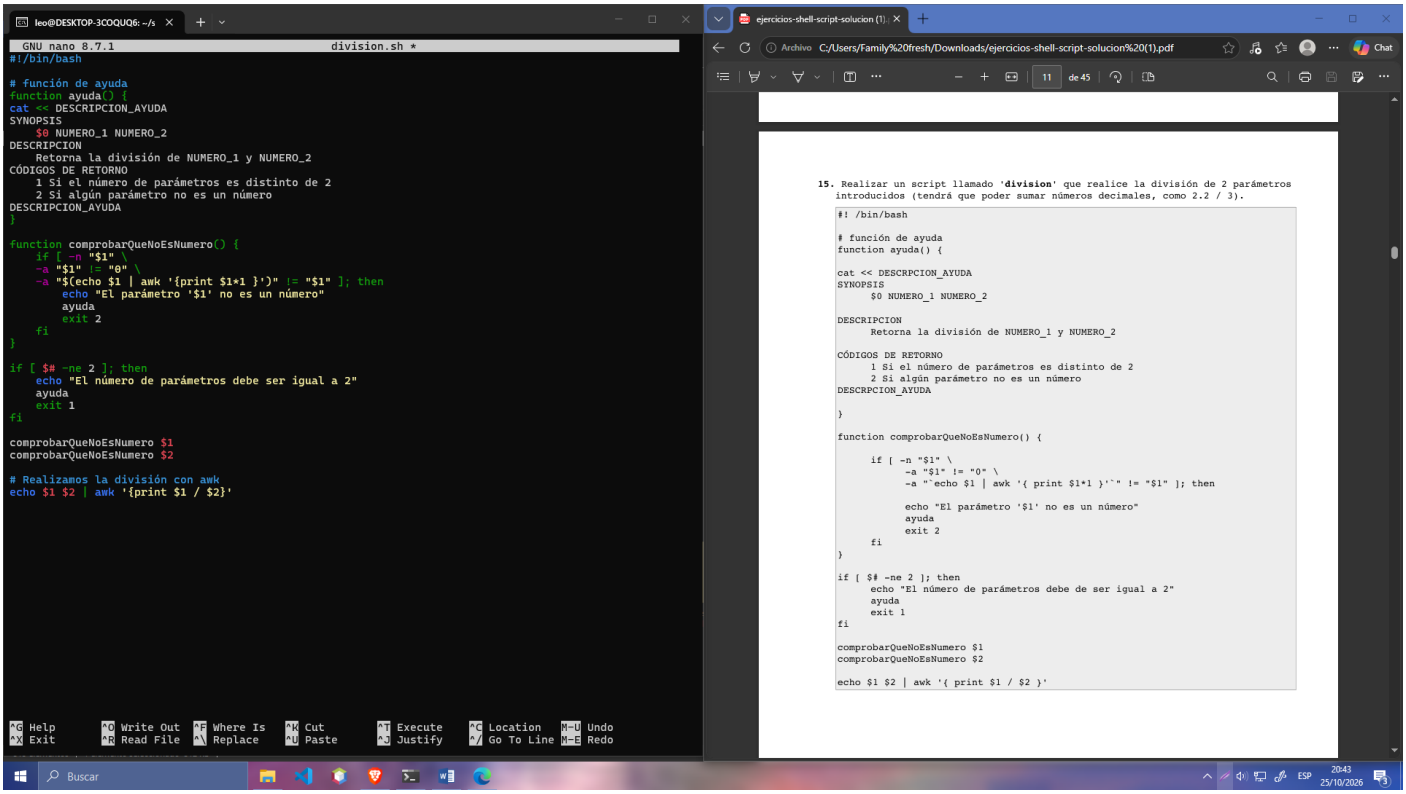


Figura 15.1. Captura original del ejercicio 15 (1 de 2).

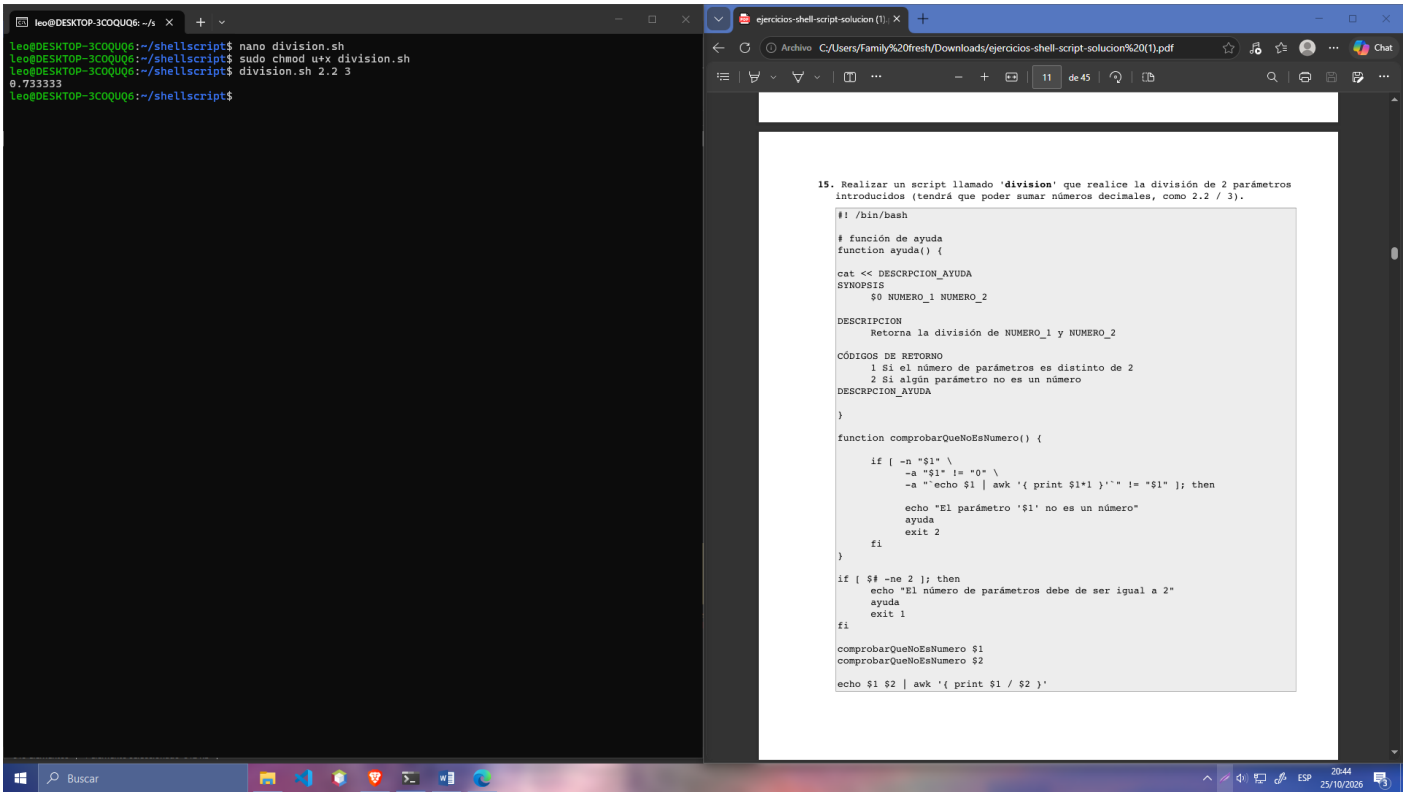


Figura 15.2. Captura original del ejercicio 15 (2 de 2).

Explicación de la salida: El programa muestra el resultado de dividir el primer número entre el segundo. También contempla situaciones en las que los datos no permiten realizar la operación, como una entrada incorrecta o un divisor no permitido.

Práctica 16. Calculadora con operaciones

Se integraron operaciones aritméticas en un script que recibe parámetros y decide qué cálculo ejecutar.

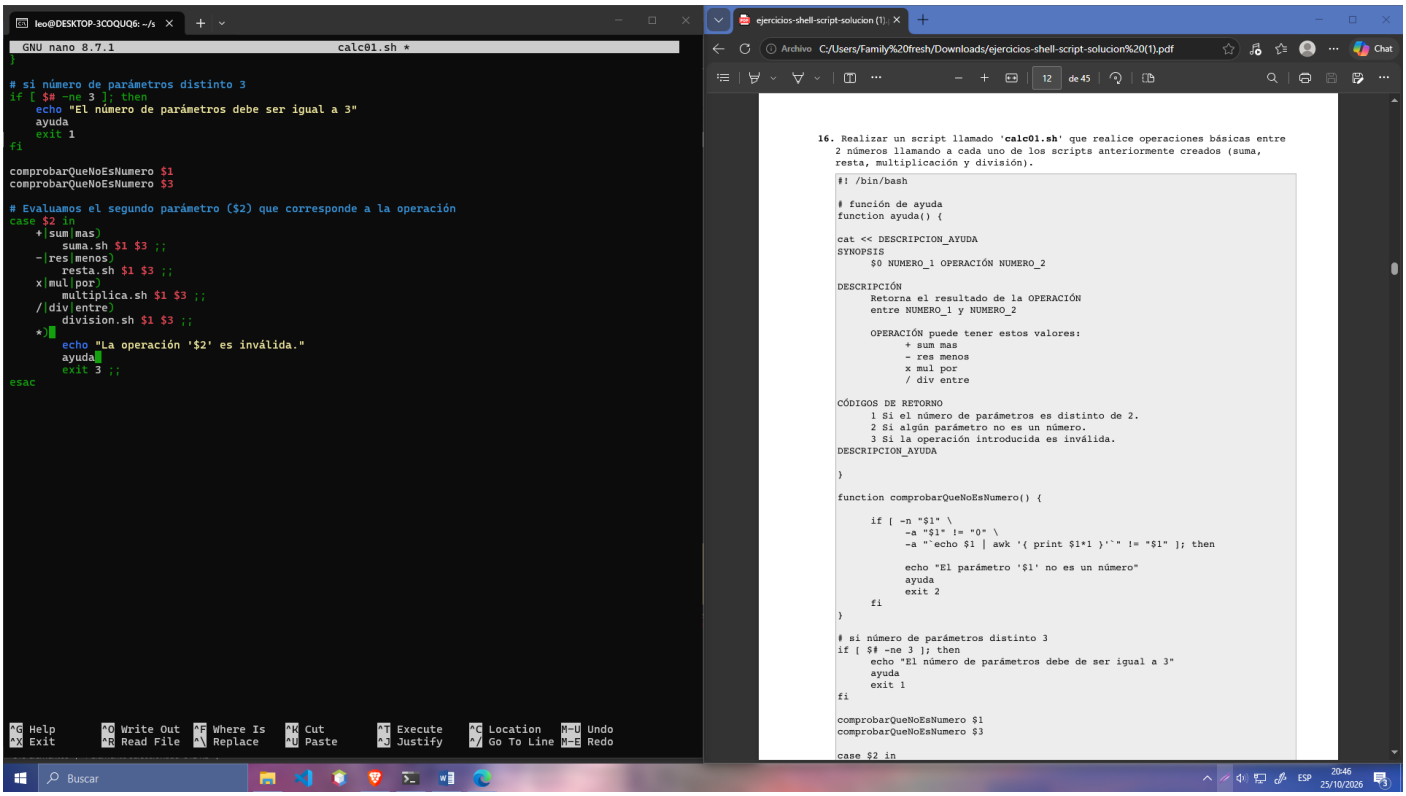


Figura 16.1. Captura original del ejercicio 16 (1 de 2).

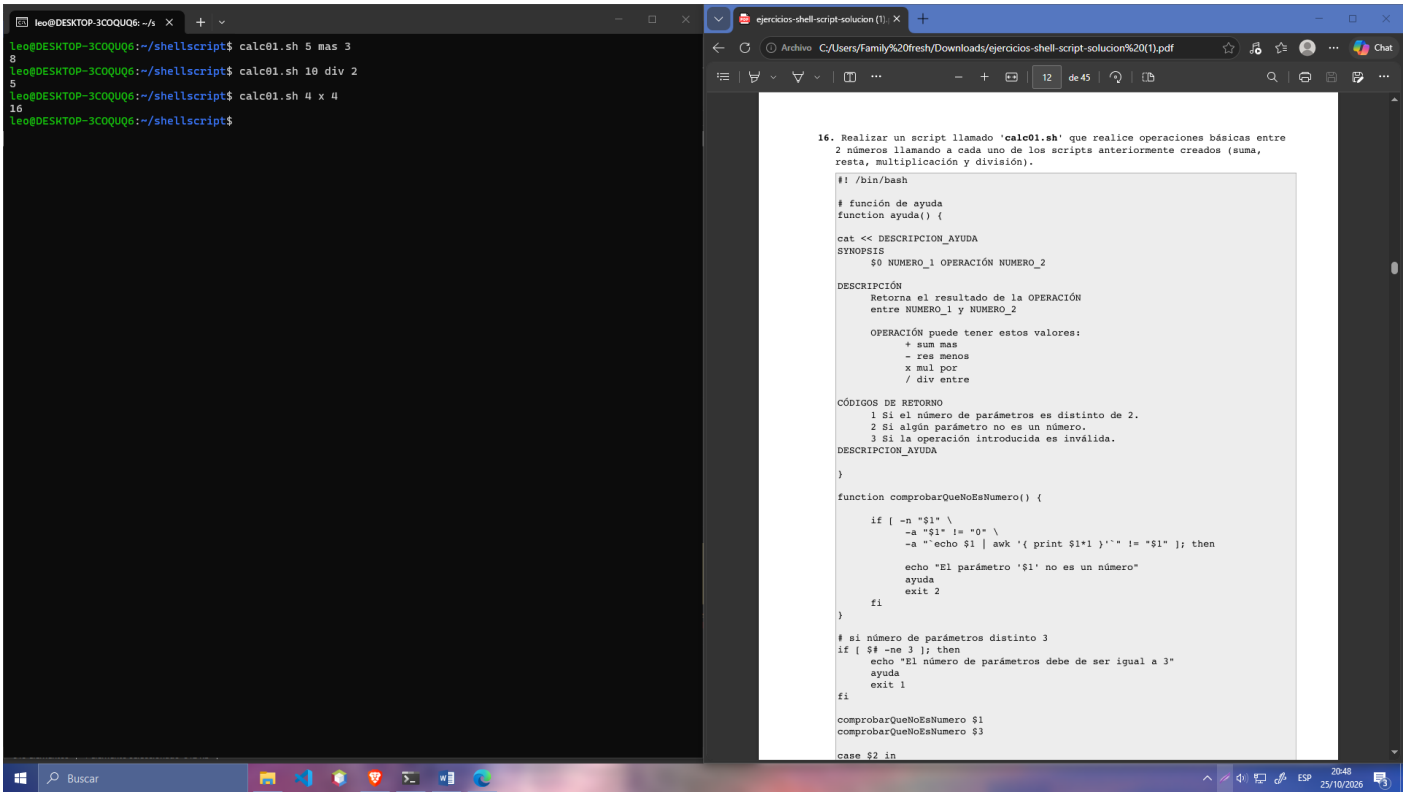


Figura 16.2. Captura original del ejercicio 16 (2 de 2).

Explicación de la salida: La calculadora muestra el resultado de la operación seleccionada a partir de los argumentos. Cuando la operación o las cantidades no cumplen con el formato esperado, informa del error mediante sus mensajes de ayuda.

Práctica 17. Ampliación de la calculadora

Se continuó desarrollando la calculadora a partir de la estructura previa, incorporando el comportamiento indicado para este ejercicio.

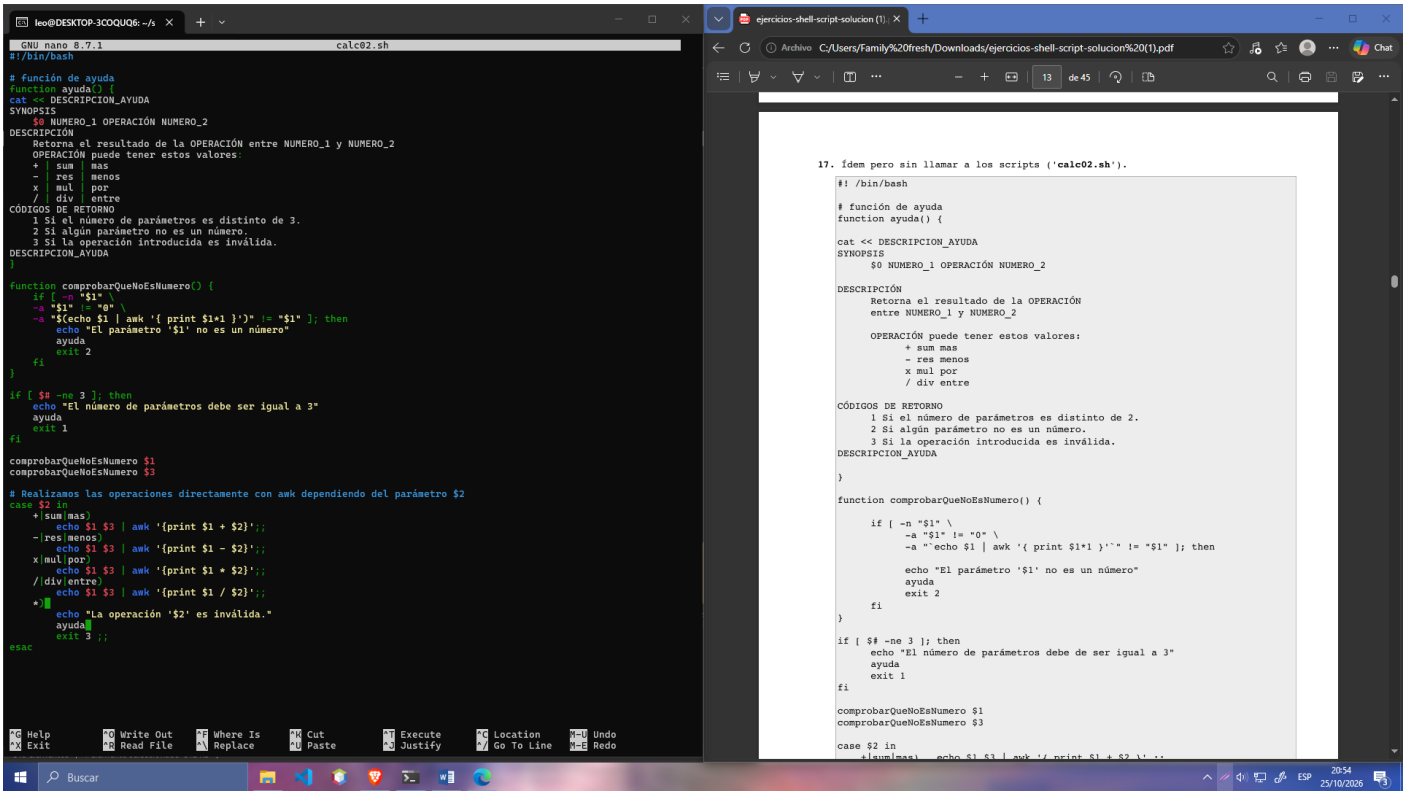


Figura 17.1. Captura original del ejercicio 17 (1 de 2).

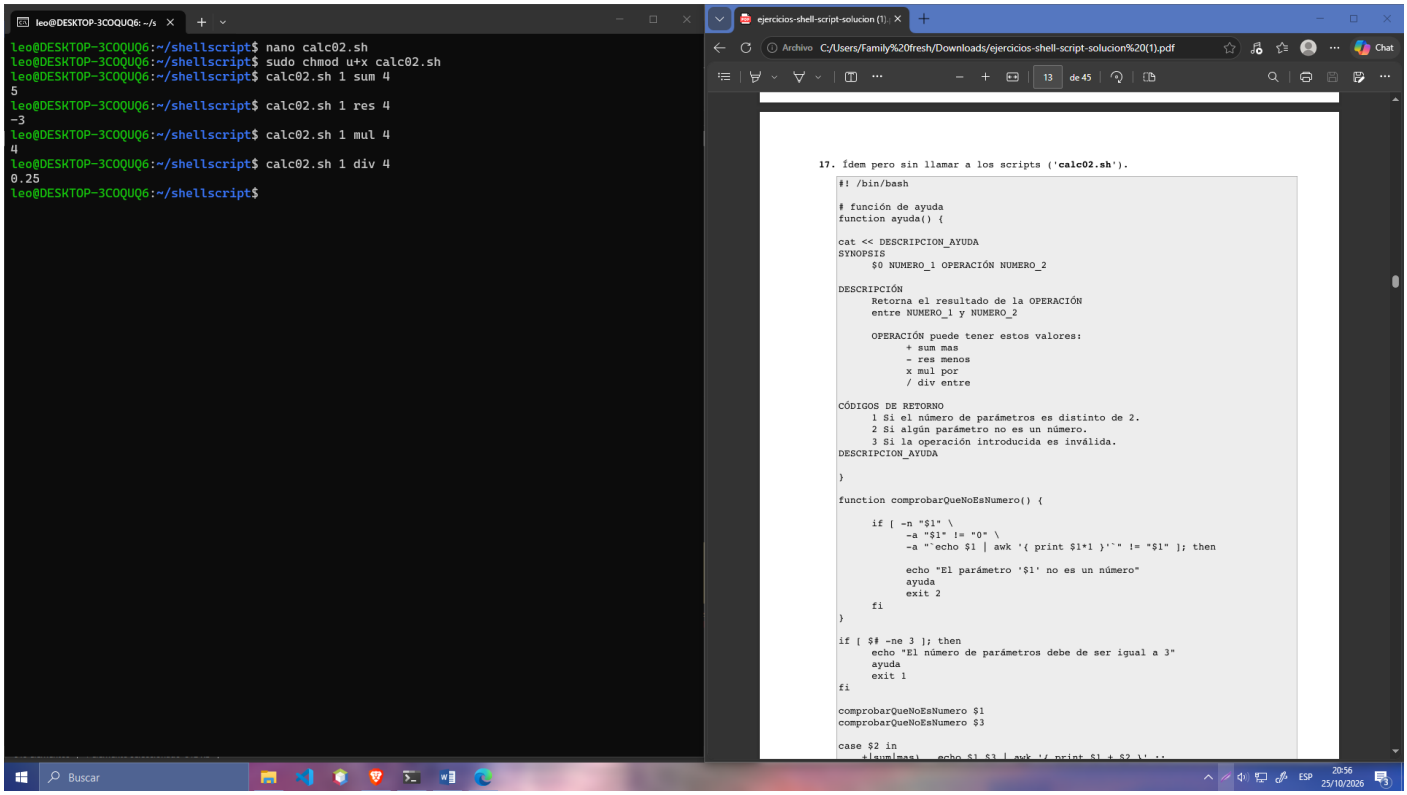


Figura 17.2. Captura original del ejercicio 17 (2 de 2).

Explicación de la salida: La salida depende de la operación indicada al utilizar esta versión de la calculadora. El programa evalúa los parámetros antes de presentar el cálculo y muestra avisos cuando encuentra datos incorrectos.

Práctica 18. Última modificación de la calculadora

Se revisó la variante de la calculadora incluida en el archivo de prácticas, con sus controles de parámetros y su operación principal.

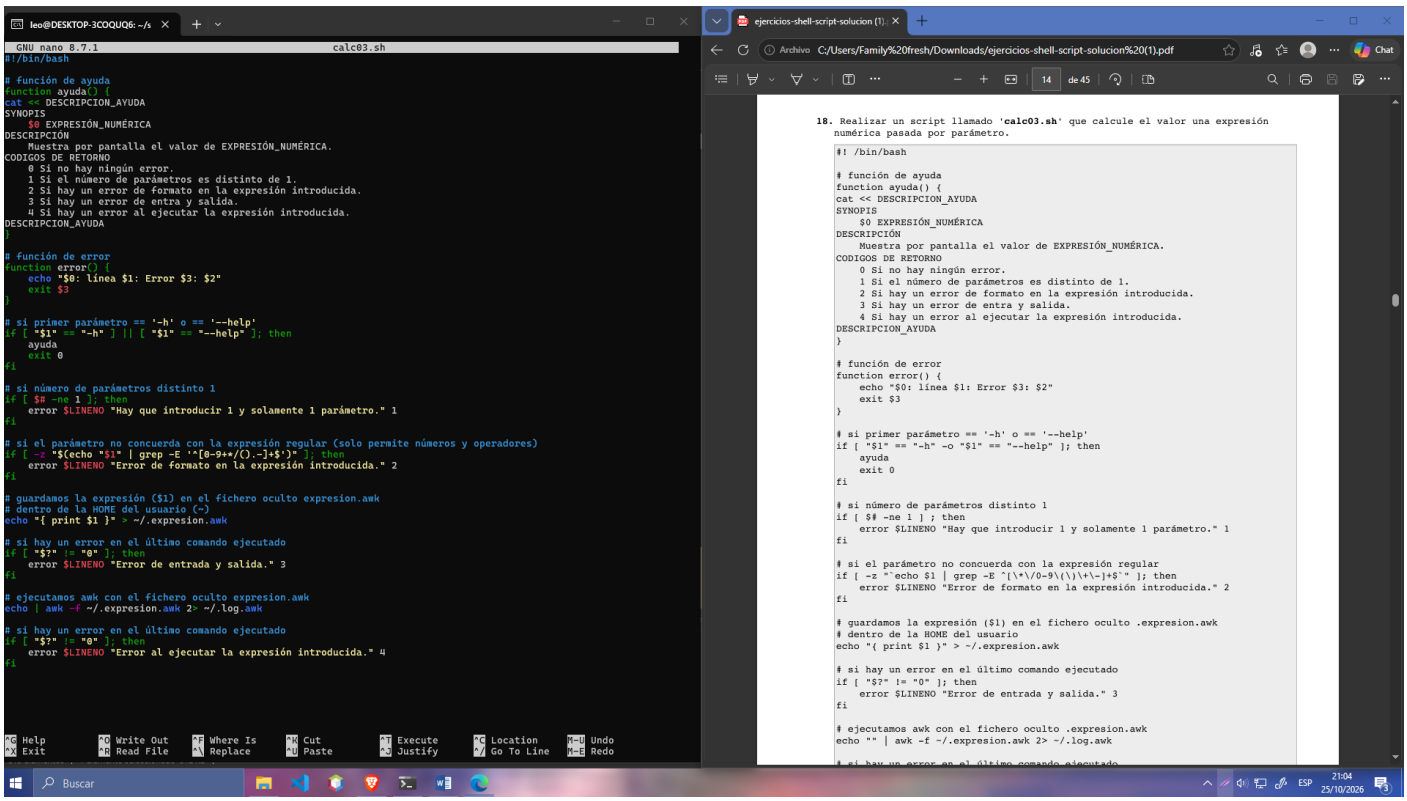


Figura 18.1. Captura original del ejercicio 18 (1 de 2).

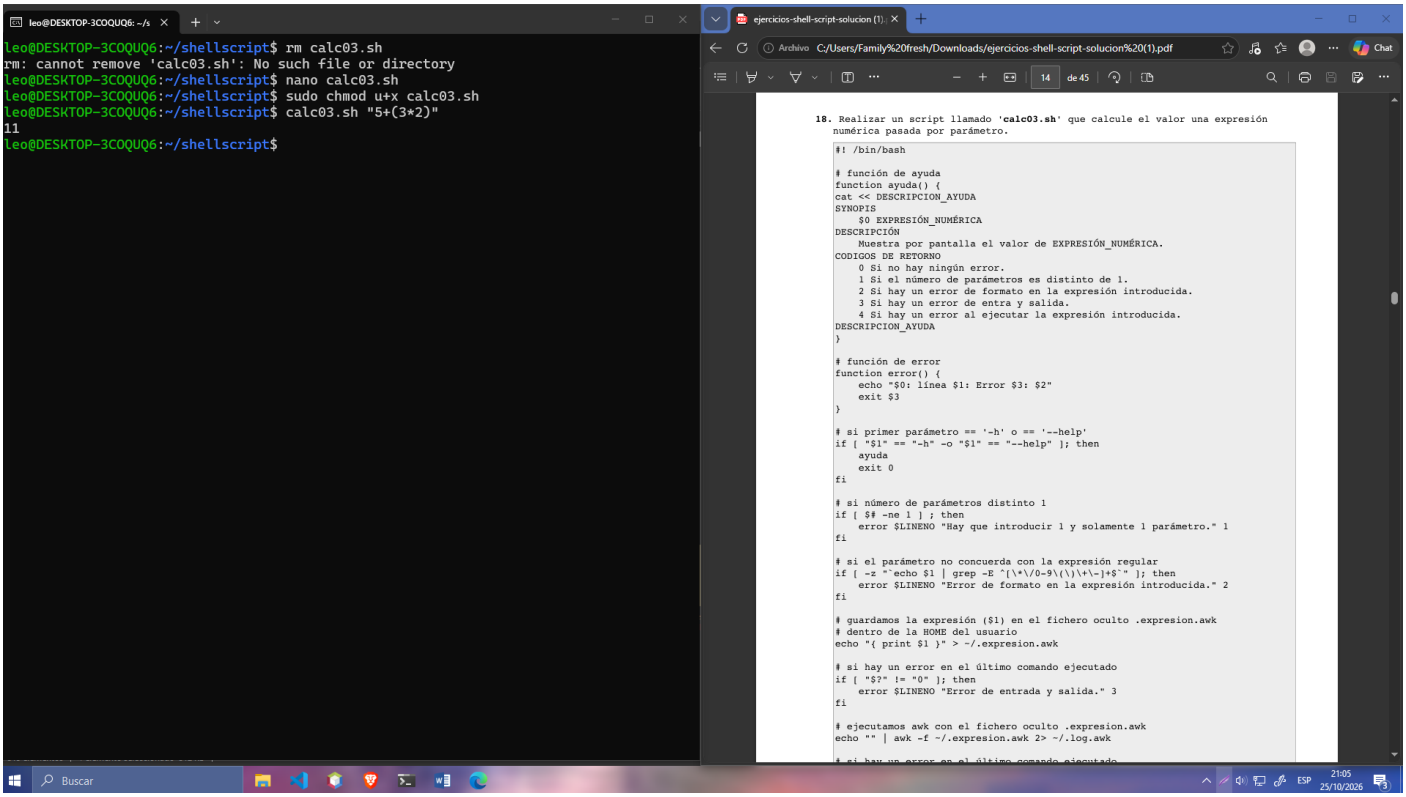


Figura 18.2. Captura original del ejercicio 18 (2 de 2).

Explicación de la salida: Esta versión responde a los argumentos ingresados y muestra el resultado cuando la operación es válida. También puede presentar información de ayuda o errores más específicos cuando se invoca con opciones o valores no admitidos.

Práctica 19. Archivo de notas de alumnos

Se prepararon datos de ejemplo con nombres y tres calificaciones por alumno para después procesarlos con AWK.

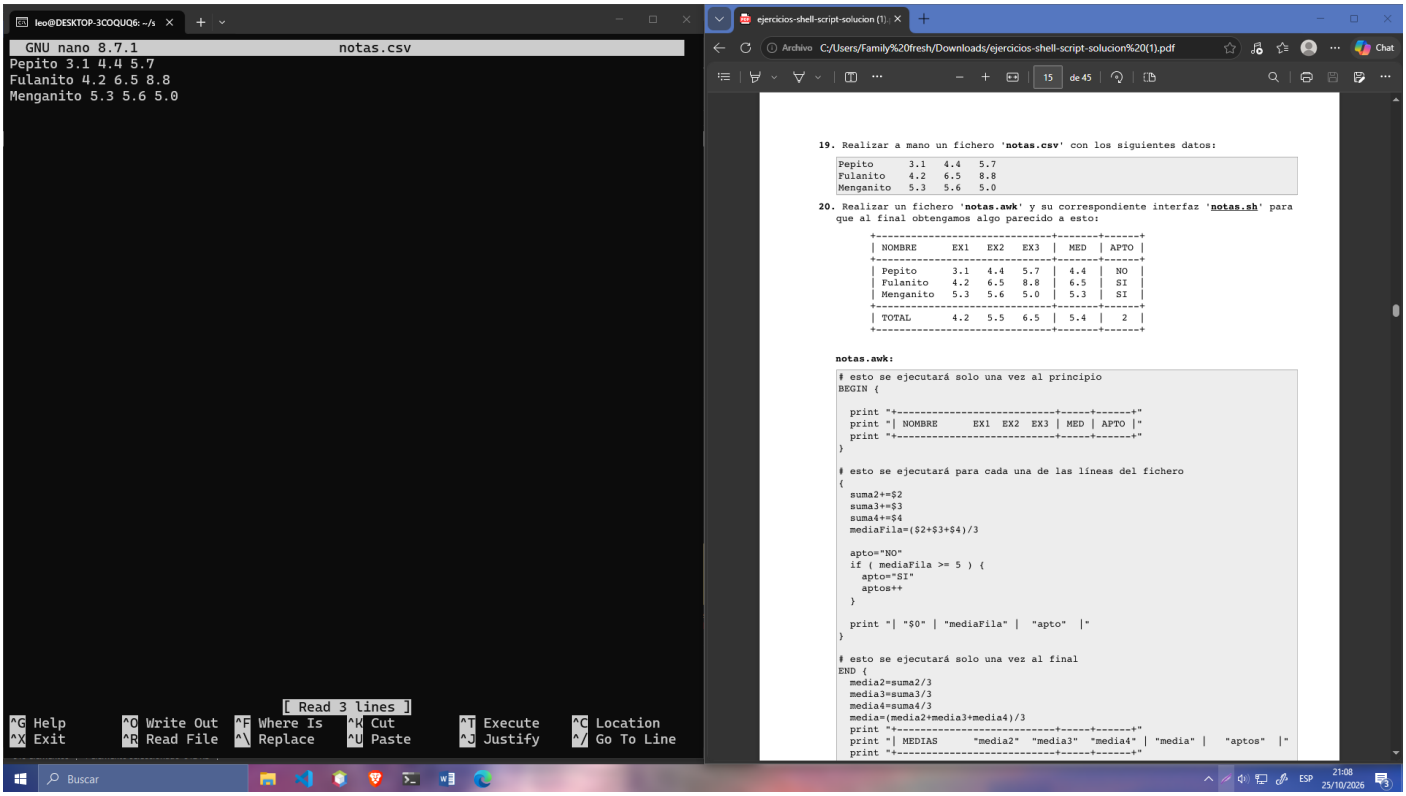


Figura 19.1. Captura original del ejercicio 19 (1 de 1).

Explicación de la salida: Este ejercicio prepara un archivo de datos con nombres de alumnos y sus calificaciones. Al mostrar su contenido en la terminal se observan los registros que después se utilizan para realizar los cálculos con AWK.

Práctica 20. Procesamiento de notas con AWK

Se creó un archivo de instrucciones de AWK para leer las notas y presentar la información en un formato de salida. También se preparó un pequeño script de Bash para ejecutarlo.

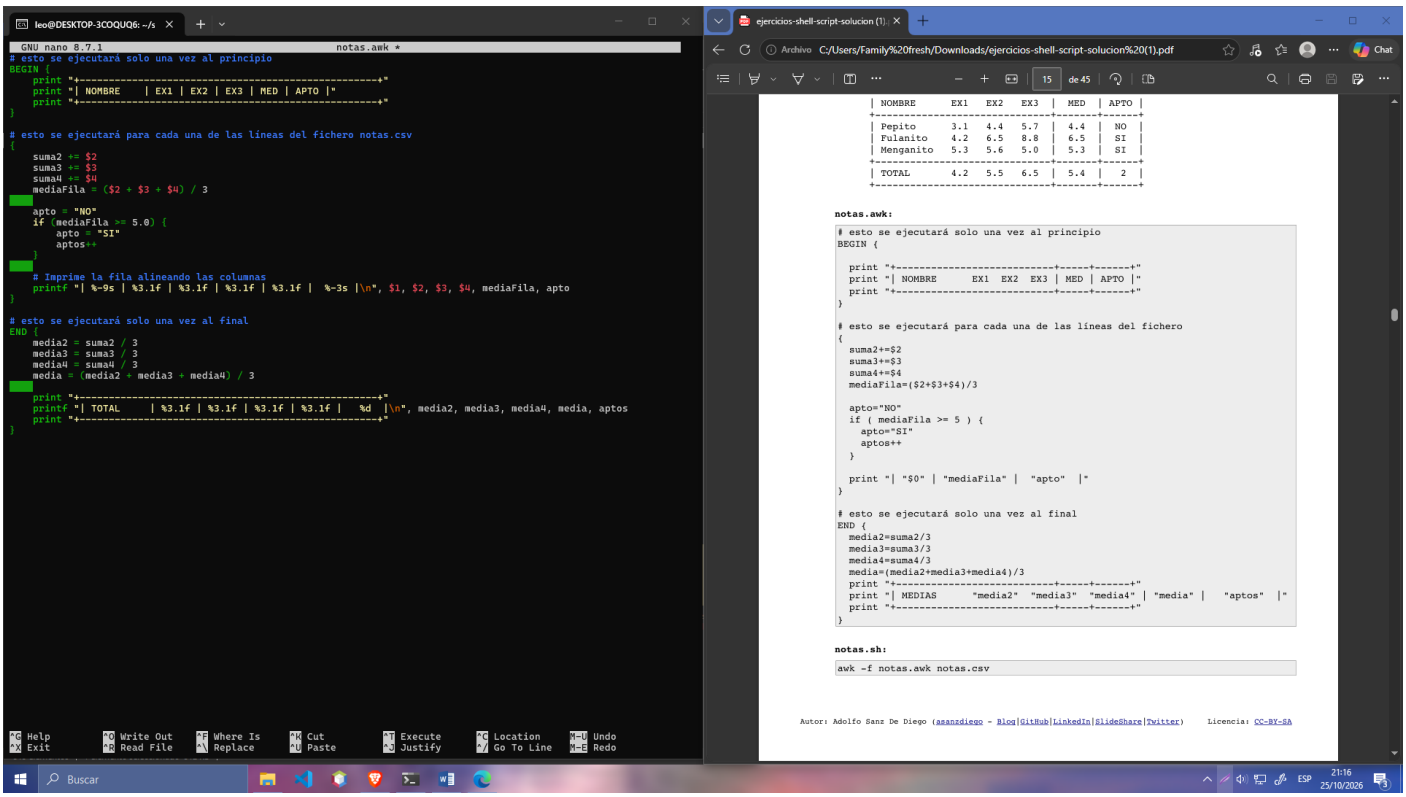


Figura 20.1. Captura original del ejercicio 20 (1 de 3).

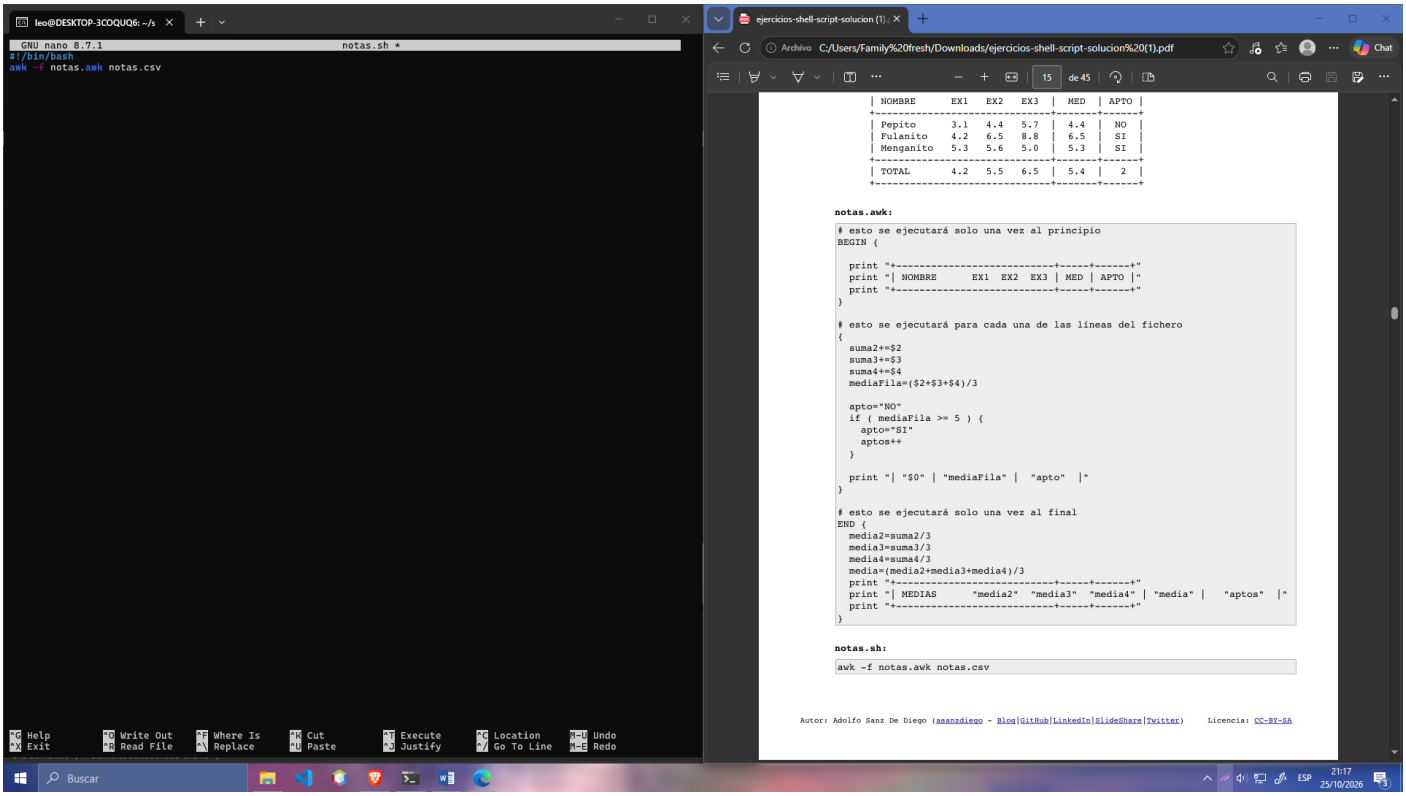


Figura 20.2. Captura original del ejercicio 20 (2 de 3).

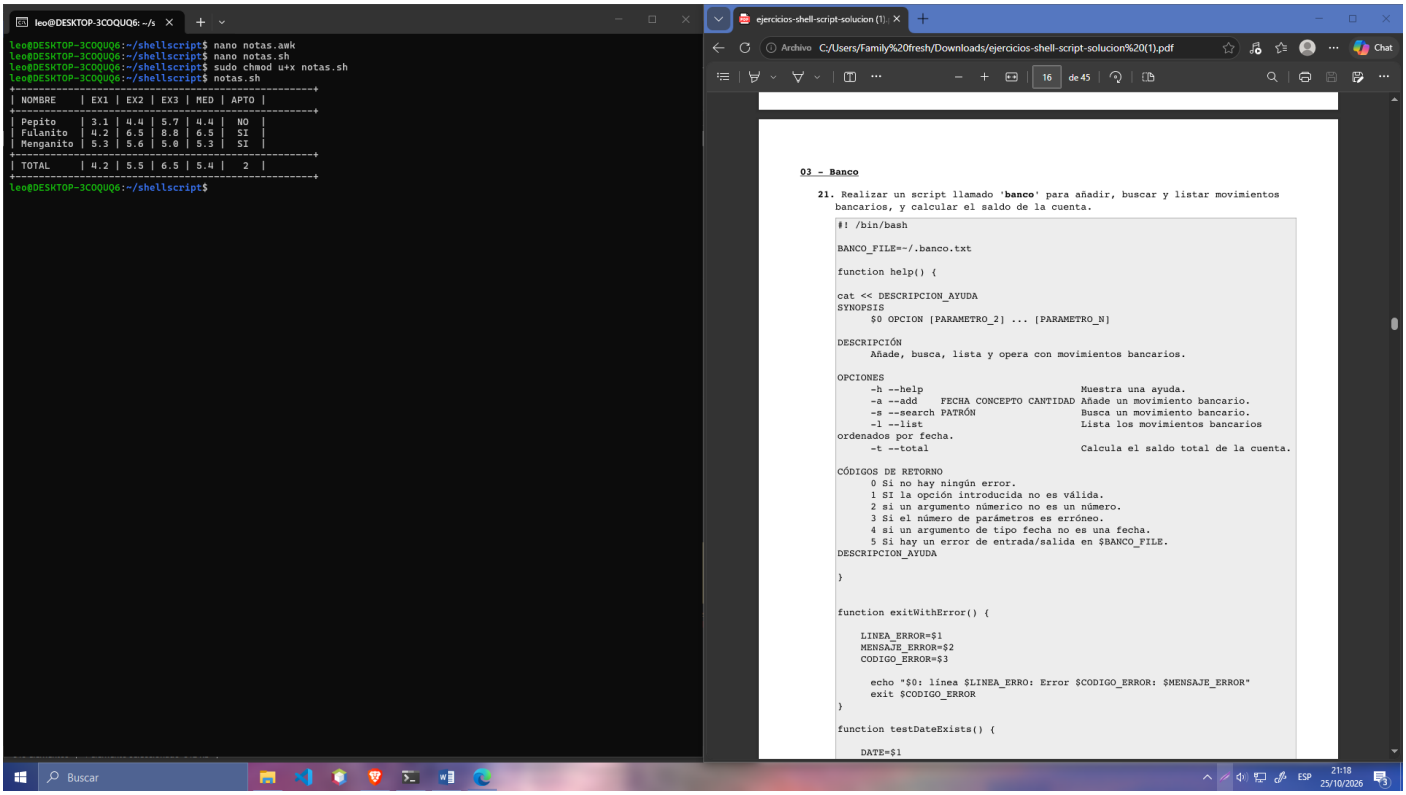


Figura 20.3. Captura original del ejercicio 20 (3 de 3).

Explicación de la salida: Al procesar el archivo de notas, AWK lee los registros y presenta los datos calculados según las instrucciones del programa. La salida permite consultar las calificaciones sin modificarlas manualmente una por una.

Práctica 21. Registro de movimientos bancarios

Se trabajó en un script que guarda información en un archivo y permite realizar operaciones como añadir, buscar o listar movimientos. El código incluye funciones y un menú para organizar las acciones.

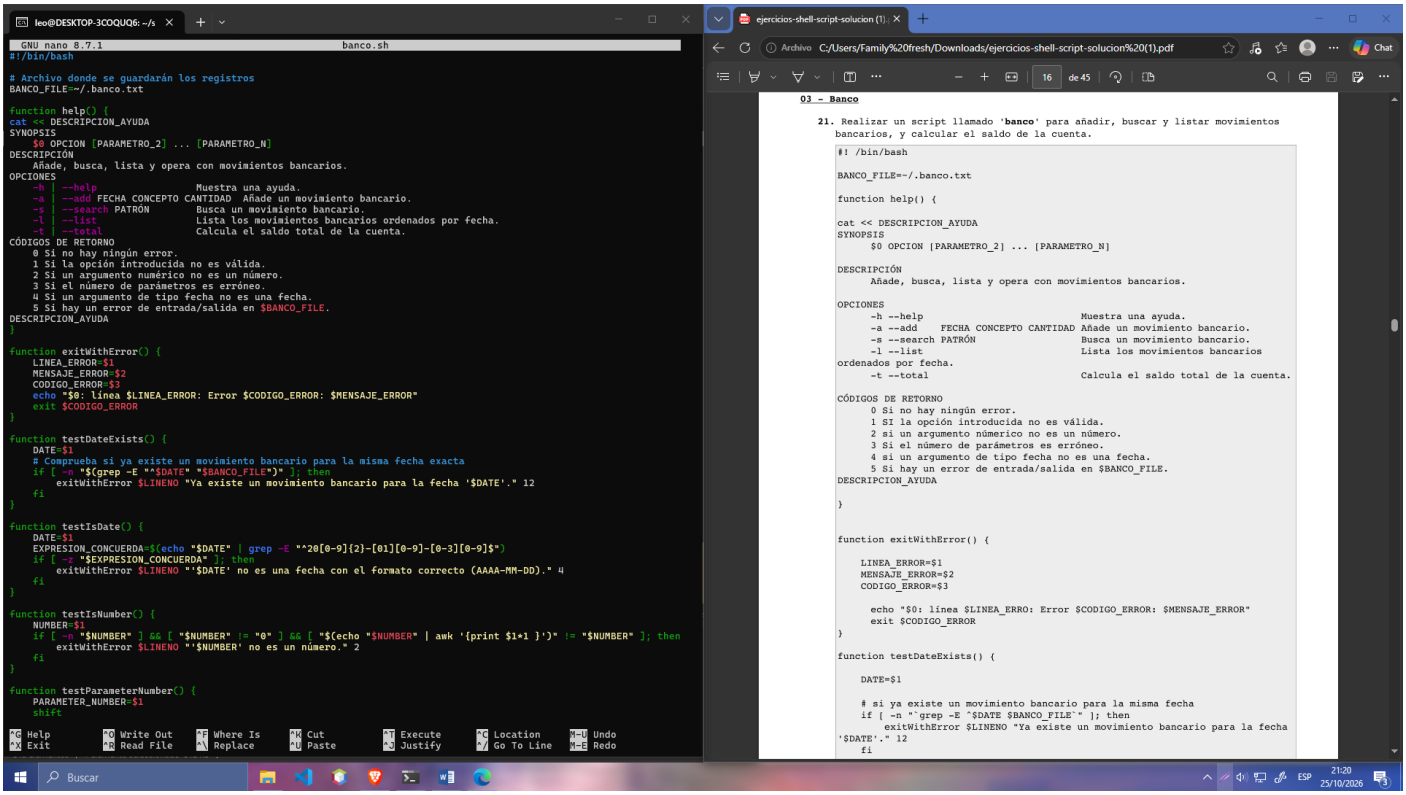


Figura 21.1. Captura original del ejercicio 21 (1 de 4).

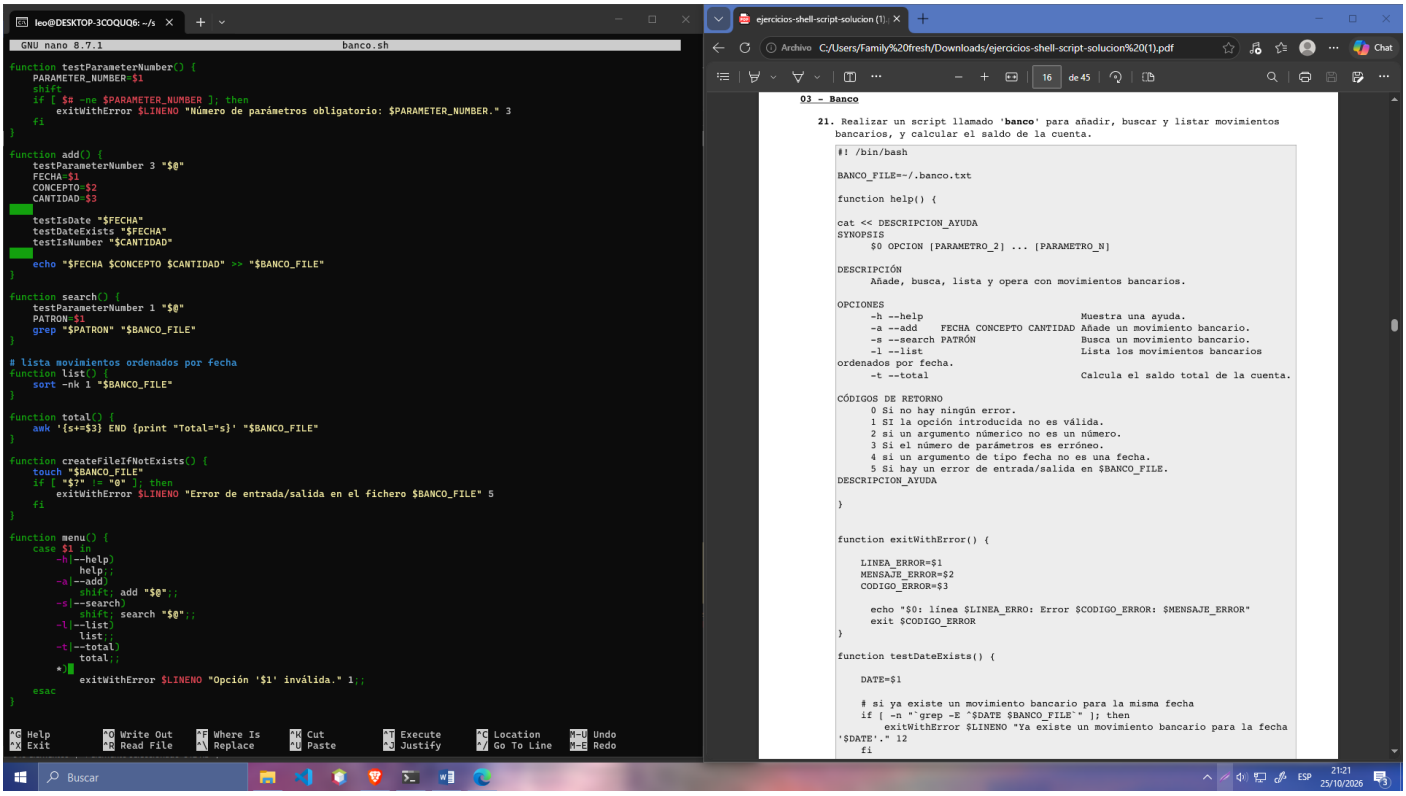


Figura 21.2. Captura original del ejercicio 21 (2 de 4).

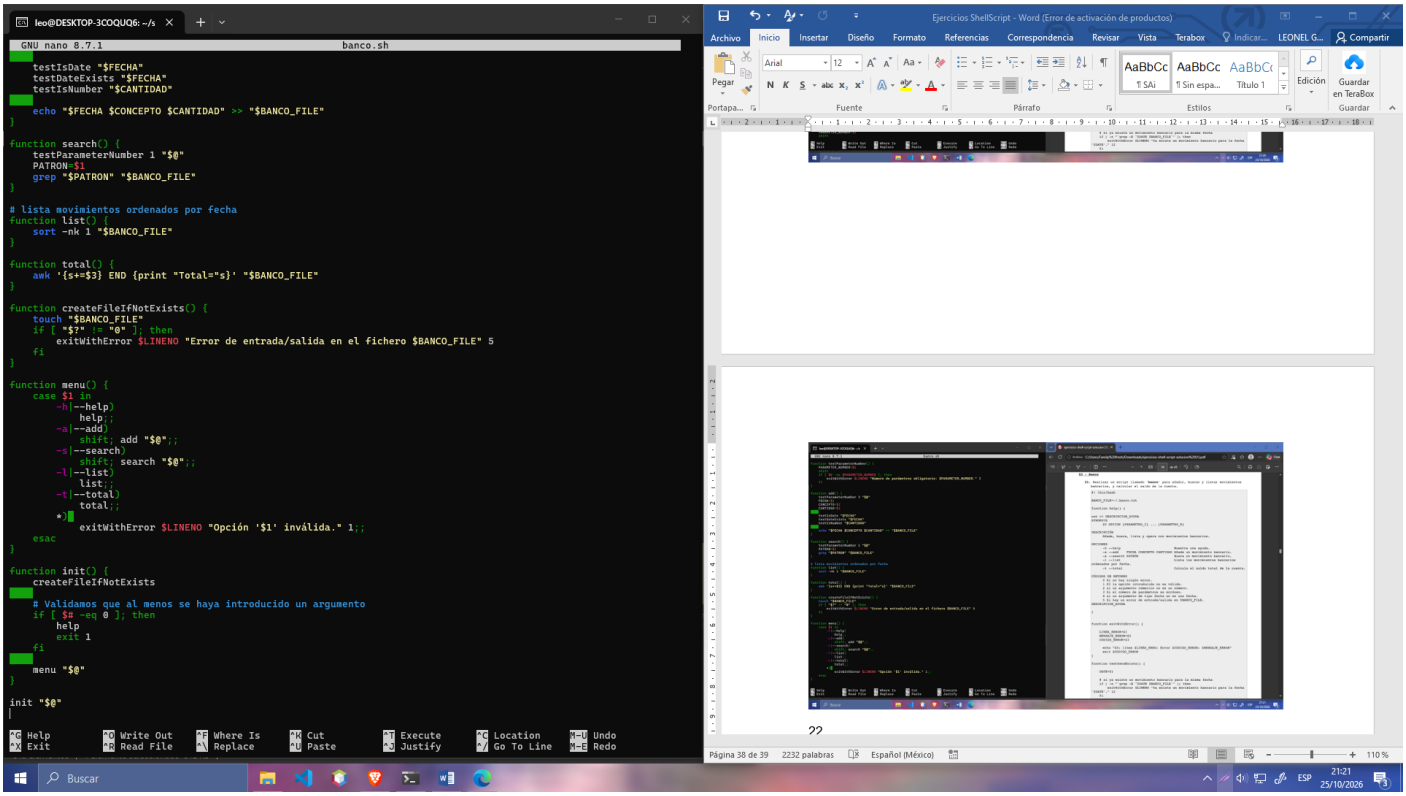


Figura 21.3. Captura original del ejercicio 21 (3 de 4).

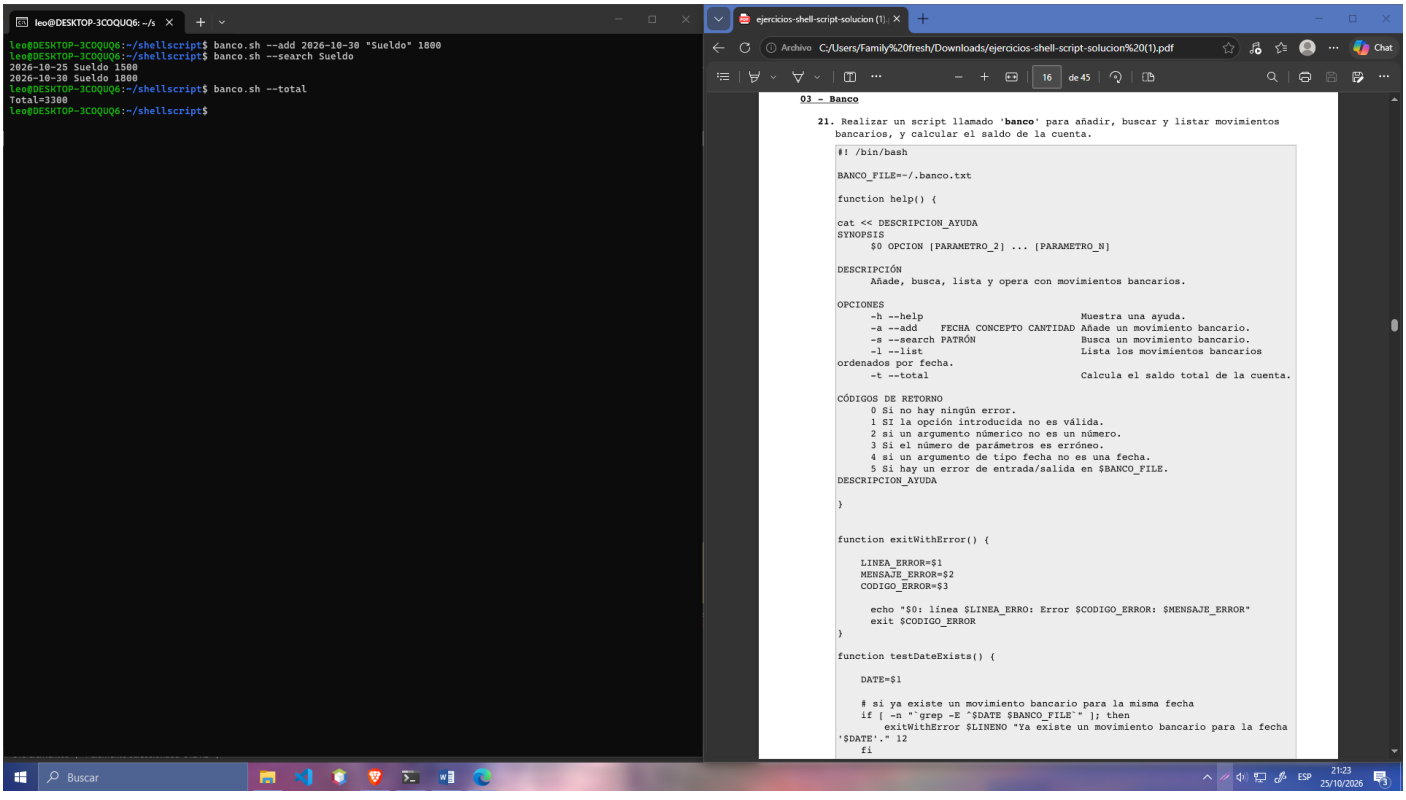


Figura 21.4. Captura original del ejercicio 21 (4 de 4).

Explicación de la salida: El programa permite registrar o consultar operaciones bancarias desde la terminal. Según la opción seleccionada, muestra información de los movimientos y avisa si encuentra datos incompletos o una operación que no puede realizar.

Práctica 22. Interfaz de menú para el banco

Se creó una interfaz que llama al script del banco y facilita seleccionar operaciones mediante un menú.

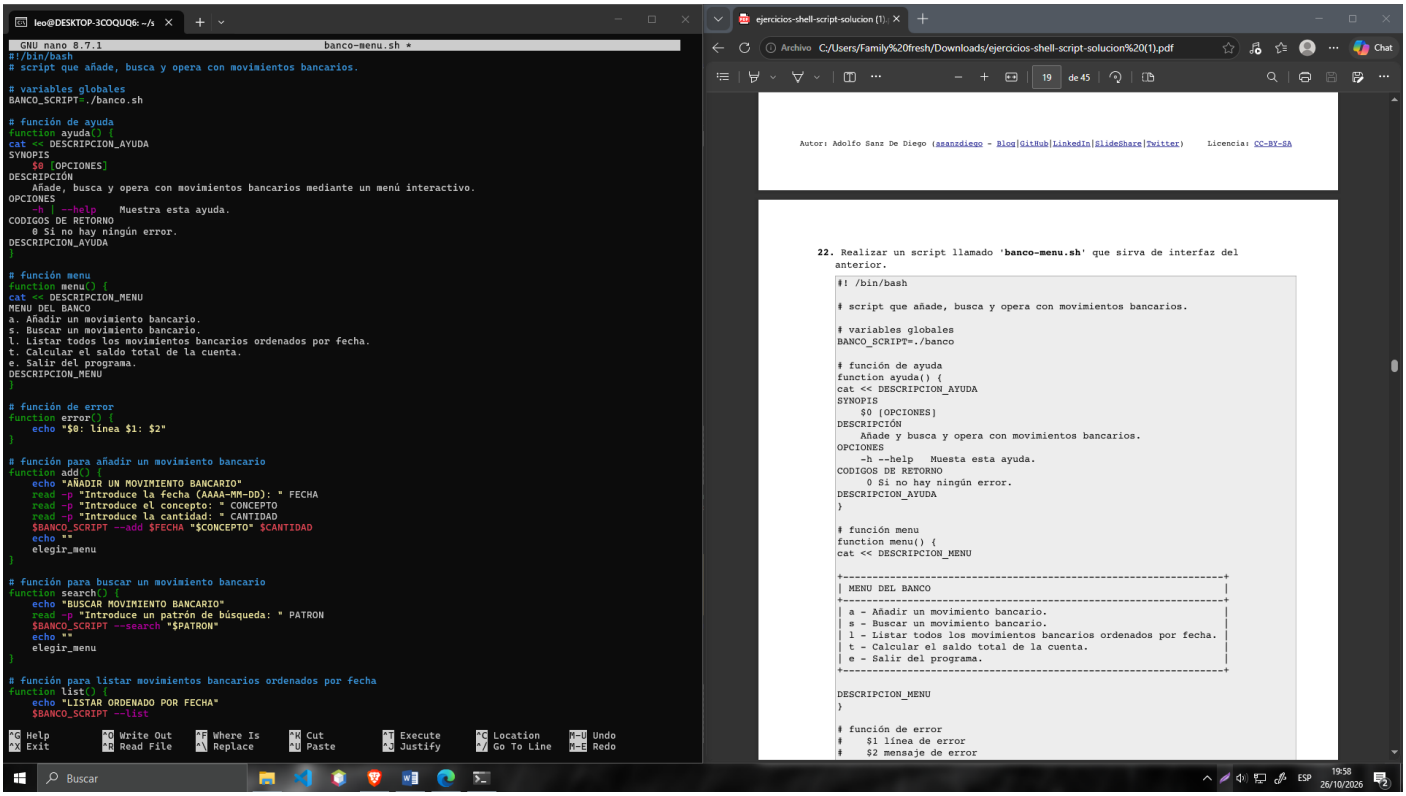


Figura 22.1. Captura original del ejercicio 22 (1 de 7).

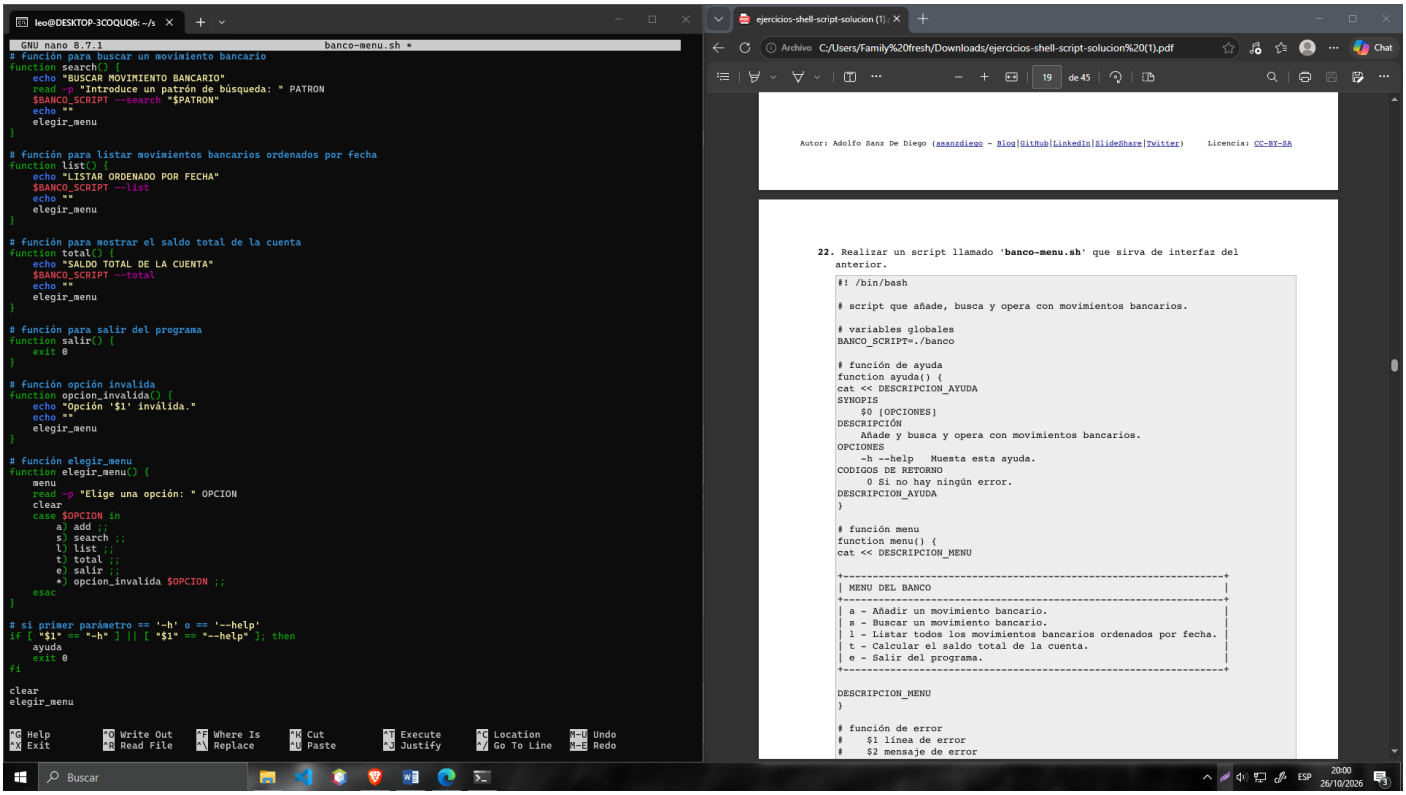


Figura 22.2. Captura original del ejercicio 22 (2 de 7).

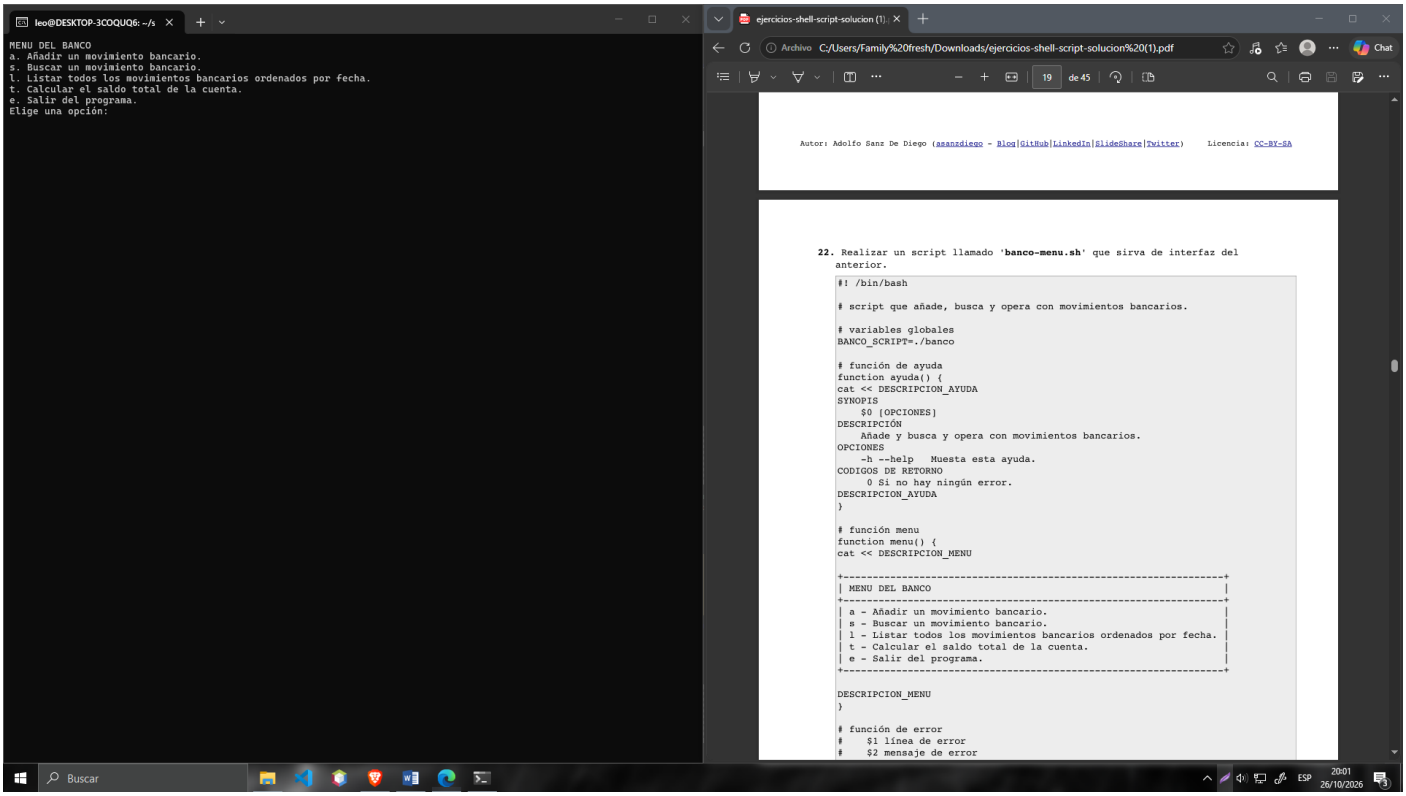


Figura 22.3. Captura original del ejercicio 22 (3 de 7).

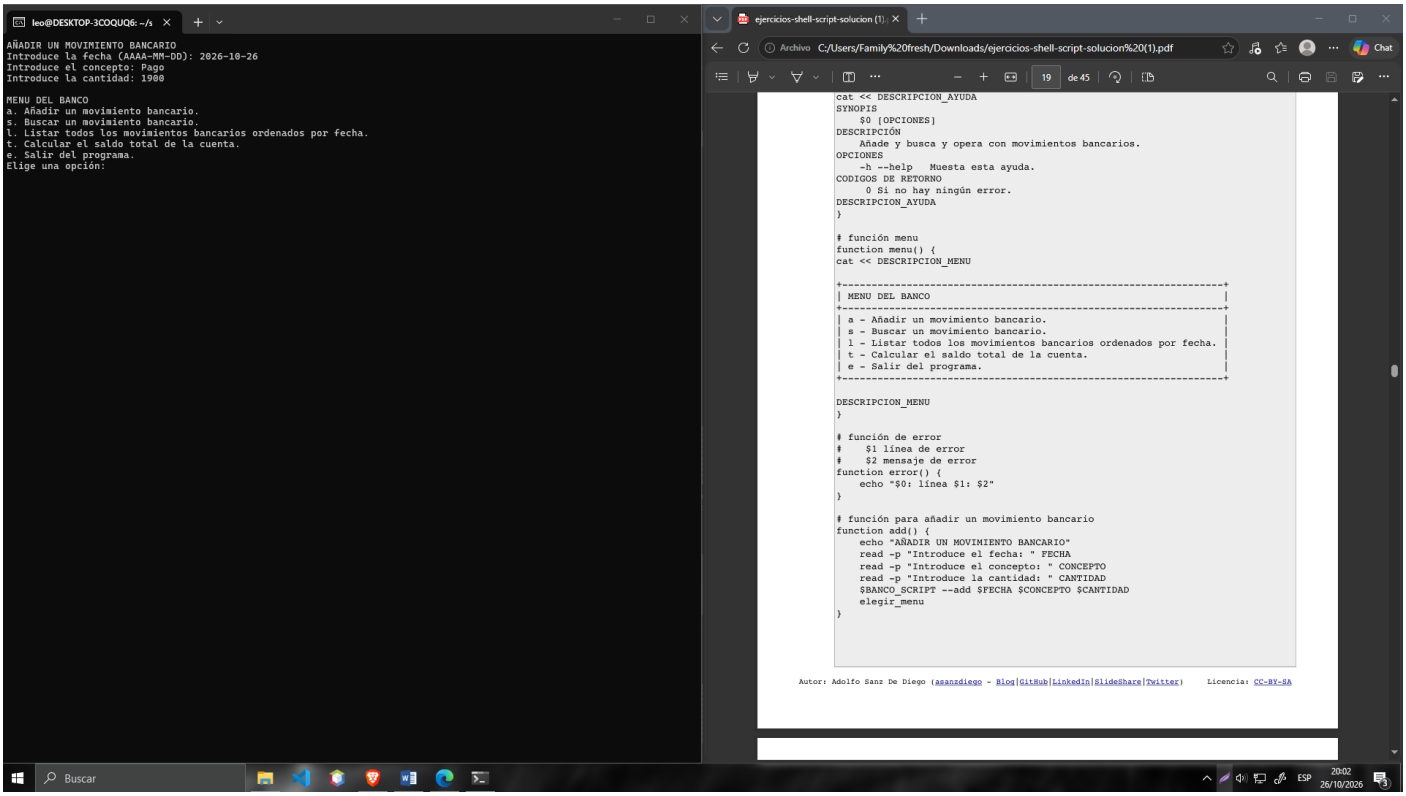


Figura 22.4. Captura original del ejercicio 22 (4 de 7).

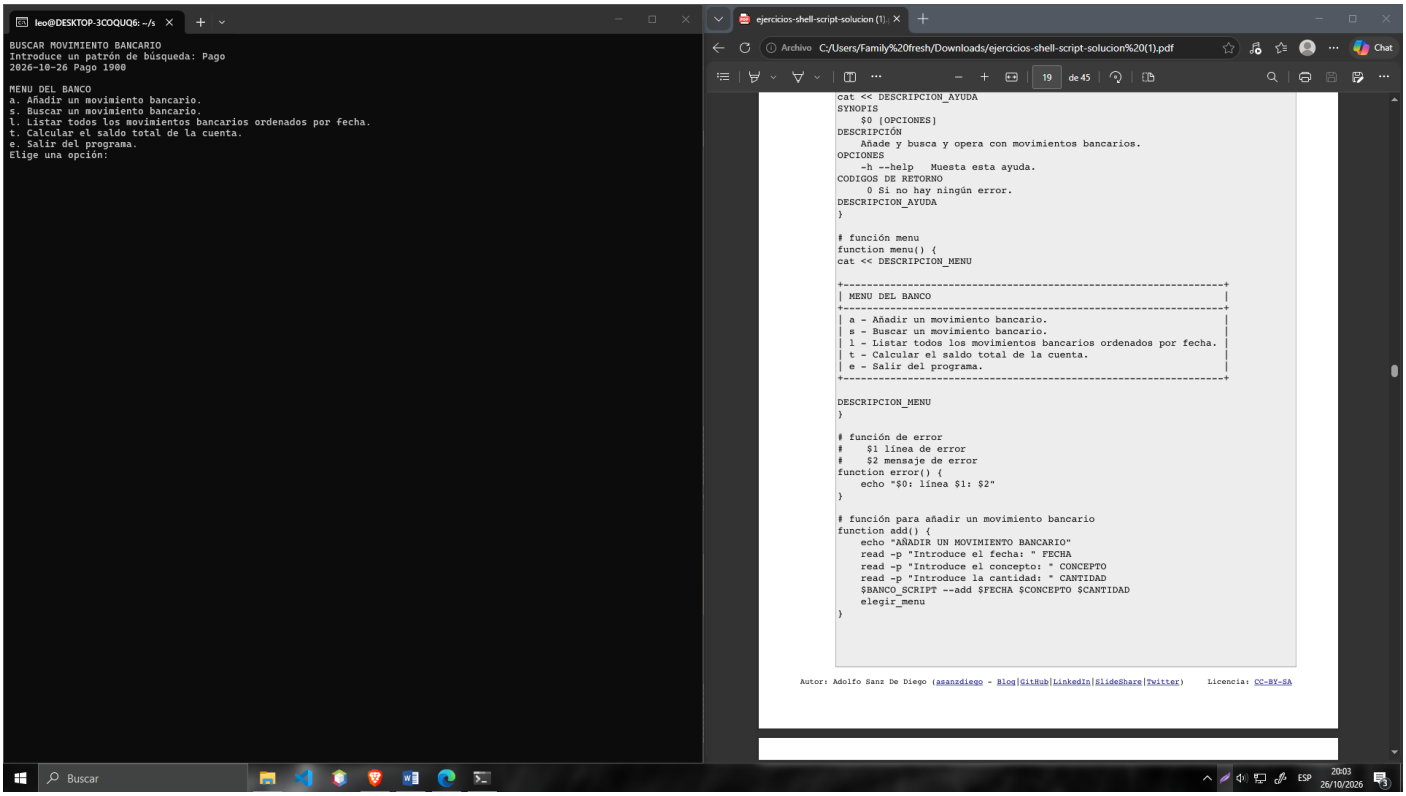


Figura 22.5. Captura original del ejercicio 22 (5 de 7).

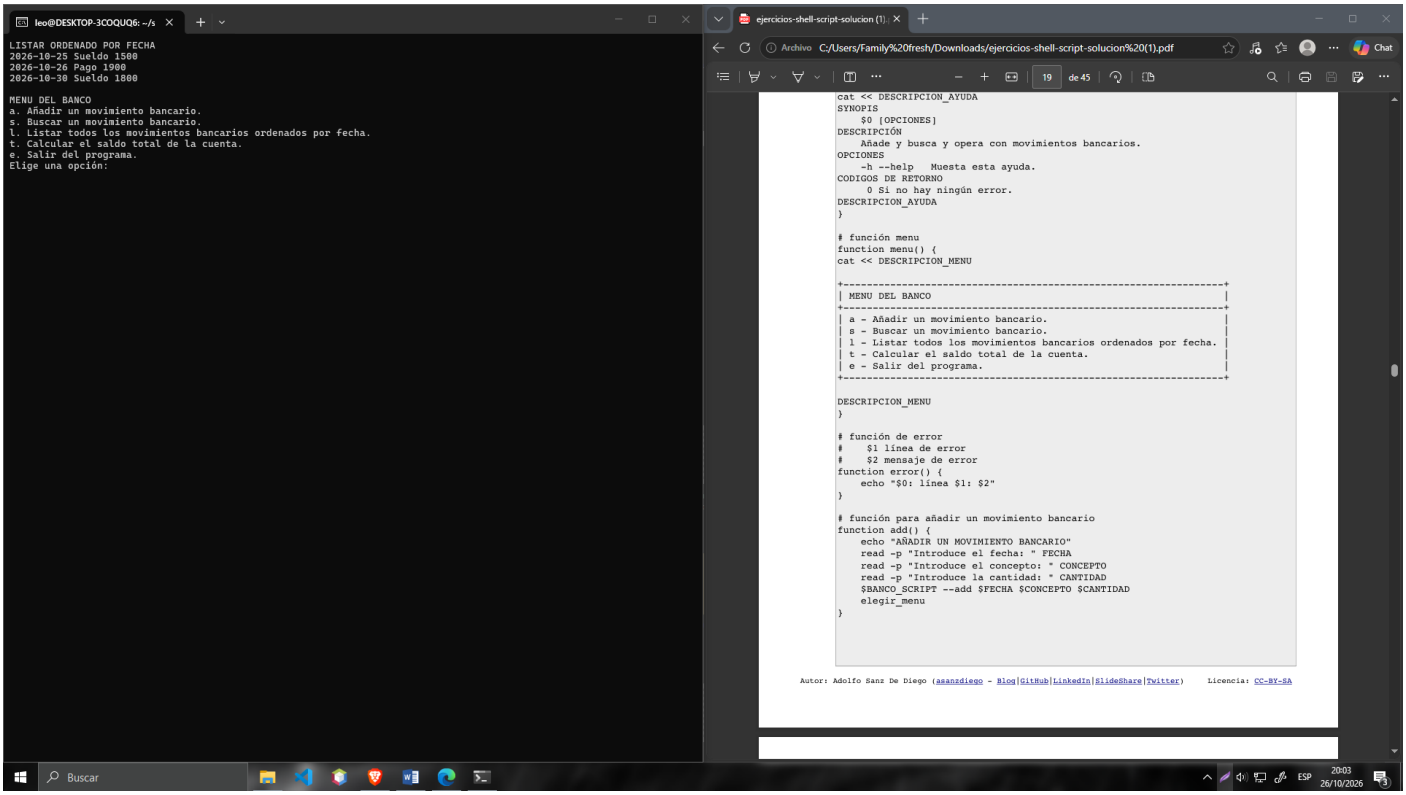


Figura 22.6. Captura original del ejercicio 22 (6 de 7).

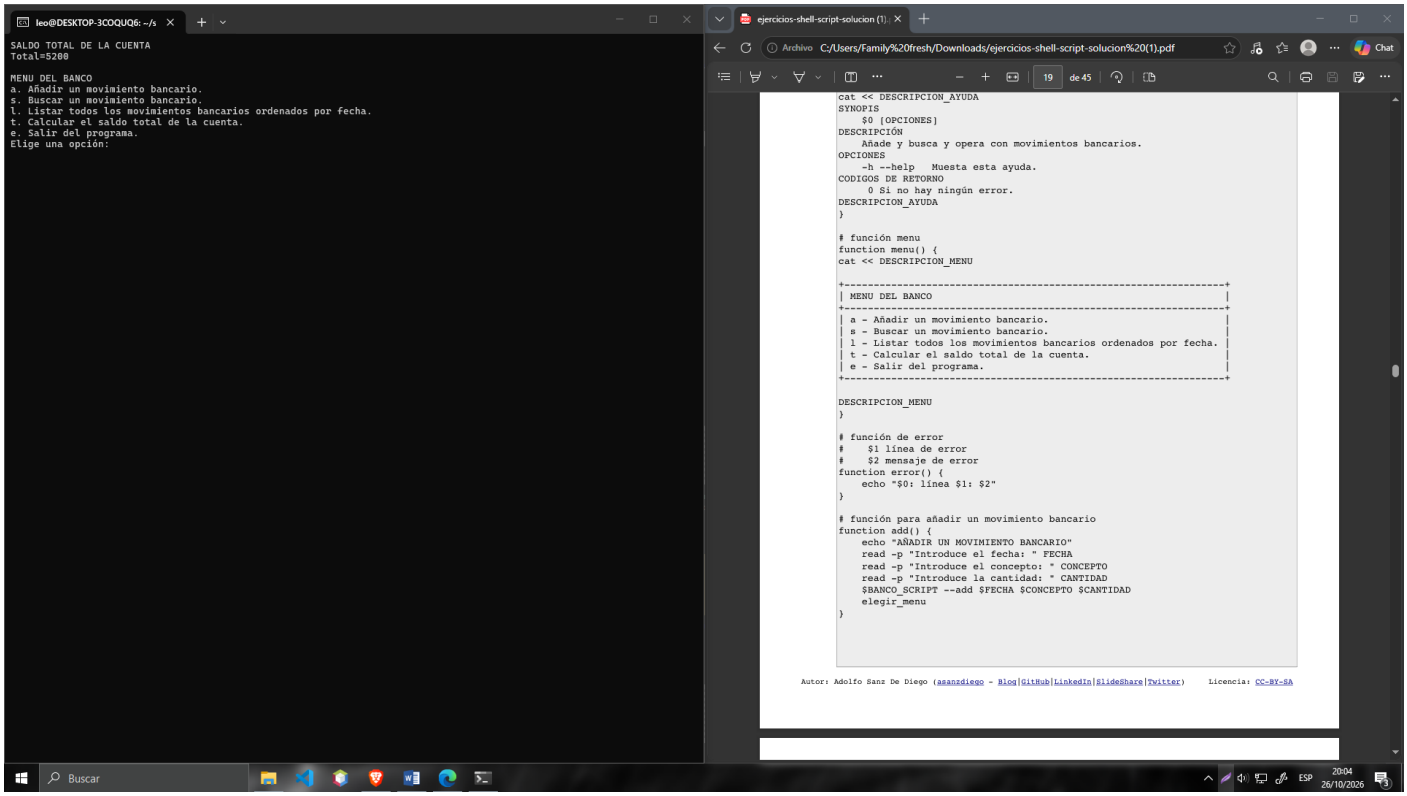


Figura 22.7. Captura original del ejercicio 22 (7 de 7).

Explicación de la salida: La salida comienza con un menú que permite elegir una operación bancaria. Después de escoger una opción, el script llama al procedimiento correspondiente y muestra el resultado o un mensaje de error.

Práctica 23. Banco mediante opciones o flags

Se adaptó la interfaz bancaria para reconocer opciones introducidas como argumentos, en lugar de depender únicamente de un menú interactivo.

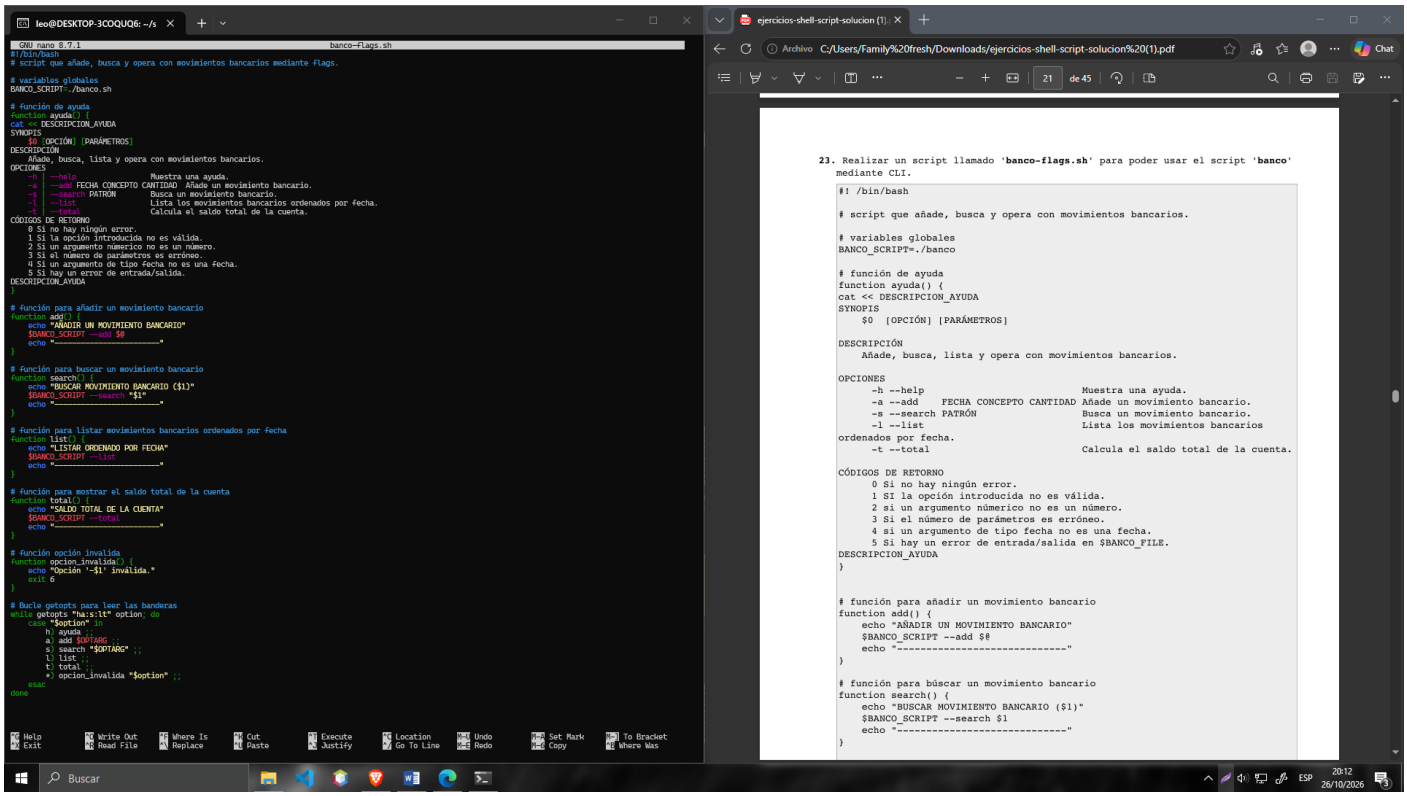


Figura 23.1. Captura original del ejercicio 23 (1 de 2).

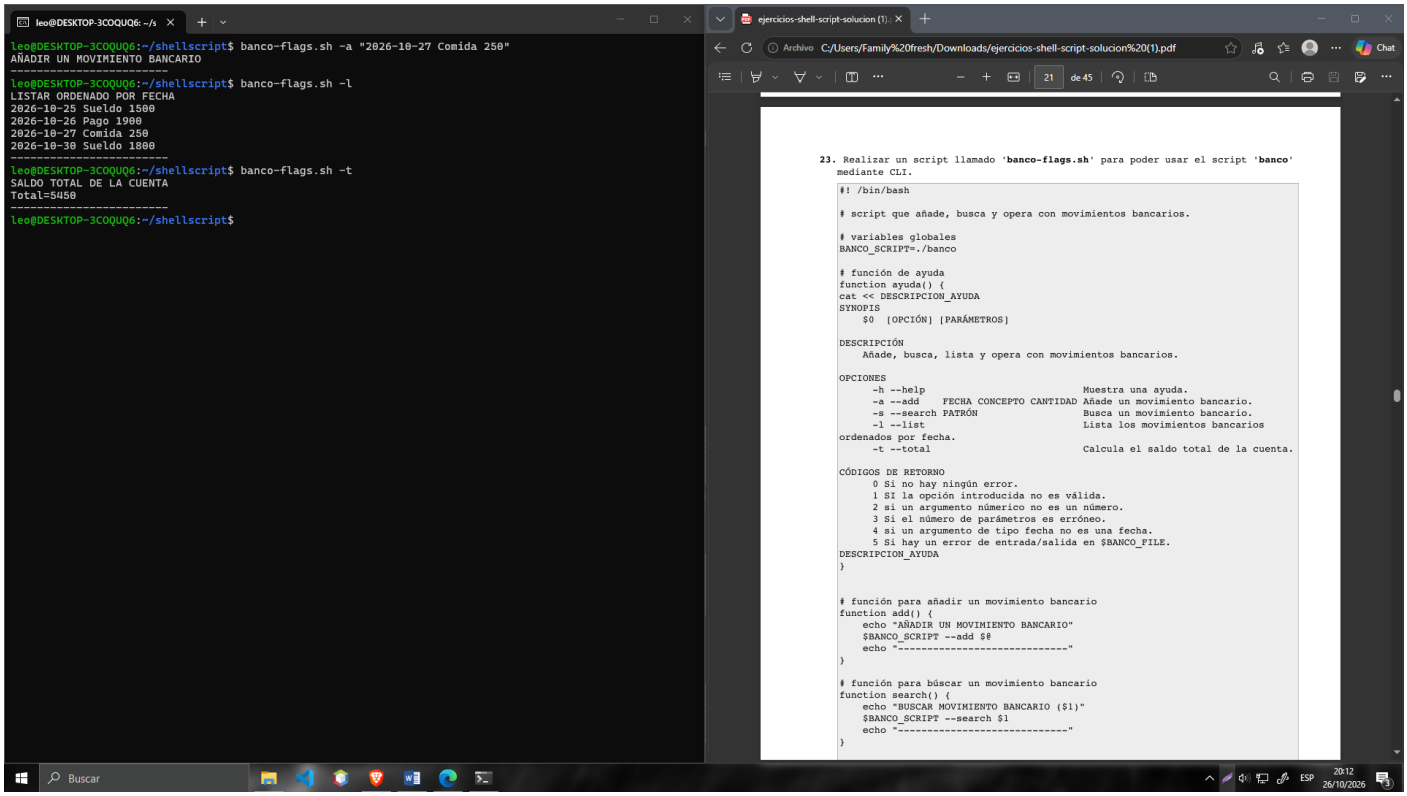


Figura 23.2. Captura original del ejercicio 23 (2 de 2).

Explicación de la salida: En esta variante, la acción se elige mediante opciones escritas directamente al ejecutar el comando. La terminal muestra el resultado de la consulta o del movimiento solicitado, sin tener que recorrer un menú interactivo.

Práctica 24. Proceso de alerta

Se implementó un script para ejecutar una tarea repetida con un intervalo definido. El código contiene ayuda y muestra el identificador del proceso.

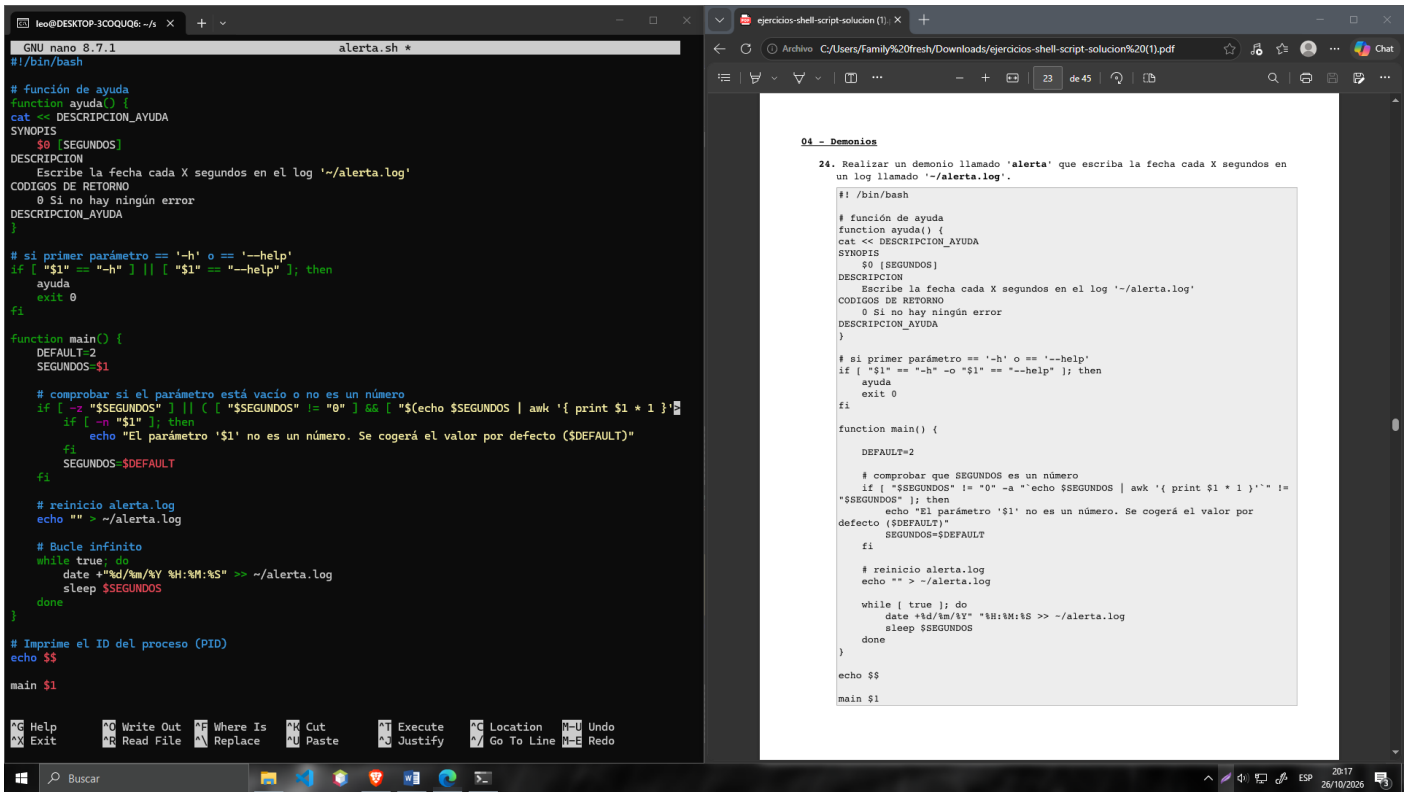


Figura 24.1. Captura original del ejercicio 24 (1 de 4).

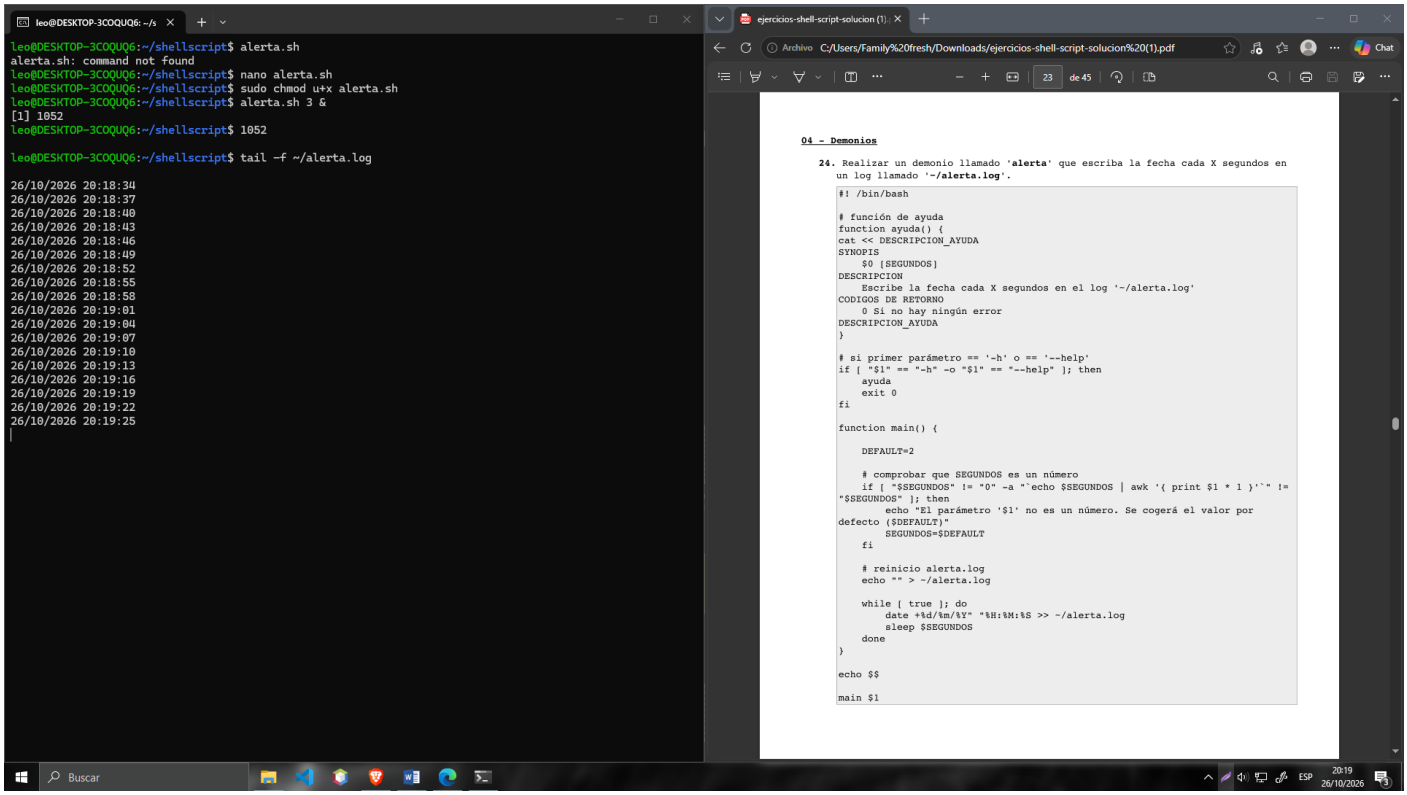


Figura 24.2. Captura original del ejercicio 24 (2 de 4).

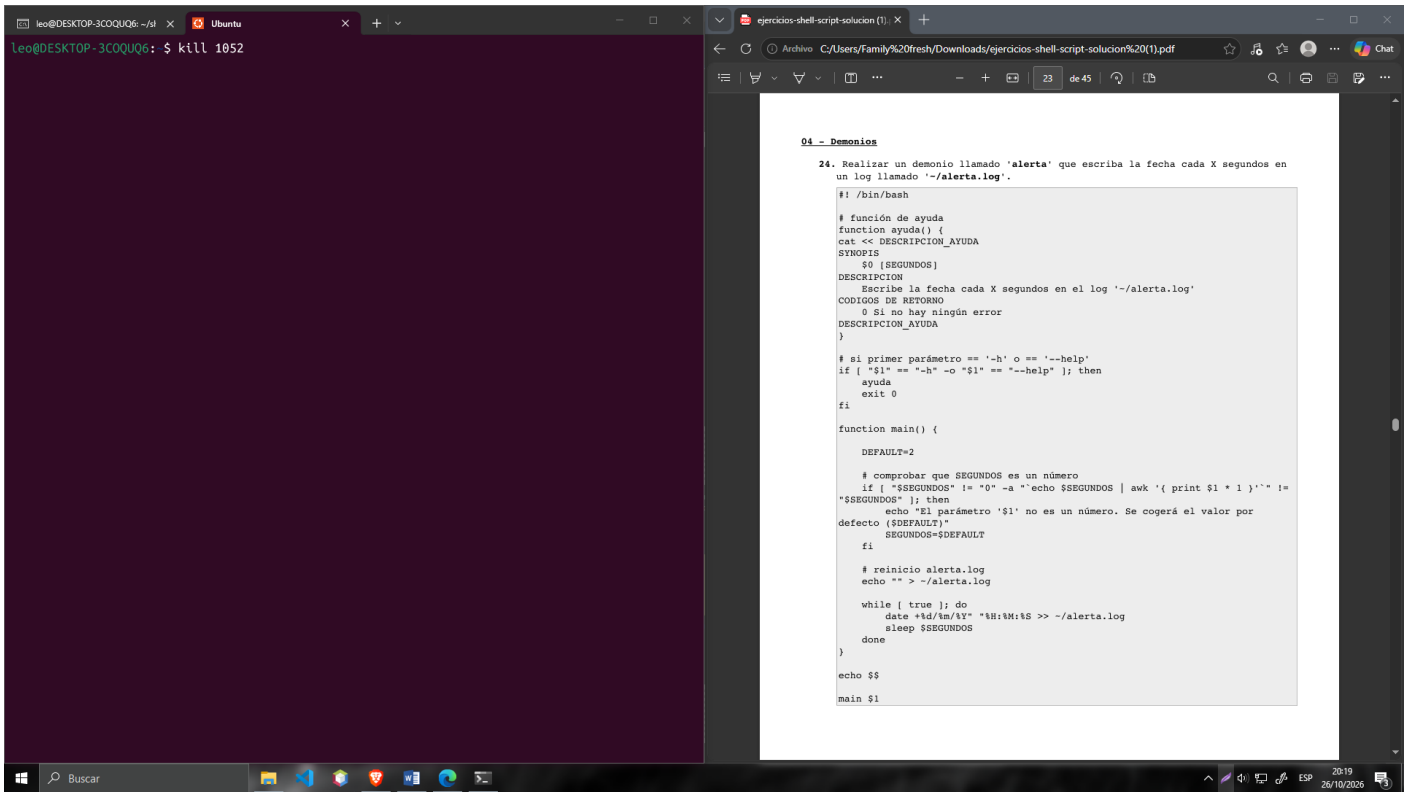


Figura 24.3. Captura original del ejercicio 24 (3 de 4).

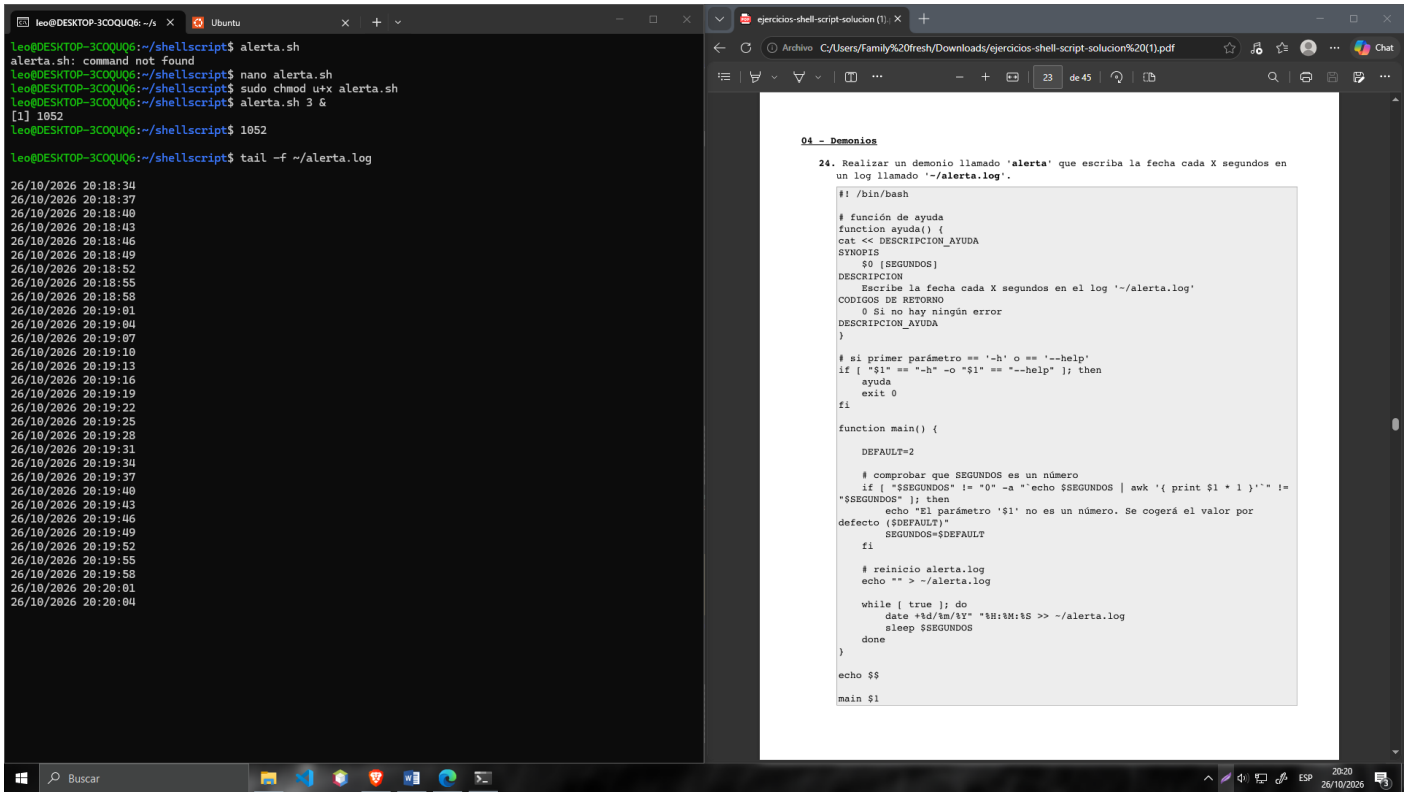


Figura 24.4. Captura original del ejercicio 24 (4 de 4).

Explicación de la salida: El script pone en funcionamiento un proceso de alerta. La salida permite identificar que la tarea está en ejecución y, según la forma en que se inicia, muestra mensajes o señales relacionadas con su actividad.

Práctica 25. Control del proceso de alerta

Se trabajó con el script de control de la alerta, empleando opciones para administrar su ejecución.

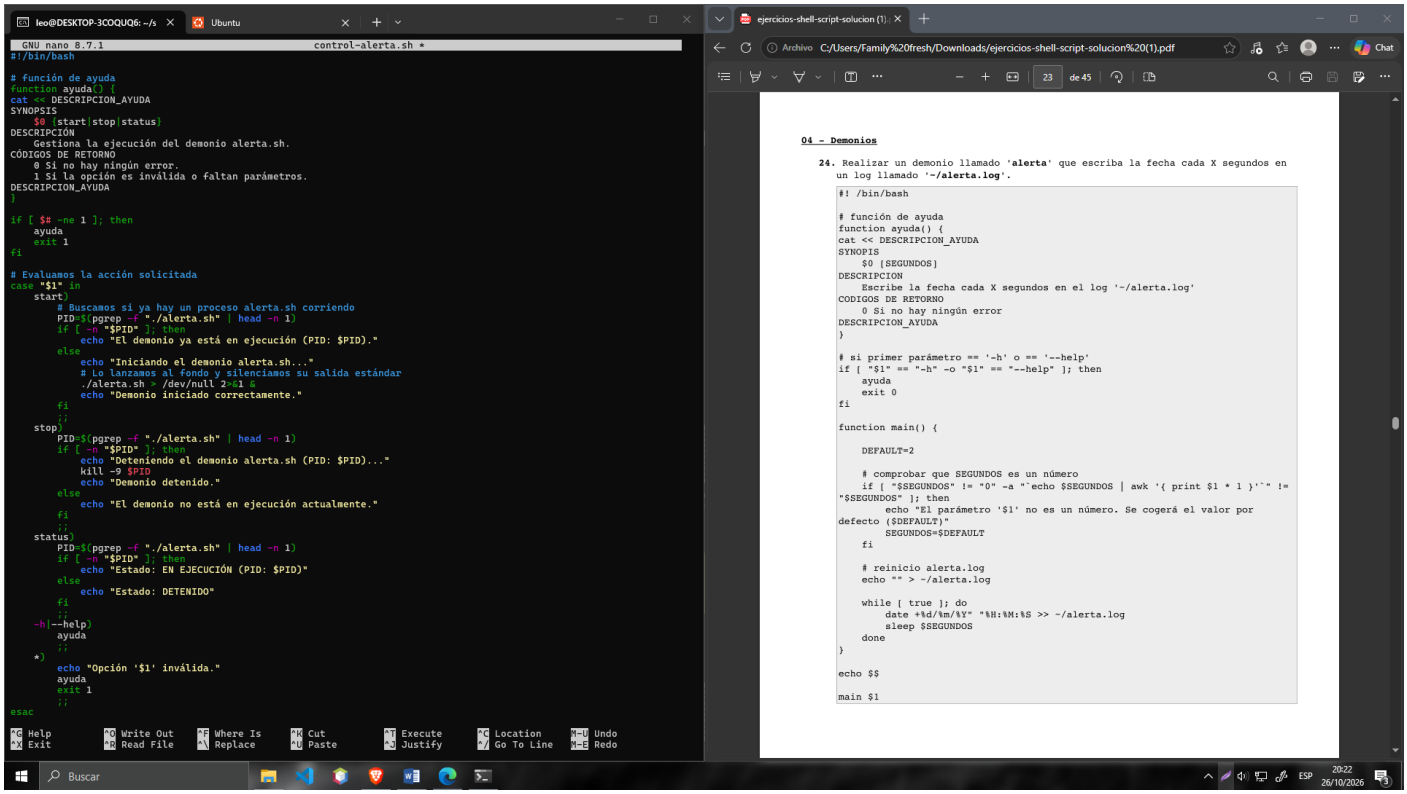


Figura 25.1. Captura original del ejercicio 25 (1 de 2).

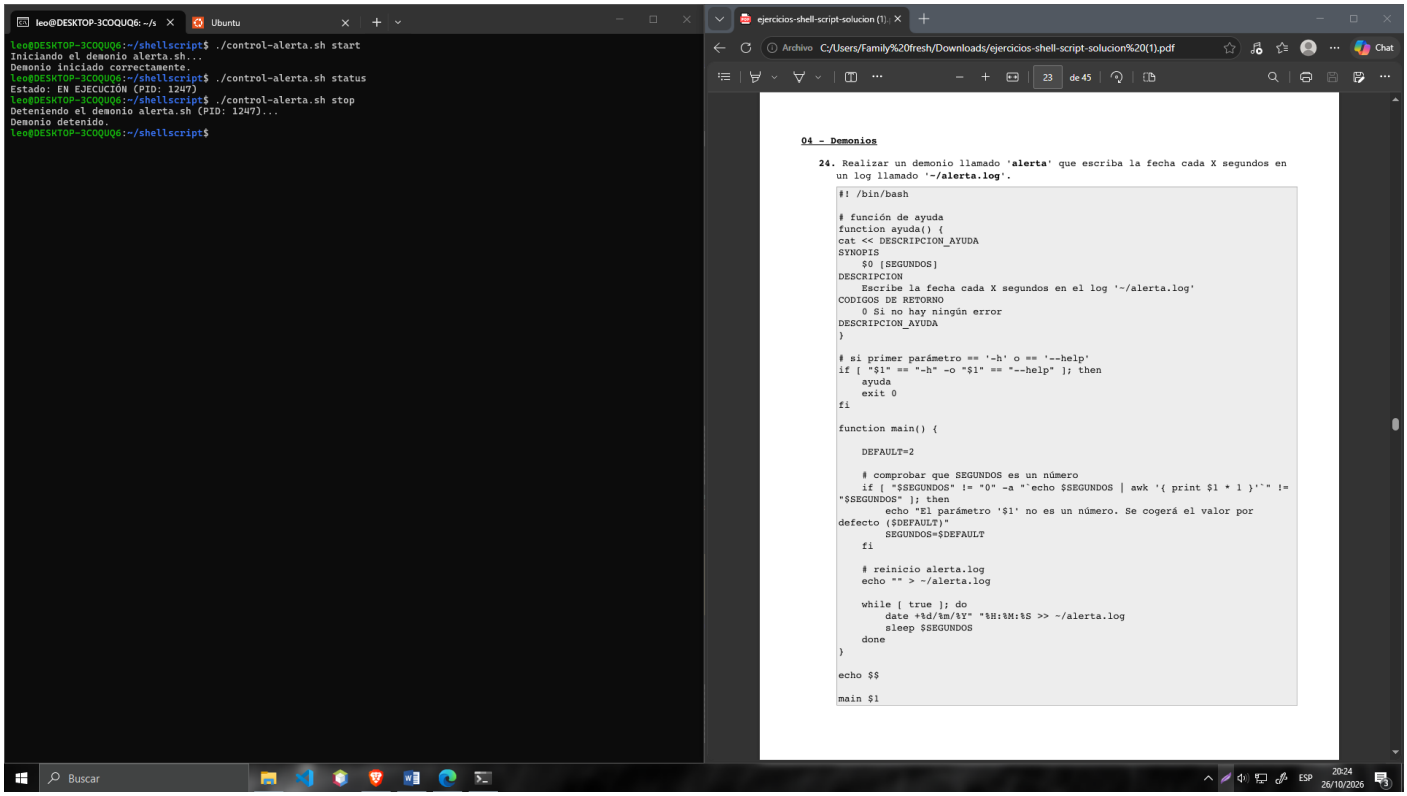


Figura 25.2. Captura original del ejercicio 25 (2 de 2).

Explicación de la salida: El programa recibe una instrucción para controlar el proceso de alerta. Según la opción utilizada, intenta iniciarlo, consultarlo o detenerlo, e informa del estado o del problema encontrado.

Práctica 26. Copia de seguridad total

Se preparó un script para generar una copia comprimida del directorio configurado y registrar información sobre la última copia total.

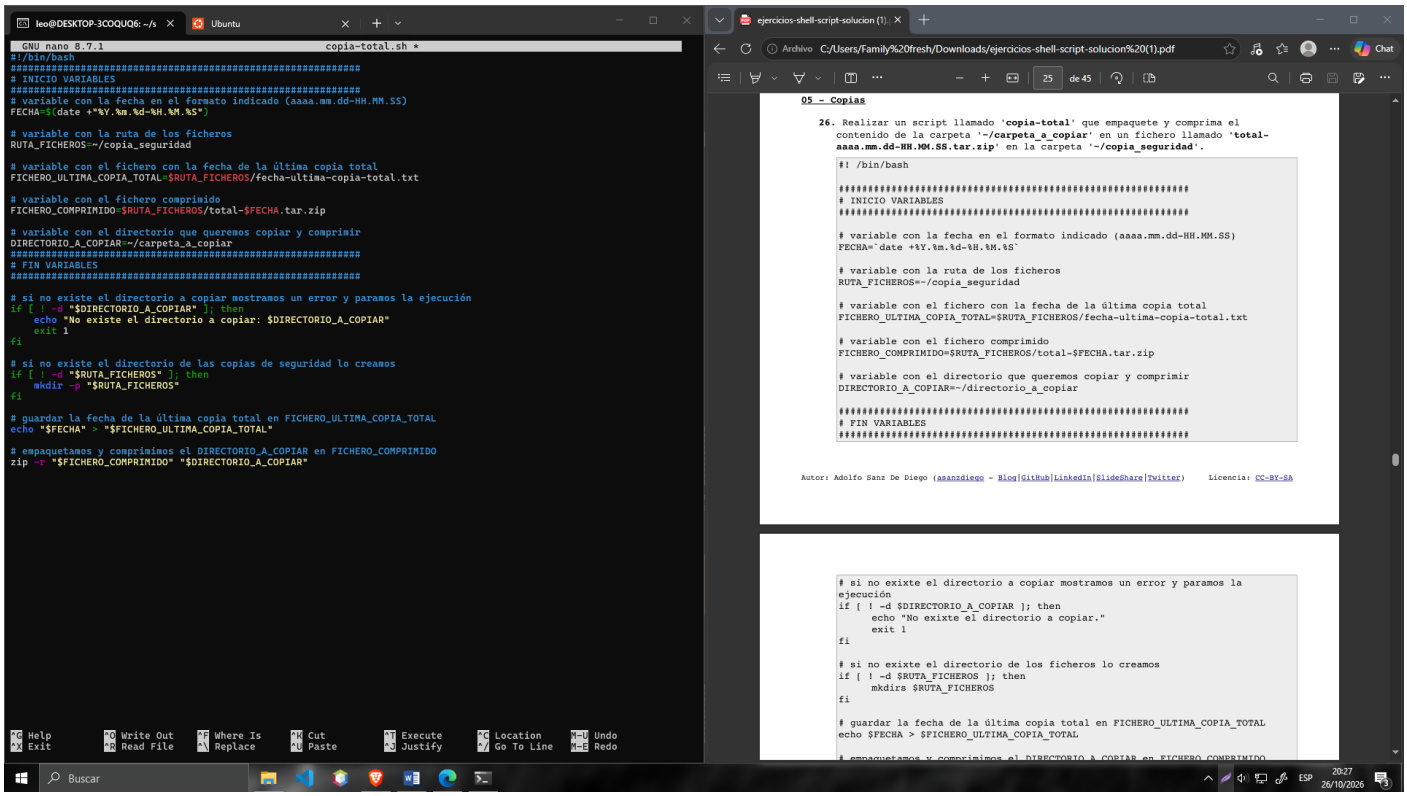


Figura 26.1. Captura original del ejercicio 26 (1 de 2).

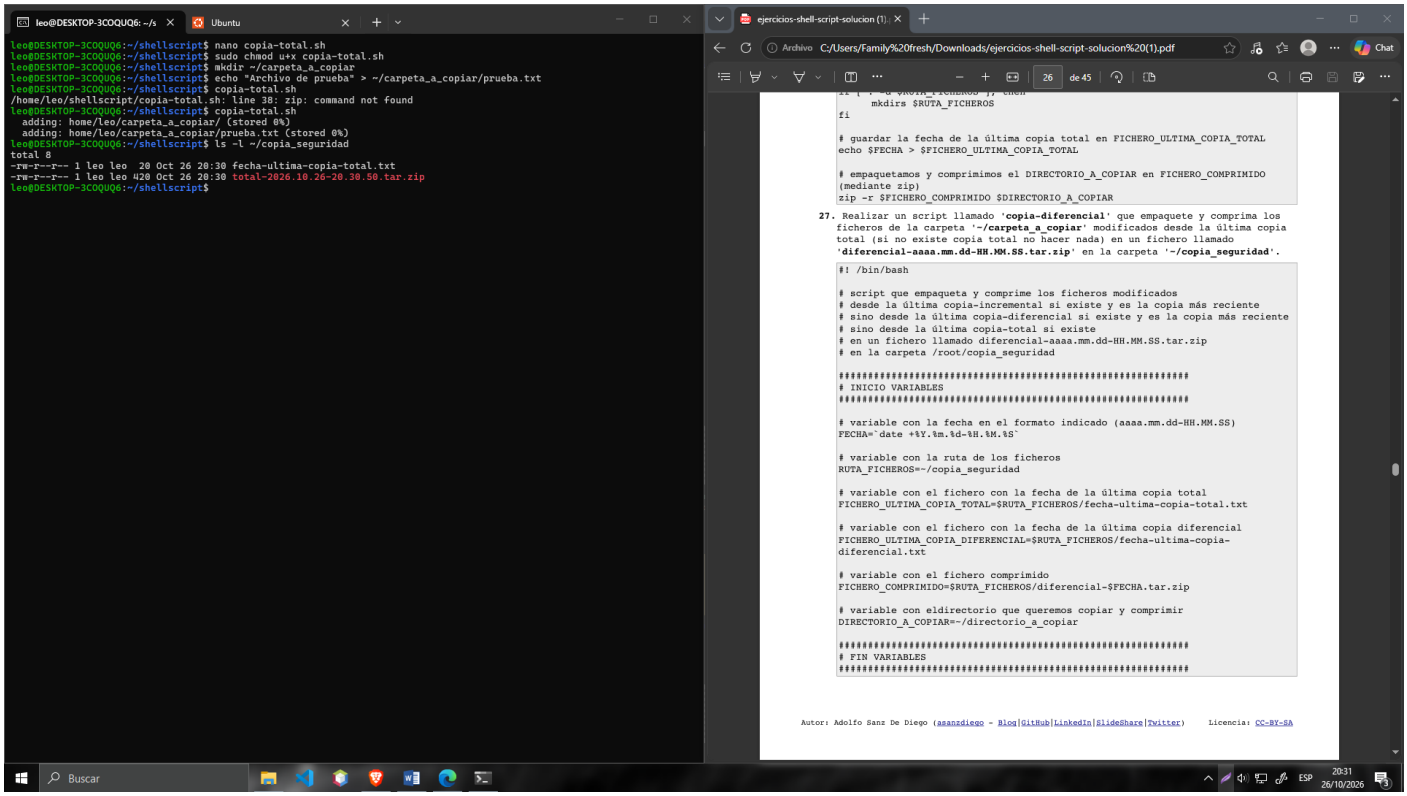


Figura 26.2. Captura original del ejercicio 26 (2 de 2).

Explicación de la salida: Al ejecutarse, el script genera una copia de seguridad completa de los archivos indicados. En la terminal pueden aparecer el nombre del respaldo y los mensajes del comando de compresión, que permiten comprobar si se produjo algún error.

Práctica 27. Copia de seguridad diferencial

Se desarrolló una copia que toma como referencia la fecha del último respaldo total para localizar archivos modificados desde entonces.

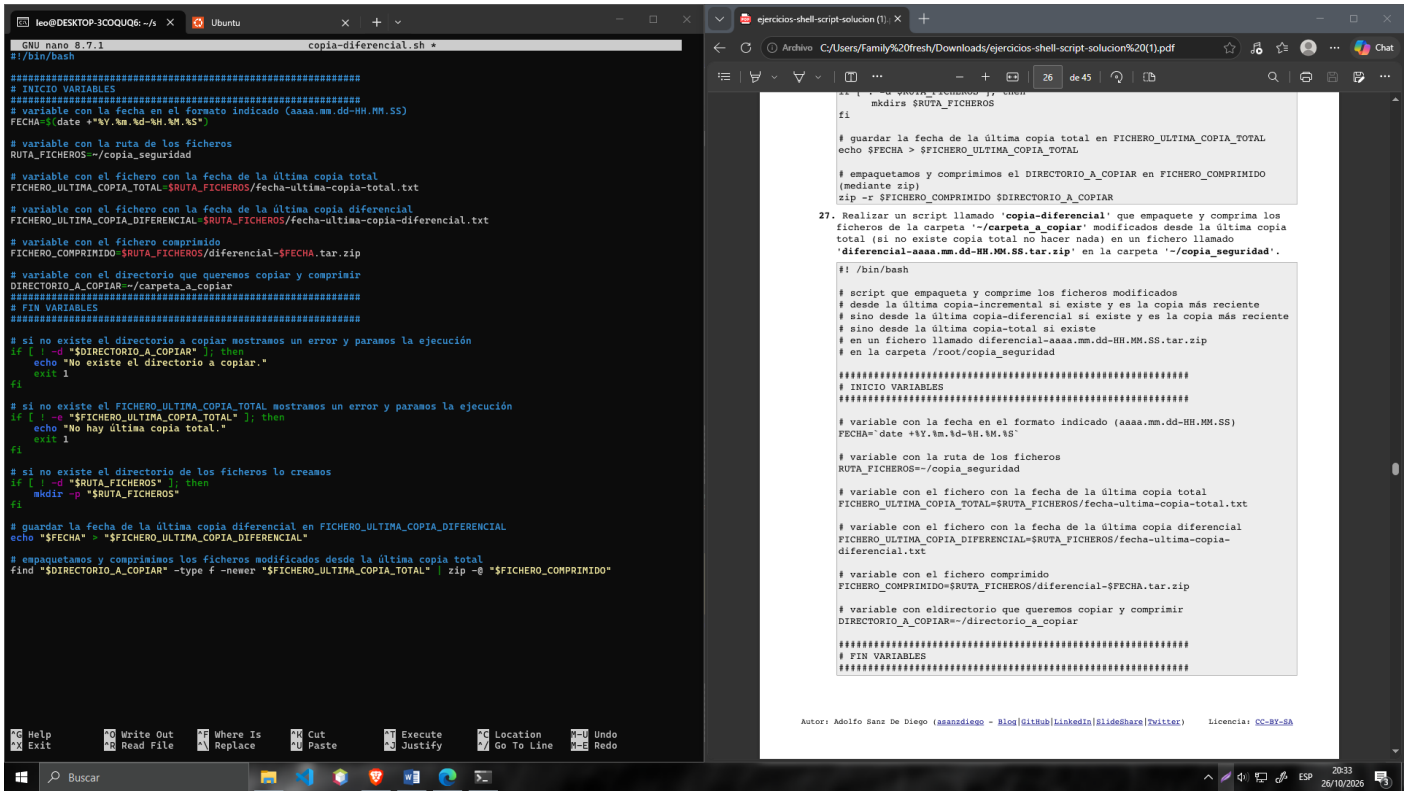


Figura 27.1. Captura original del ejercicio 27 (1 de 2).

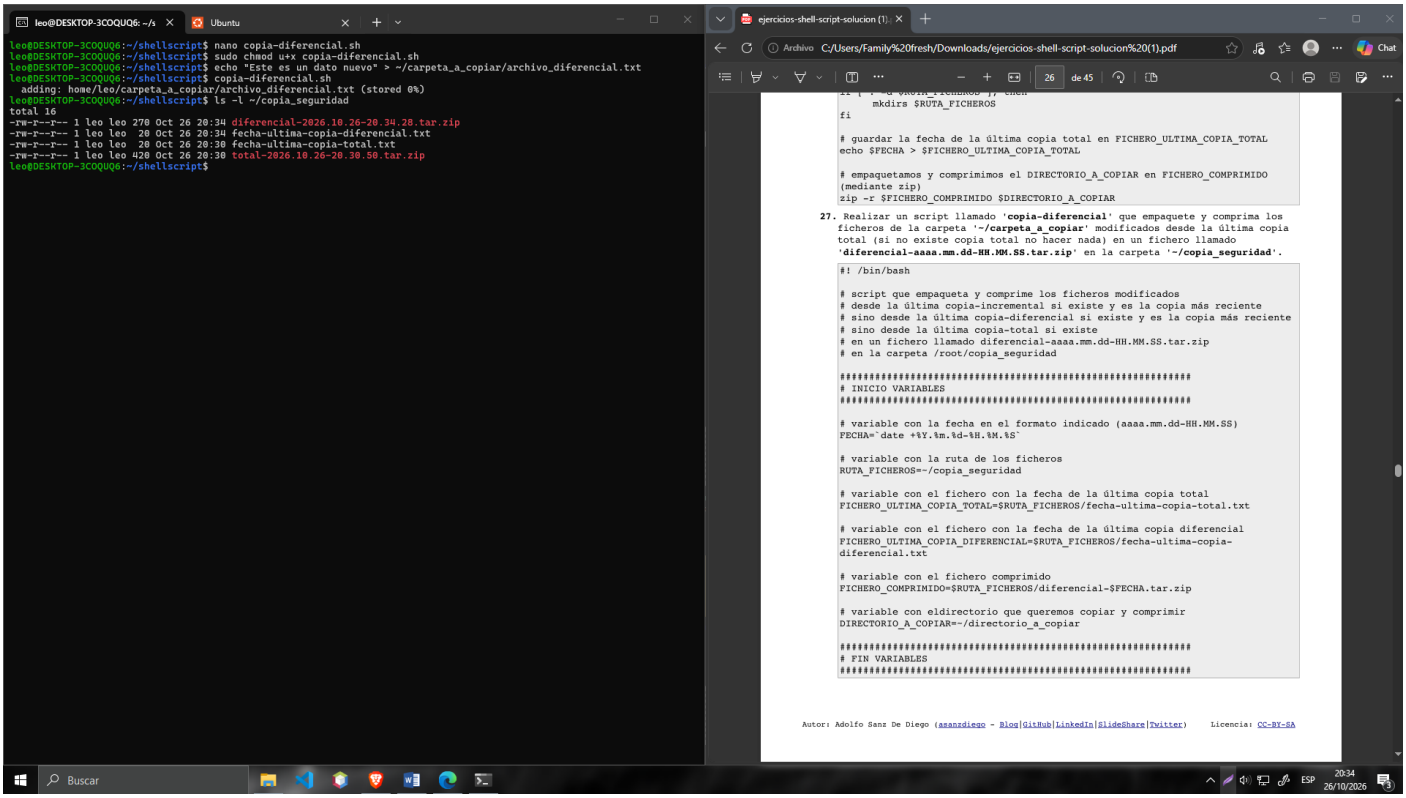


Figura 27.2. Captura original del ejercicio 27 (2 de 2).

Explicación de la salida: La salida corresponde a una copia diferencial, que reúne los archivos cambiados desde el respaldo completo de referencia. Los mensajes de la terminal ayudan a revisar si se generó el archivo de respaldo.

Práctica 28. Copia de seguridad incremental

Se programó una variante que considera los cambios posteriores a los respaldos previos, de acuerdo con la lógica del código.

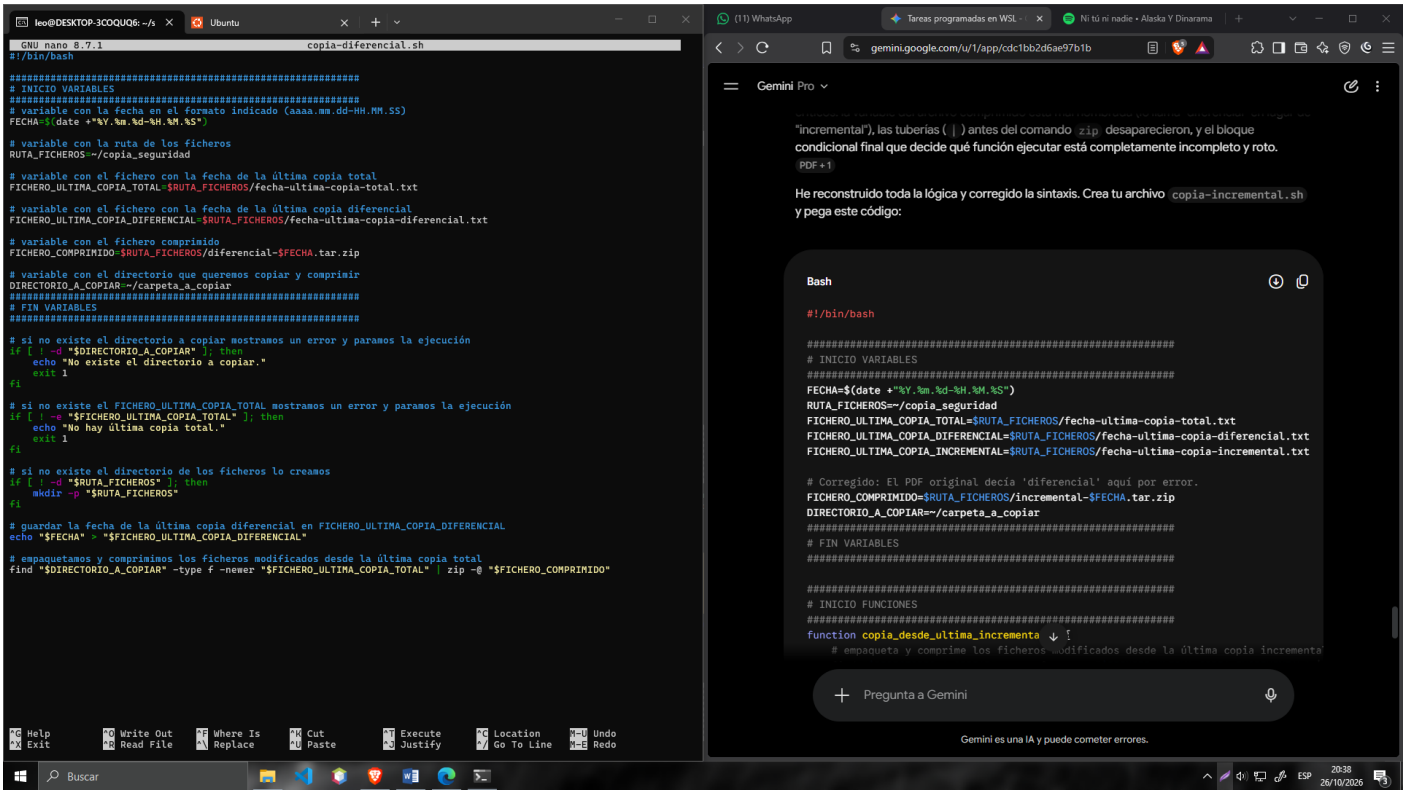


Figura 28.1. Captura original del ejercicio 28 (1 de 3).

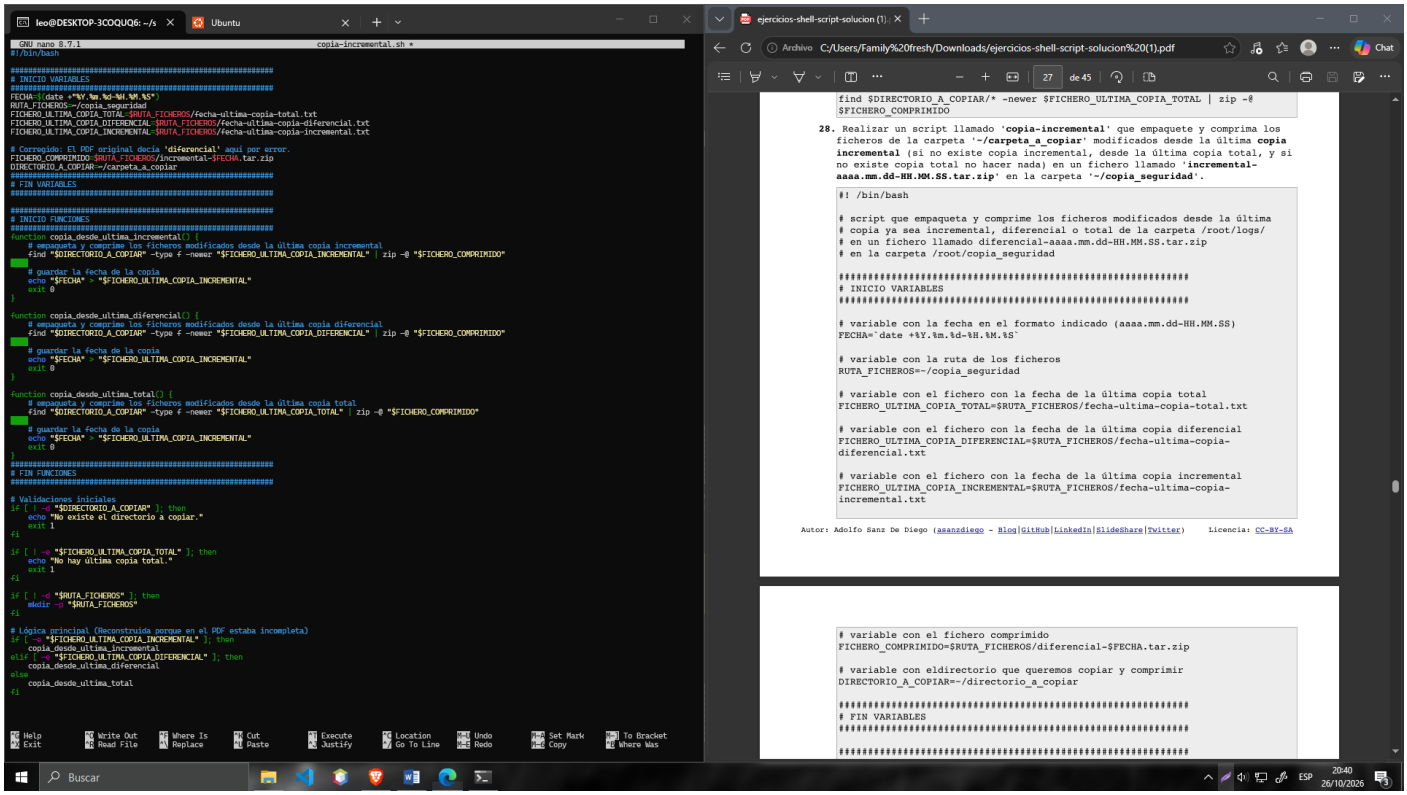


Figura 28.2. Captura original del ejercicio 28 (2 de 3).

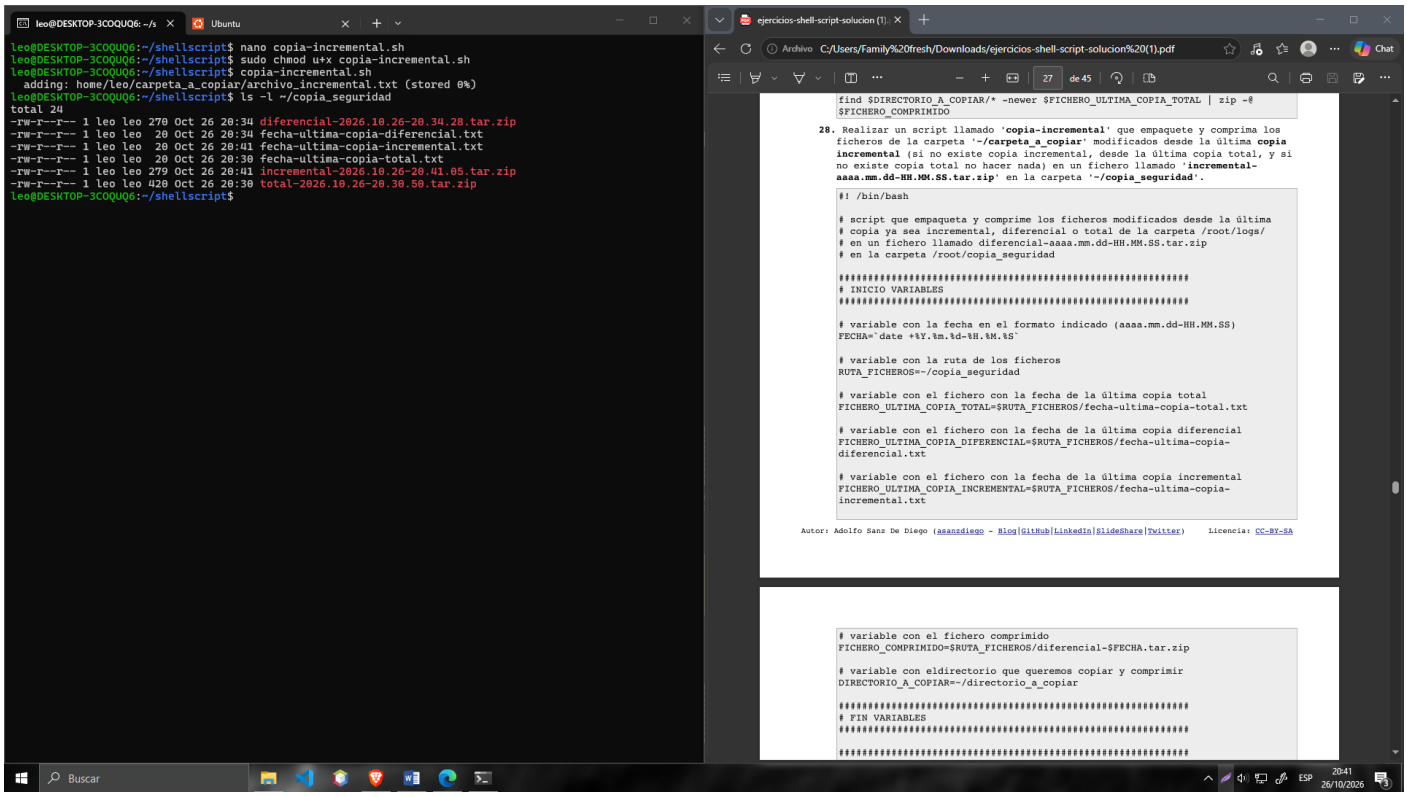


Figura 28.3. Captura original del ejercicio 28 (3 de 3).

Explicación de la salida: El script crea un respaldo incremental con los archivos modificados desde la referencia utilizada para la copia anterior. Al terminar, se puede revisar el archivo generado y cualquier aviso que haya mostrado la terminal.

Práctica 29. Programación de tareas con cron

Se incluyó la práctica de configuración de una tarea programada con cron, apoyada en las capturas del documento.

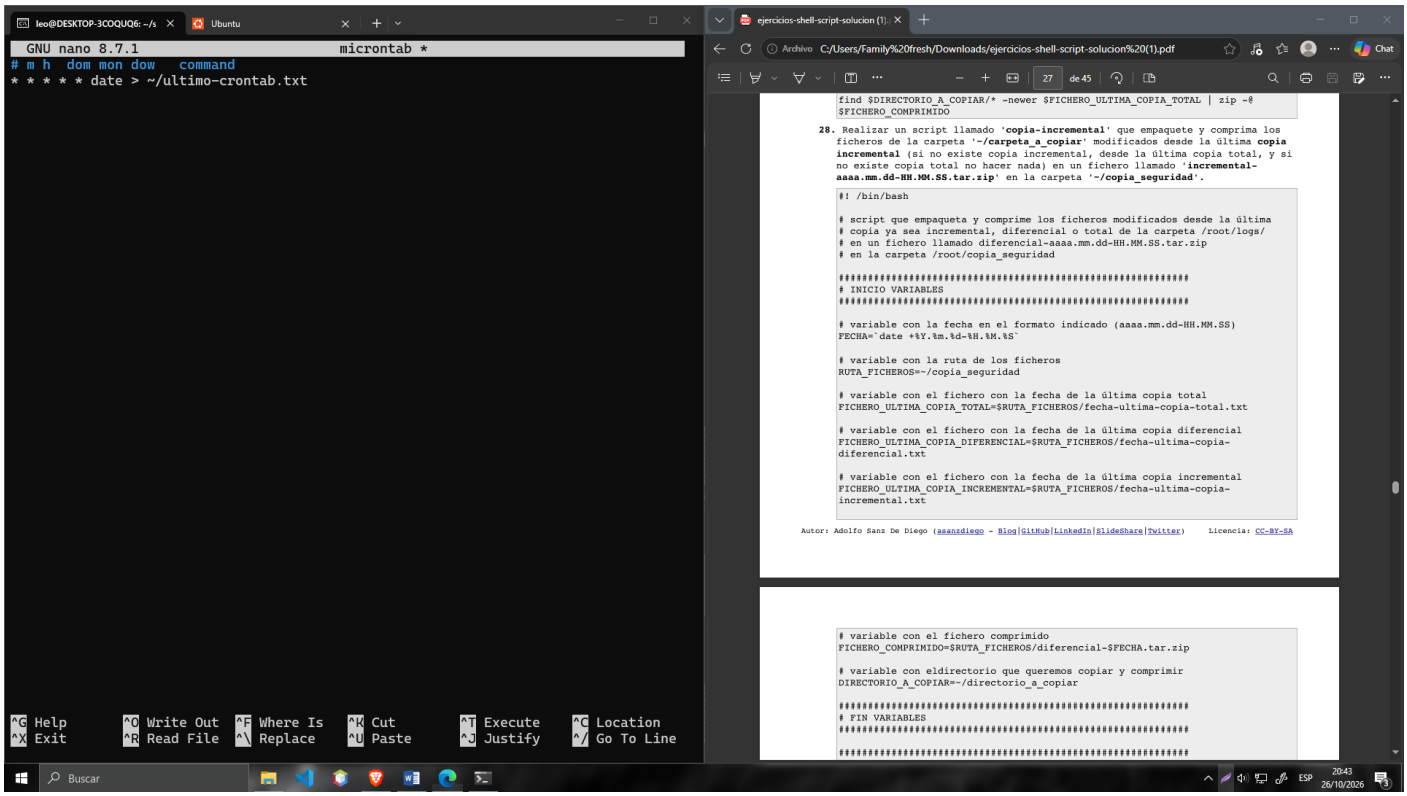


Figura 29.1. Captura original del ejercicio 29 (1 de 2).

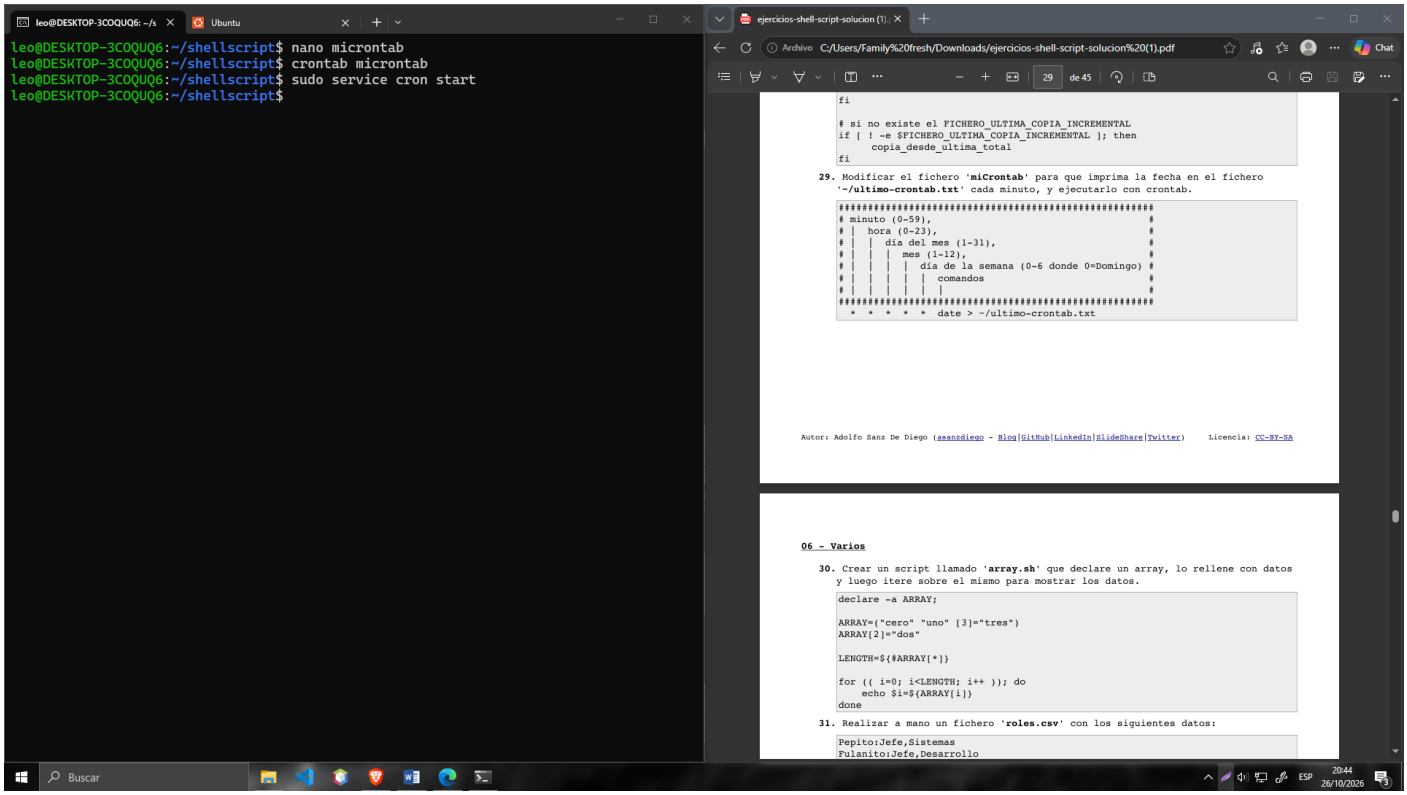


Figura 29.2. Captura original del ejercicio 29 (2 de 2).

Explicación de la salida: En este ejercicio se configura una ejecución automática con cron. La evidencia permite revisar la programación de la tarea y comprobar sus resultados en el momento indicado, sin ejecutar el script manualmente.

Práctica 30. Arreglos en Bash

Se declaró un arreglo, se llenó con datos y se recorrieron sus posiciones mediante un ciclo for.

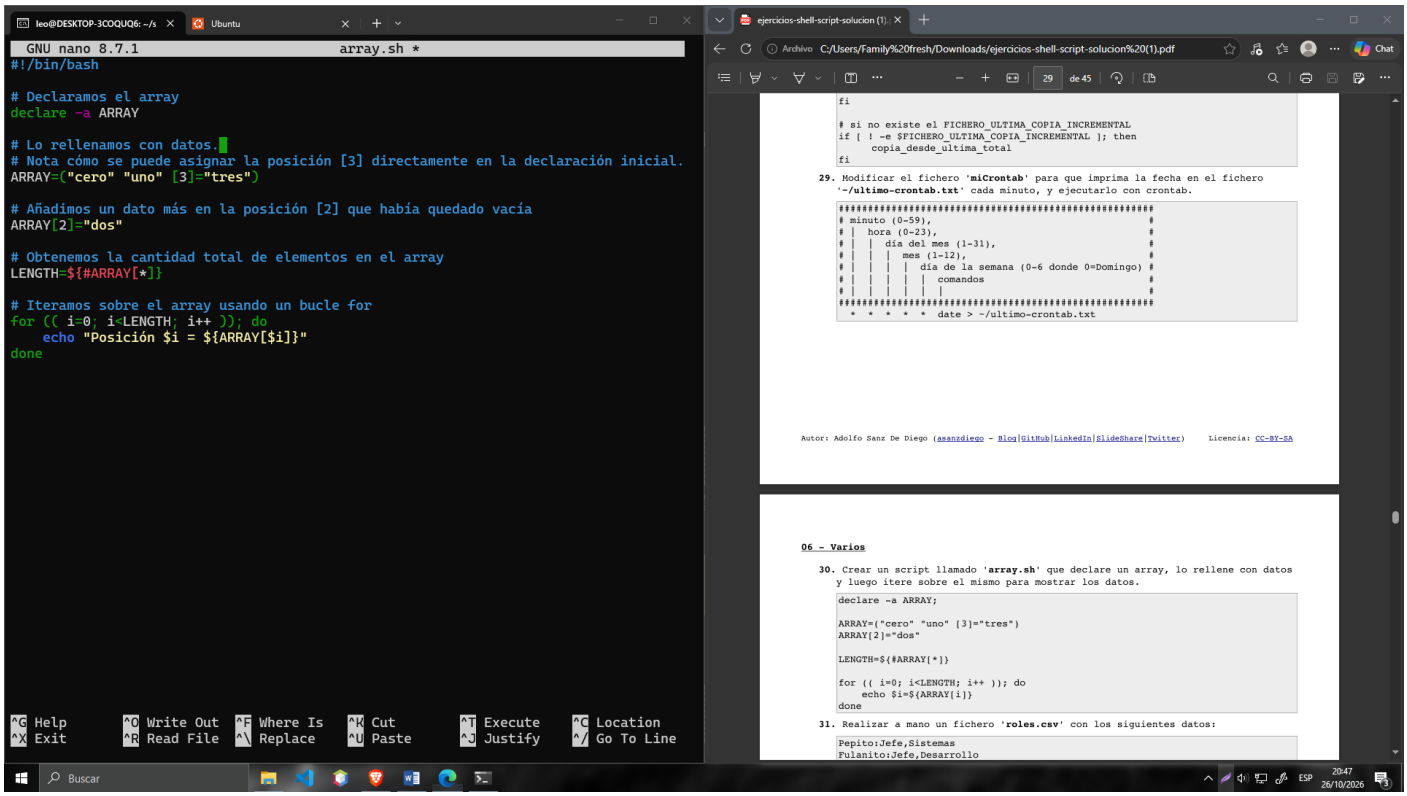


Figura 30.1. Captura original del ejercicio 30 (1 de 2).

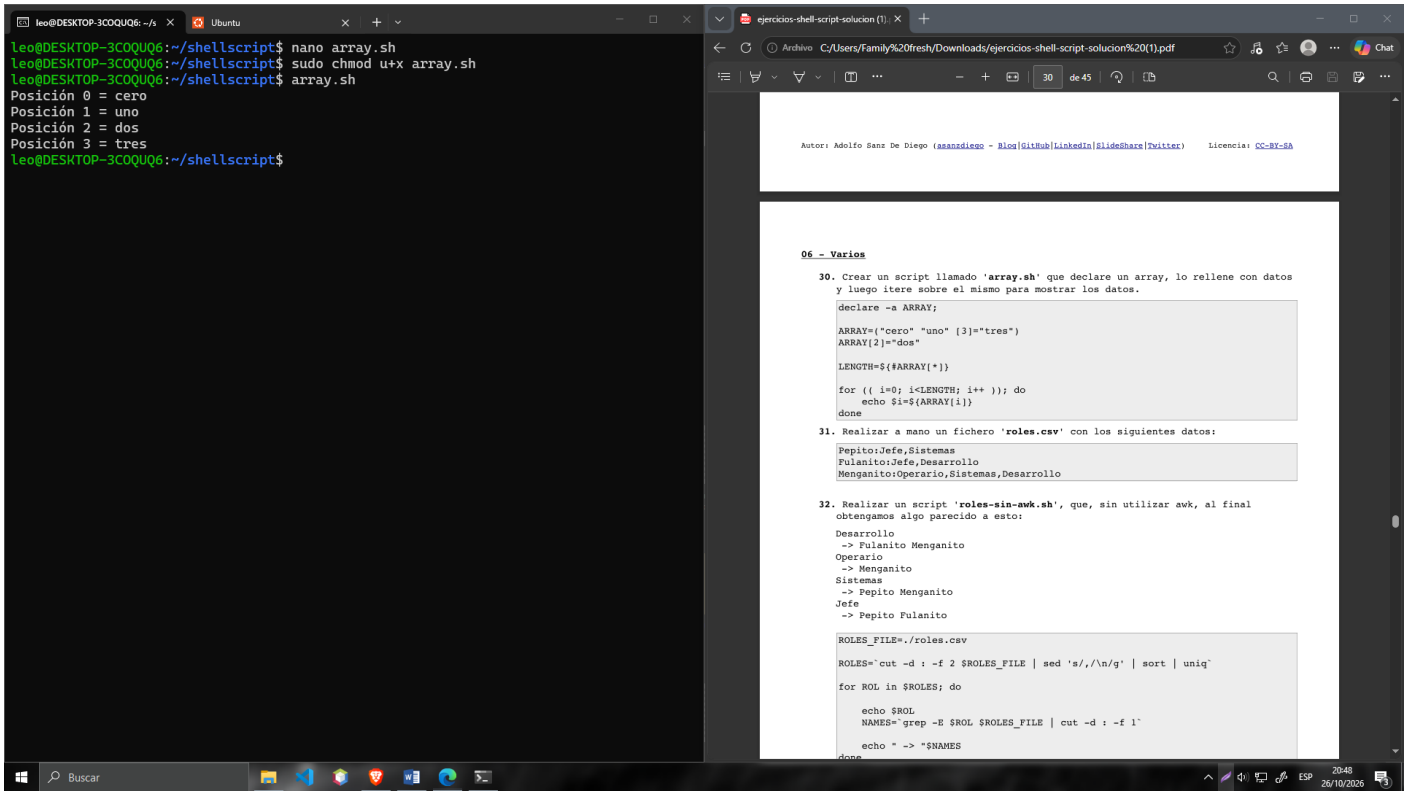


Figura 30.2. Captura original del ejercicio 30 (2 de 2).

Explicación de la salida: El programa muestra los elementos de un arreglo de Bash al acceder a sus posiciones. La salida permite comprobar que los valores guardados se pueden recuperar individualmente o recorrer mediante un ciclo.

Práctica 31. Datos de roles en CSV

Se preparó el archivo de roles que se utilizaría en los siguientes ejercicios de procesamiento de texto.

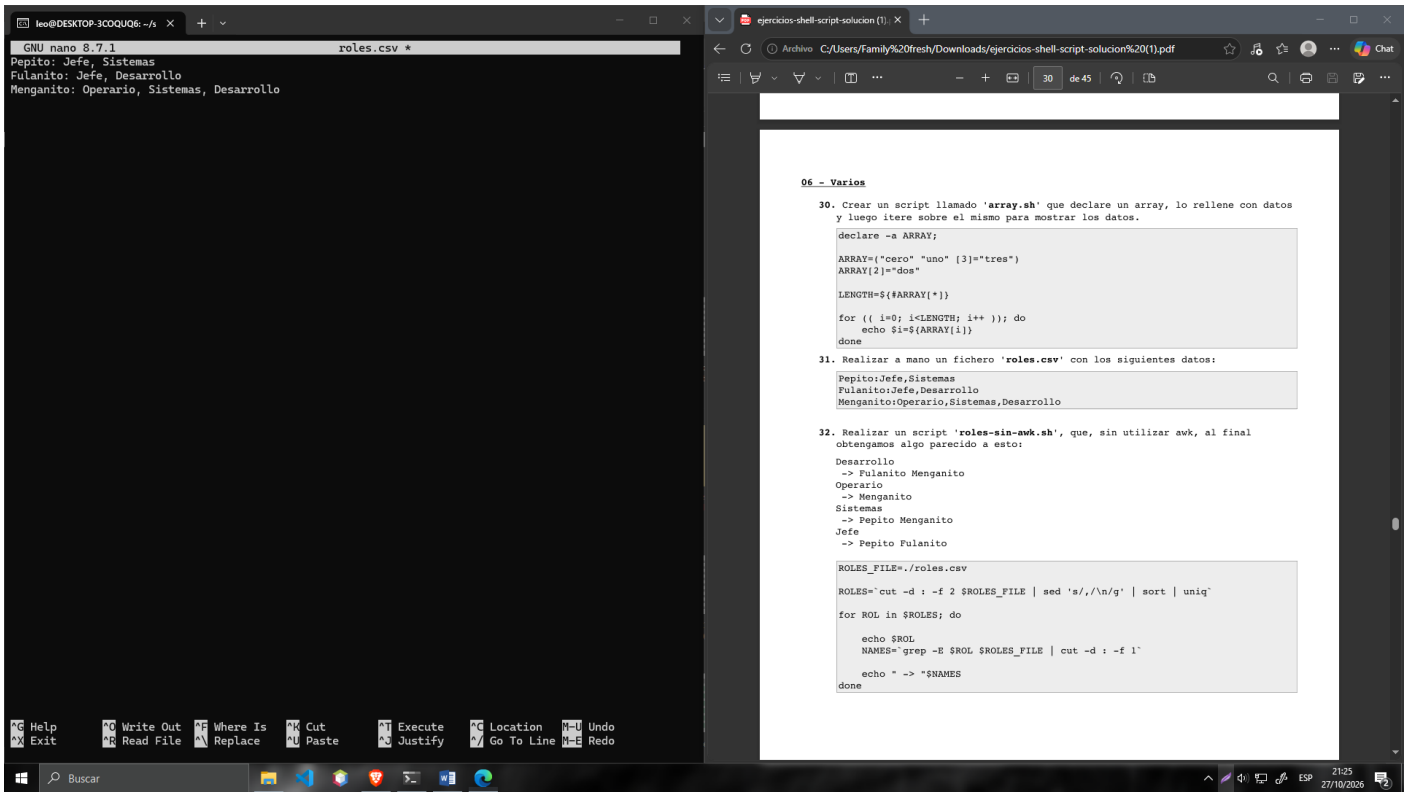


Figura 31.1. Captura original del ejercicio 31 (1 de 1).

Explicación de la salida: Se prepara un archivo CSV con información sobre roles. Al consultar su contenido, se observan registros organizados en campos separados, que sirven como entrada para los siguientes ejercicios.

Práctica 32. Procesamiento de roles sin AWK

Se escribió un script que lee el archivo roles.csv y separa su contenido utilizando herramientas y operaciones de Shell, sin depender de AWK.

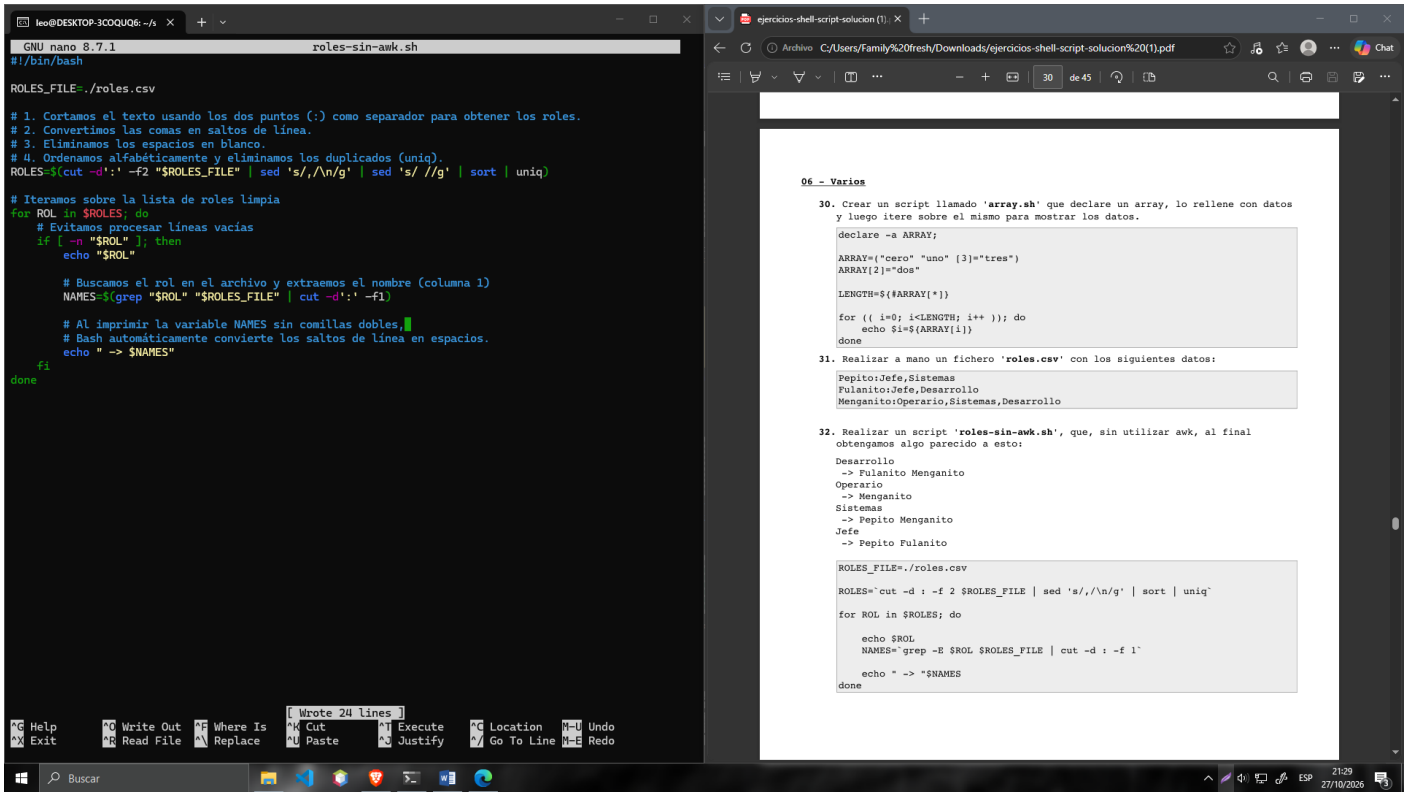


Figura 32.1. Captura original del ejercicio 32 (1 de 2).

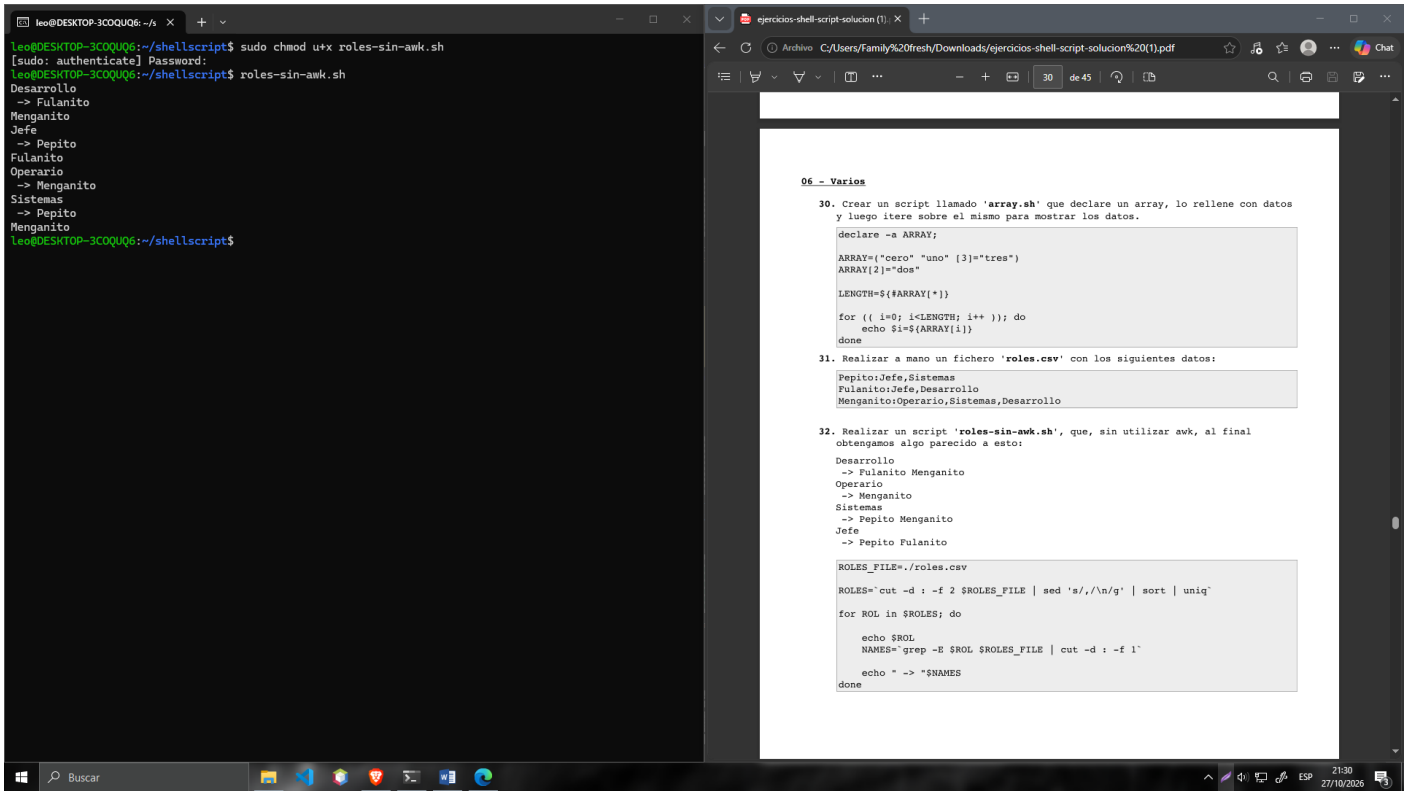


Figura 32.2. Captura original del ejercicio 32 (2 de 2).

Explicación de la salida: El script lee los datos de los roles y muestra los campos seleccionados usando herramientas de la terminal, sin AWK. La salida organiza la información que originalmente estaba almacenada en el archivo CSV.

Práctica 33. Procesamiento de roles con AWK

Se realizó una segunda solución de los datos de roles, esta vez usando un archivo AWK para definir separadores y dar formato a la salida.

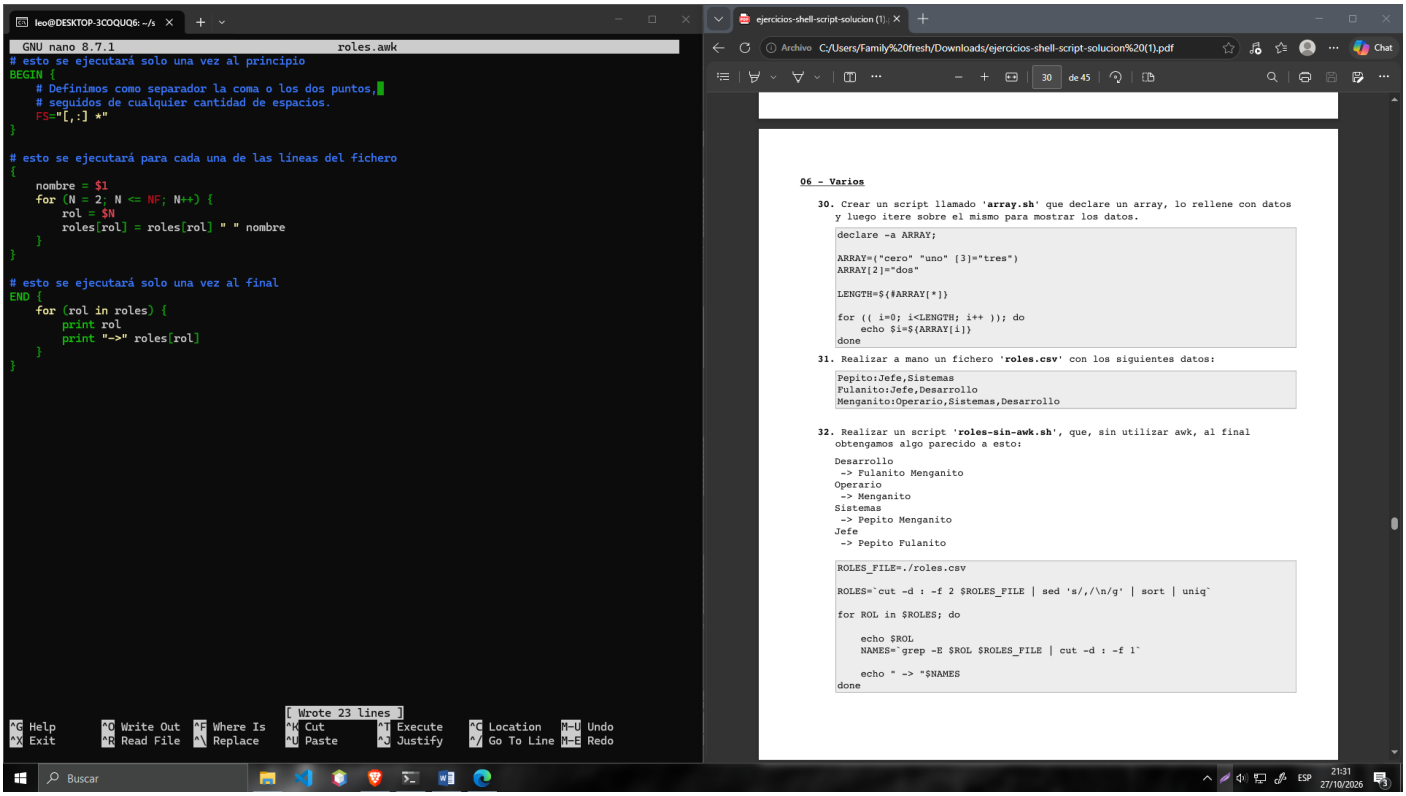


Figura 33.1. Captura original del ejercicio 33 (1 de 3).

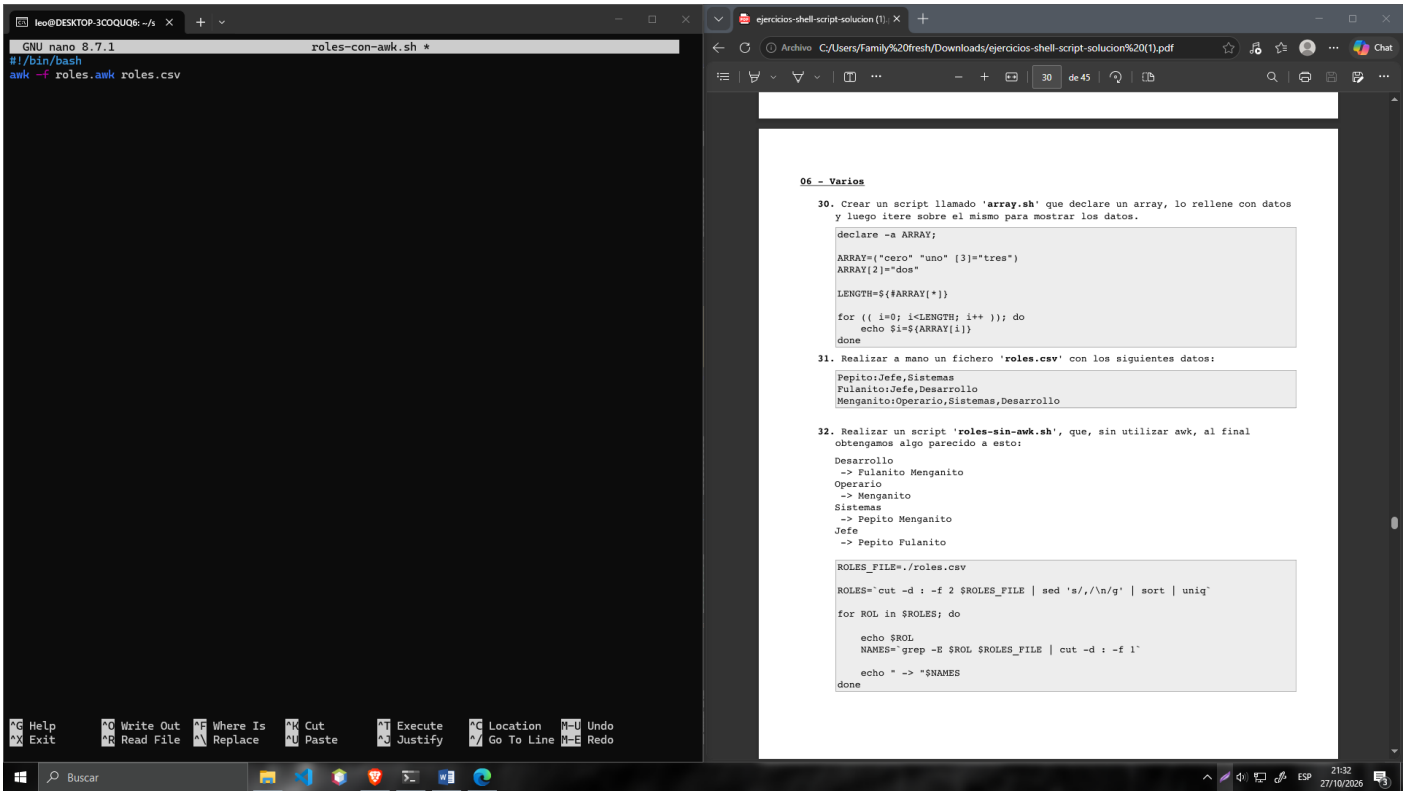


Figura 33.2. Captura original del ejercicio 33 (2 de 3).

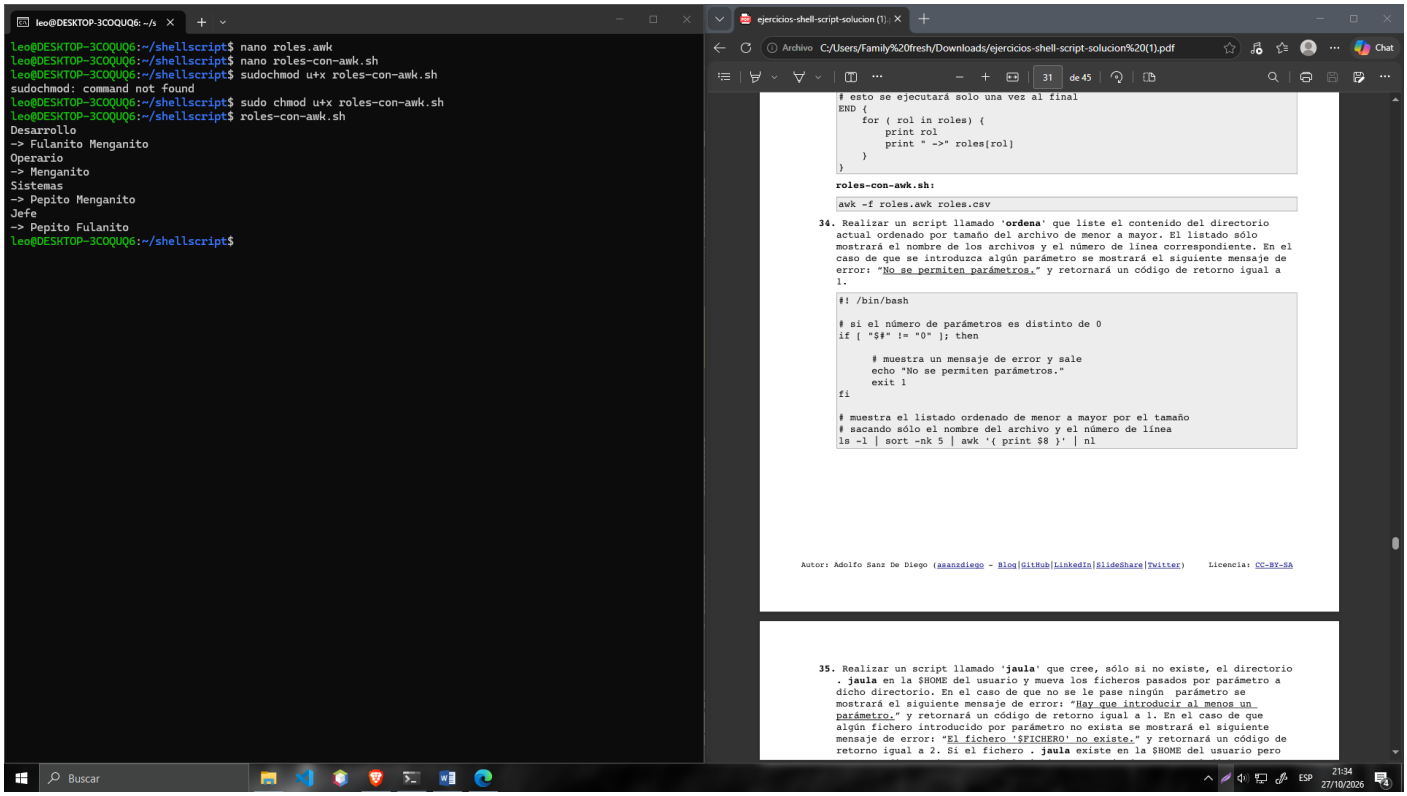


Figura 33.3. Captura original del ejercicio 33 (3 de 3).

Explicación de la salida: En esta versión, AWK procesa las columnas del archivo de roles y presenta los datos solicitados. La salida sirve para comparar el resultado con el método del ejercicio anterior.

Práctica 34. Listado ordenado de archivos

Se elaboró un script que obtiene información de los archivos del directorio y organiza los datos usando una combinación de comandos.

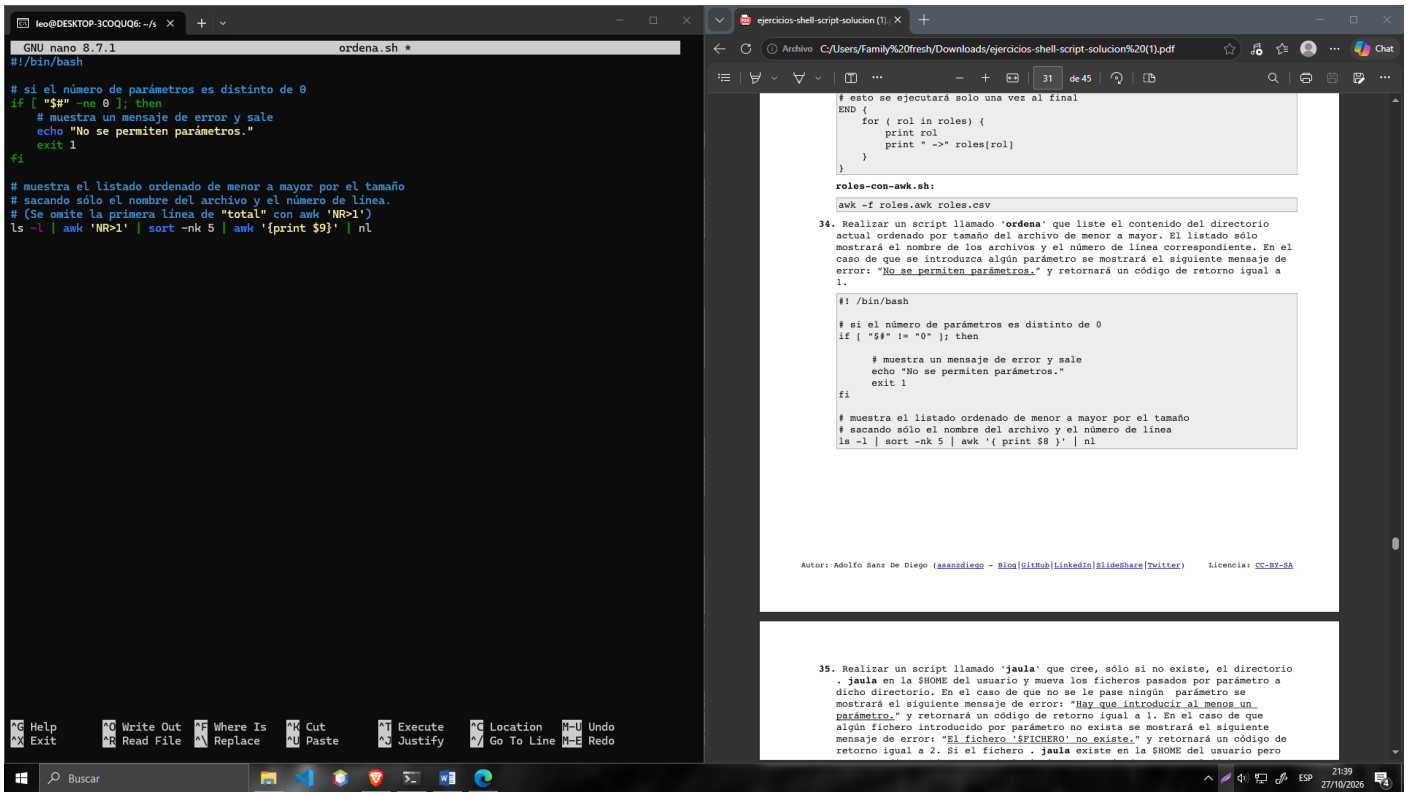


Figura 34.1. Captura original del ejercicio 34 (1 de 2).

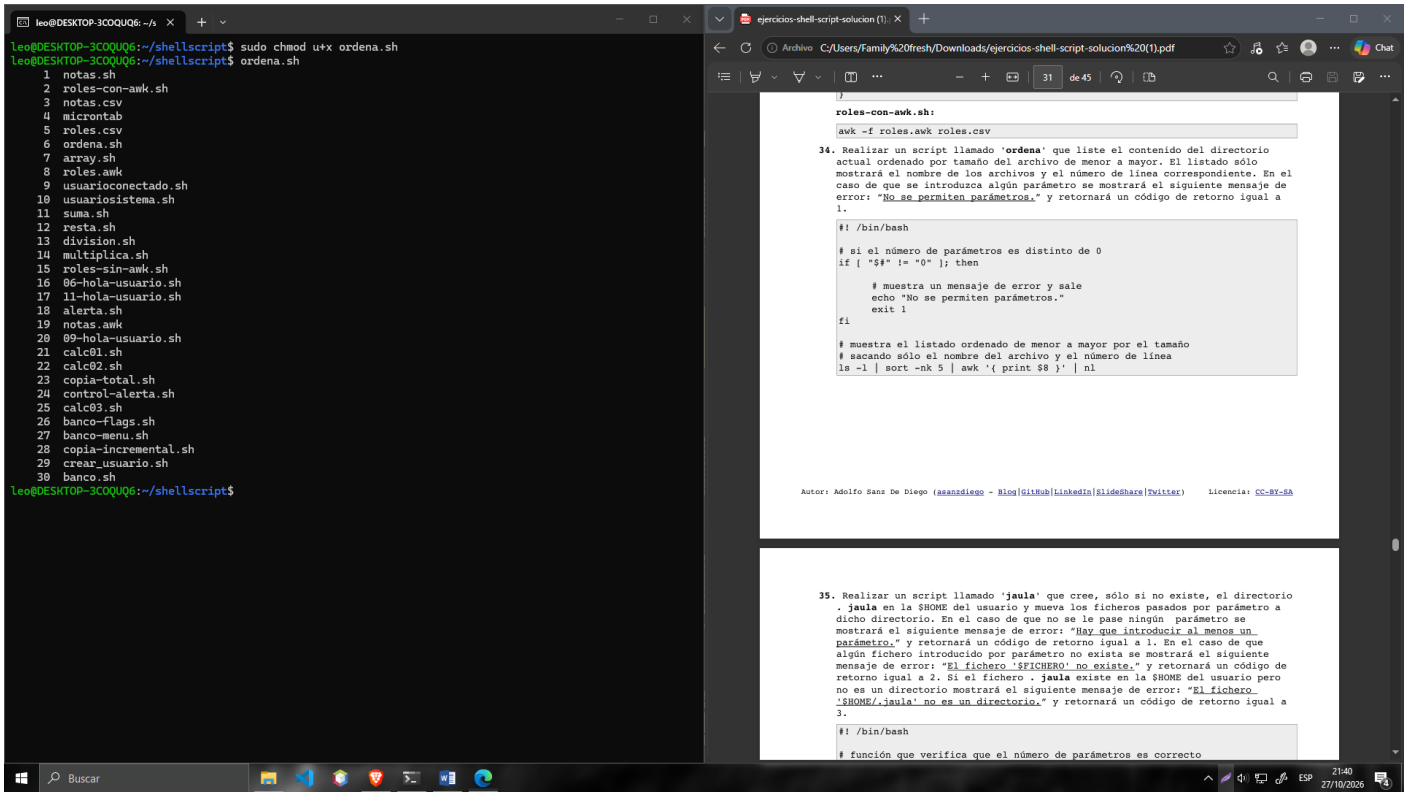


Figura 34.2. Captura original del ejercicio 34 (2 de 2).

Explicación de la salida: La terminal presenta un listado de archivos ordenado según el criterio del script. Se combinan comandos para obtener los nombres y acomodarlos antes de mostrarlos en pantalla.

Práctica 35. Organización de archivos en una jaula

Se programaron funciones para comprobar parámetros, preparar un directorio y mover los archivos indicados.

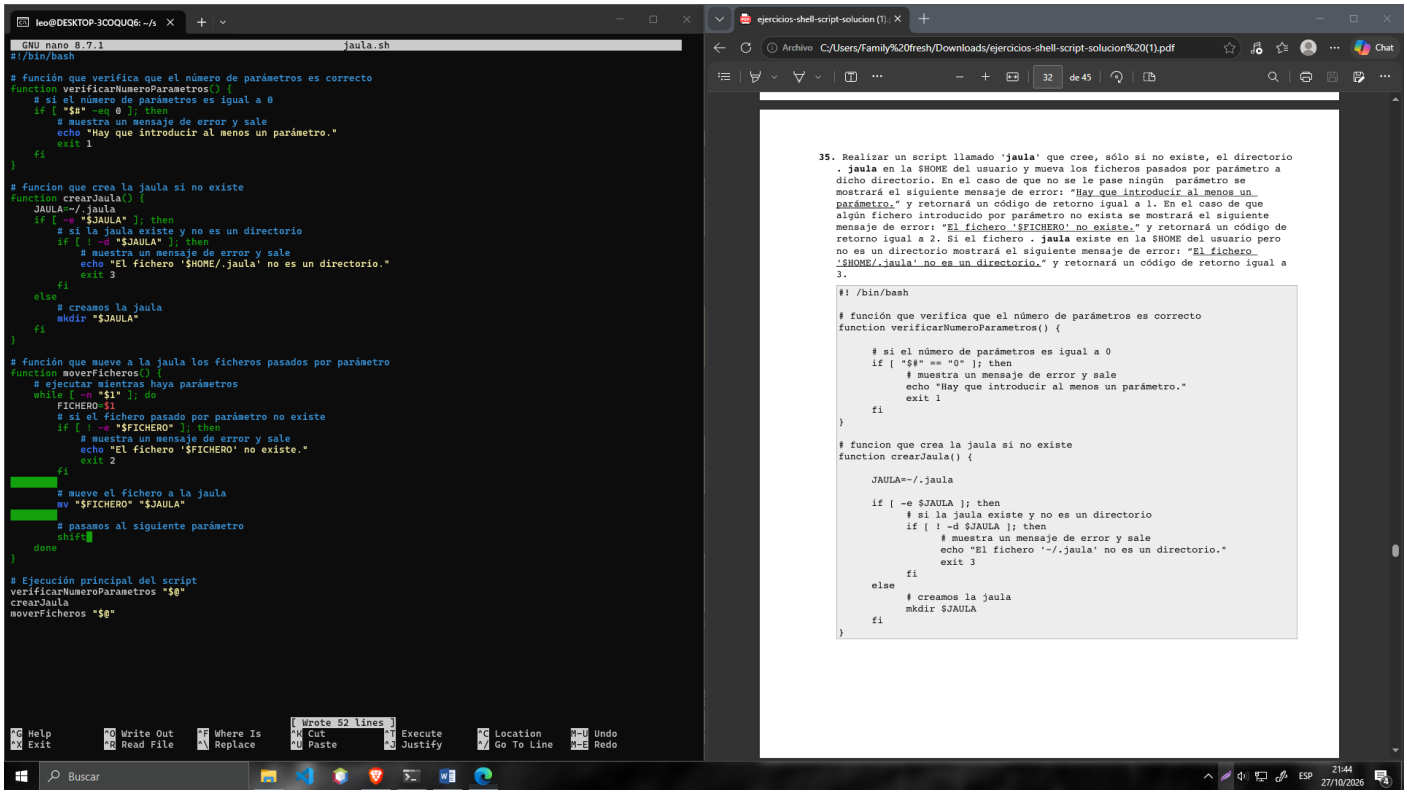


Figura 35.1. Captura original del ejercicio 35 (1 de 2).

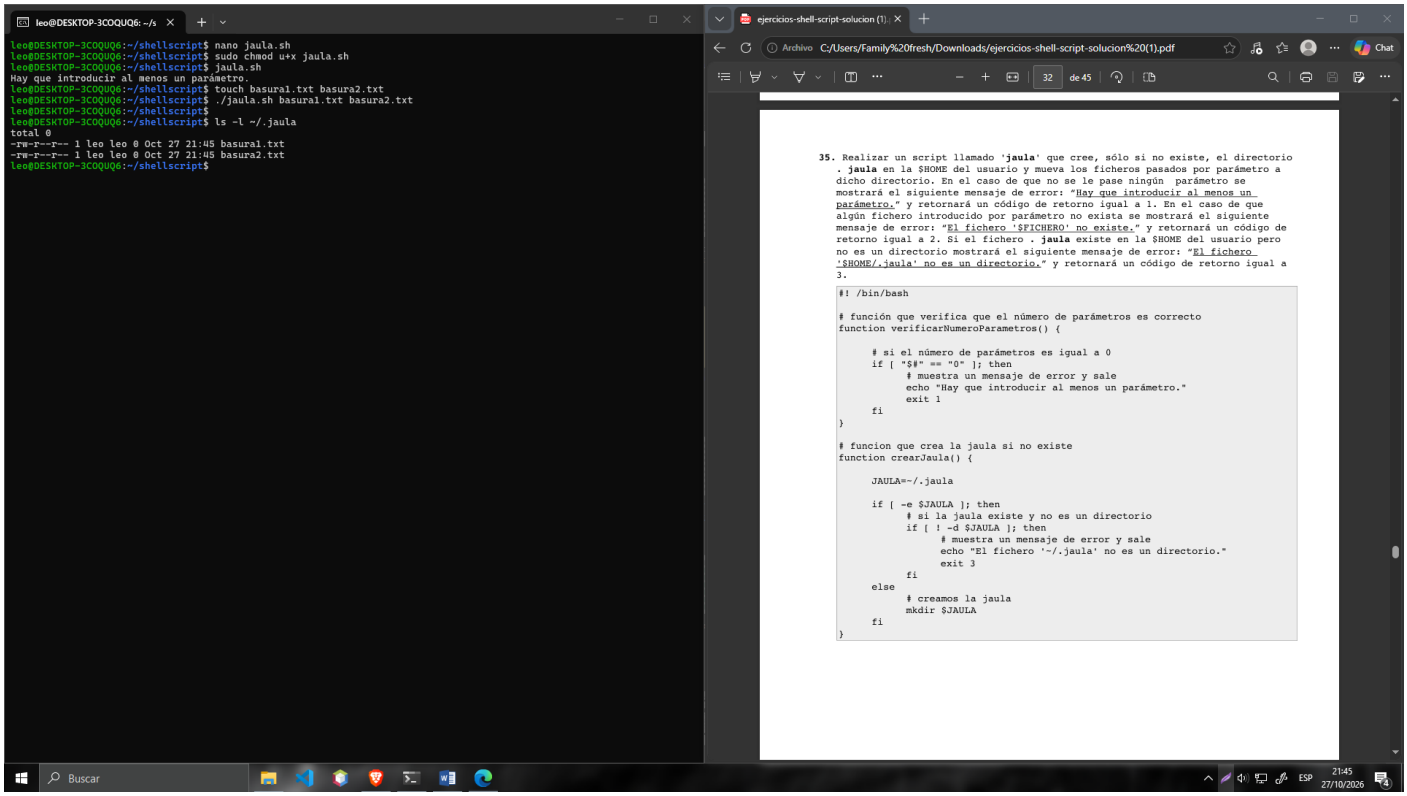


Figura 35.2. Captura original del ejercicio 35 (2 de 2).

Explicación de la salida: El programa organiza archivos dentro de la estructura de carpetas prevista por el ejercicio. Según la entrada proporcionada, puede mostrar confirmaciones de las operaciones o avisar cuando no se cumplen las condiciones necesarias.

Práctica 36. Consulta de calendario

Se desarrolló un script que muestra el calendario según los parámetros recibidos, mediante condicionales y selección de opciones.

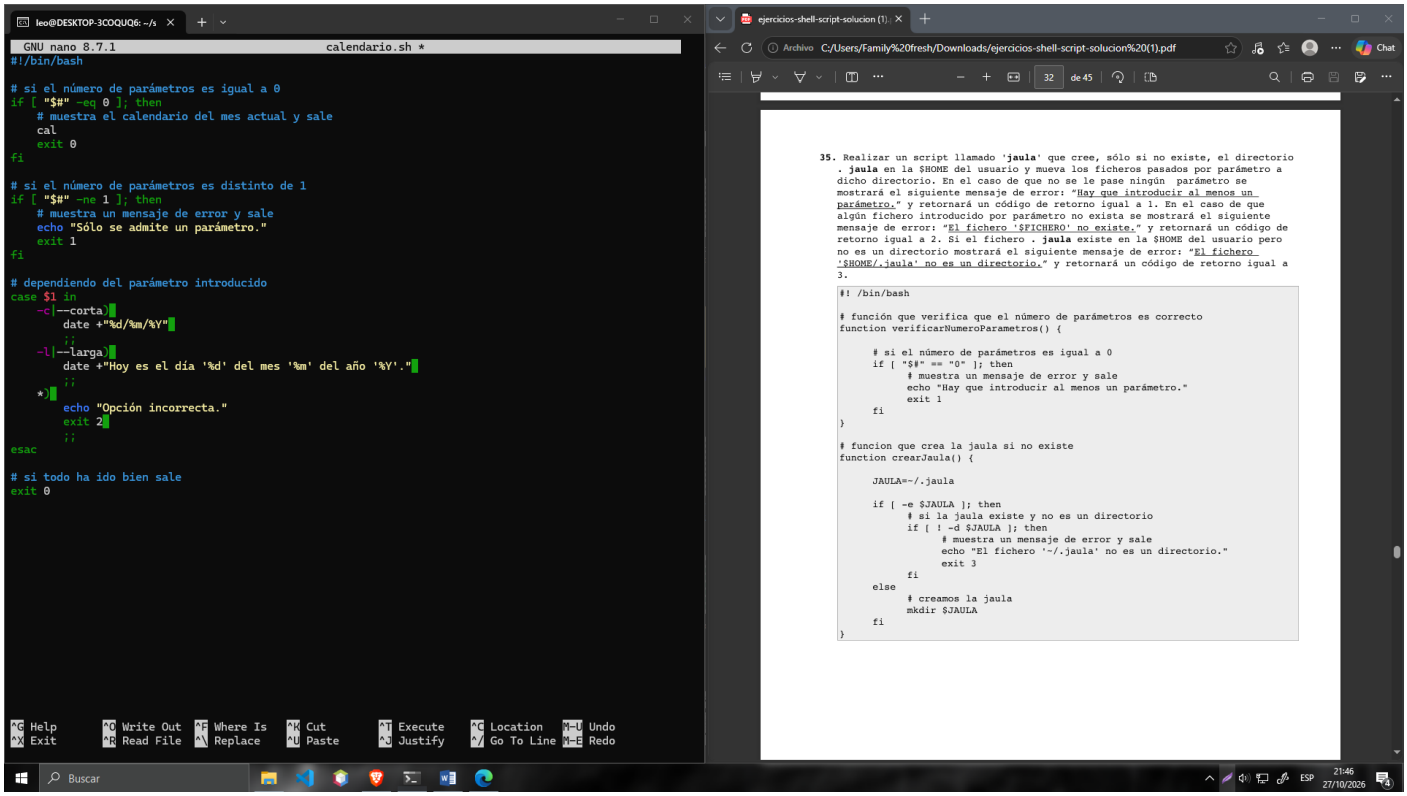


Figura 36.1. Captura original del ejercicio 36 (1 de 2).

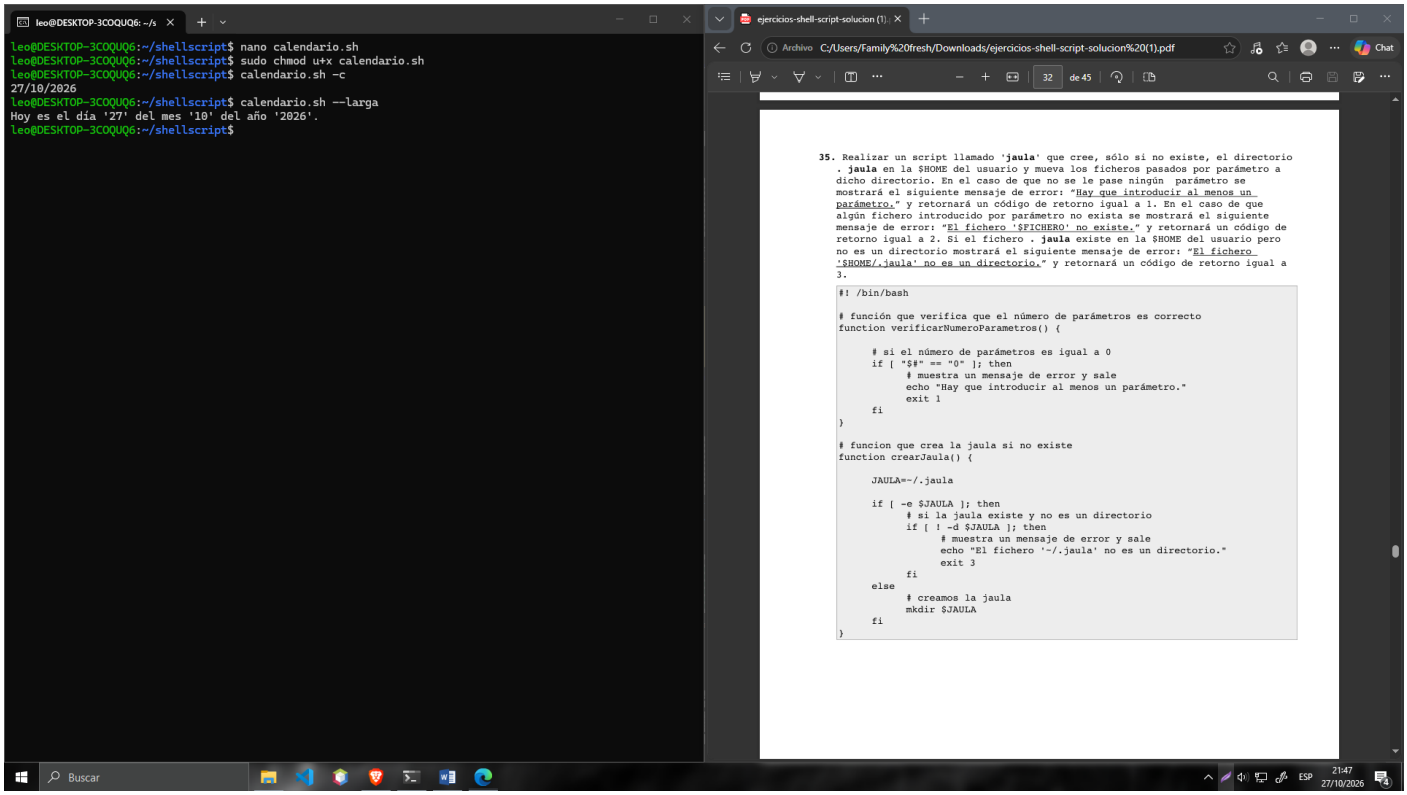


Figura 36.2. Captura original del ejercicio 36 (2 de 2).

Explicación de la salida: La salida muestra la información del calendario solicitada mediante los parámetros. El script cambia la consulta de acuerdo con las opciones recibidas y avisa si no se utiliza correctamente.

Práctica 37. Cálculo de potencias

Se implementó el cálculo de una potencia por medio de un ciclo que multiplica repetidamente la base.

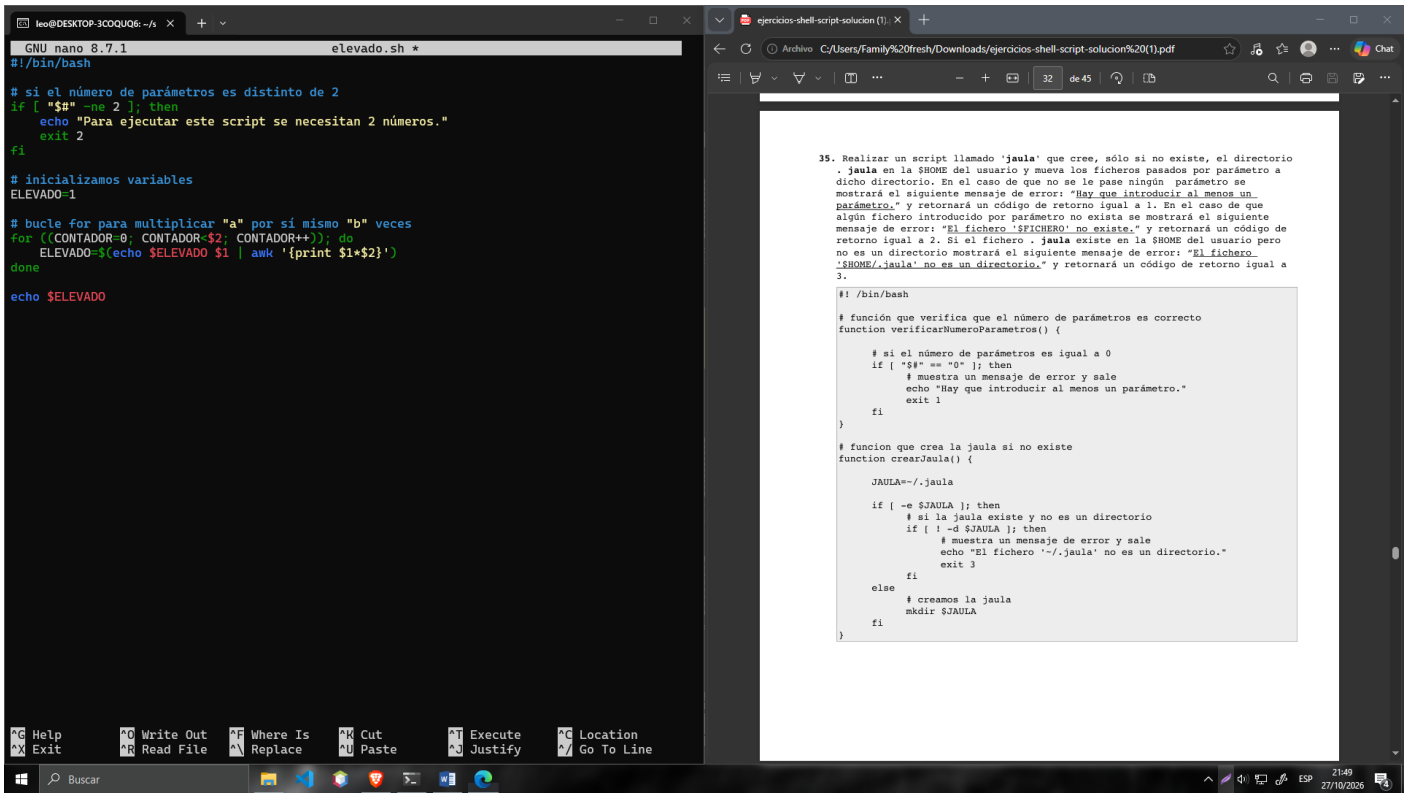


Figura 37.1. Captura original del ejercicio 37 (1 de 2).

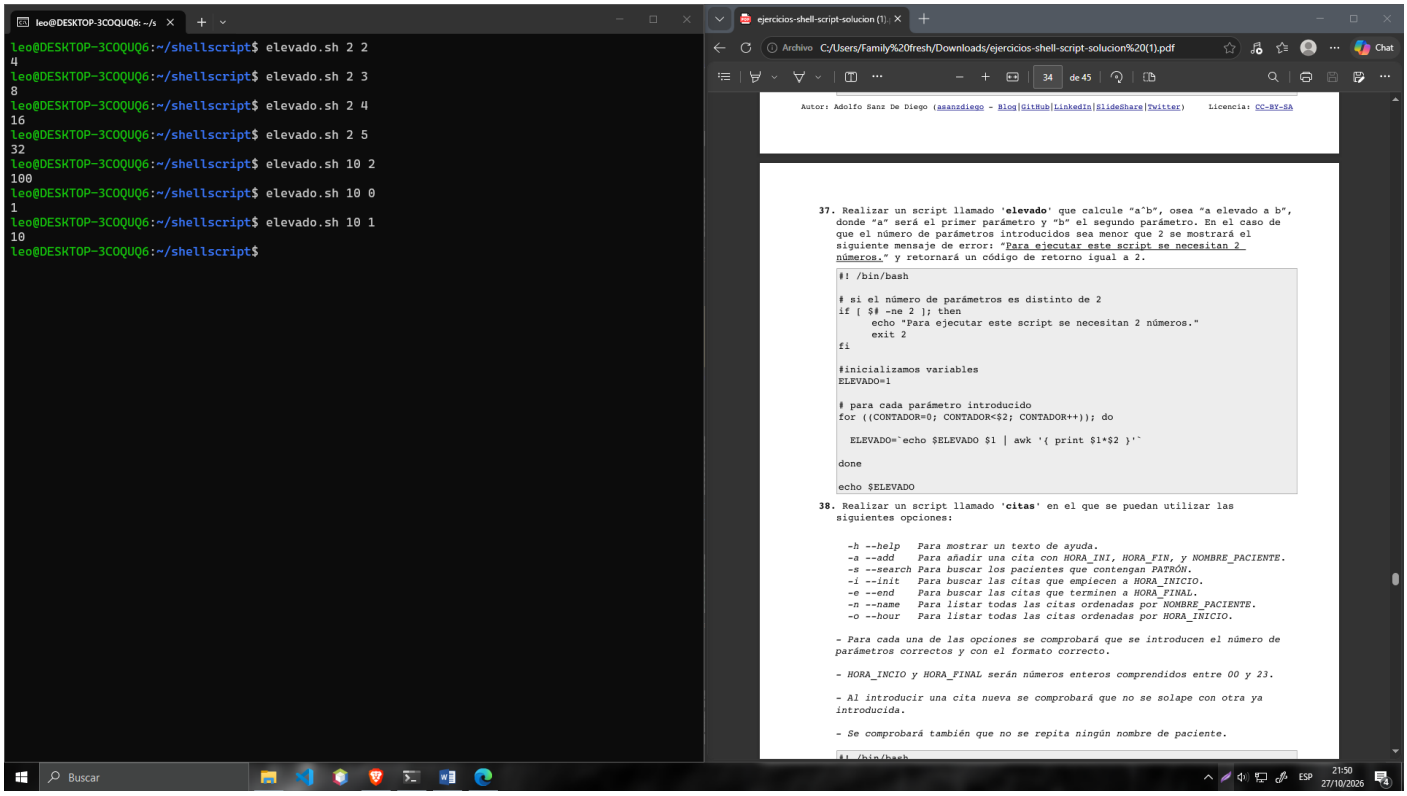


Figura 37.2. Captura original del ejercicio 37 (2 de 2).

Explicación de la salida: El programa calcula una potencia a partir de la base y el exponente indicados. En pantalla aparece el resultado de repetir las multiplicaciones necesarias o un aviso cuando faltan argumentos.

Práctica 38. Gestión de citas

Se creó un script para administrar citas con información almacenada en un archivo. Su código organiza las operaciones en funciones y contempla distintas opciones.

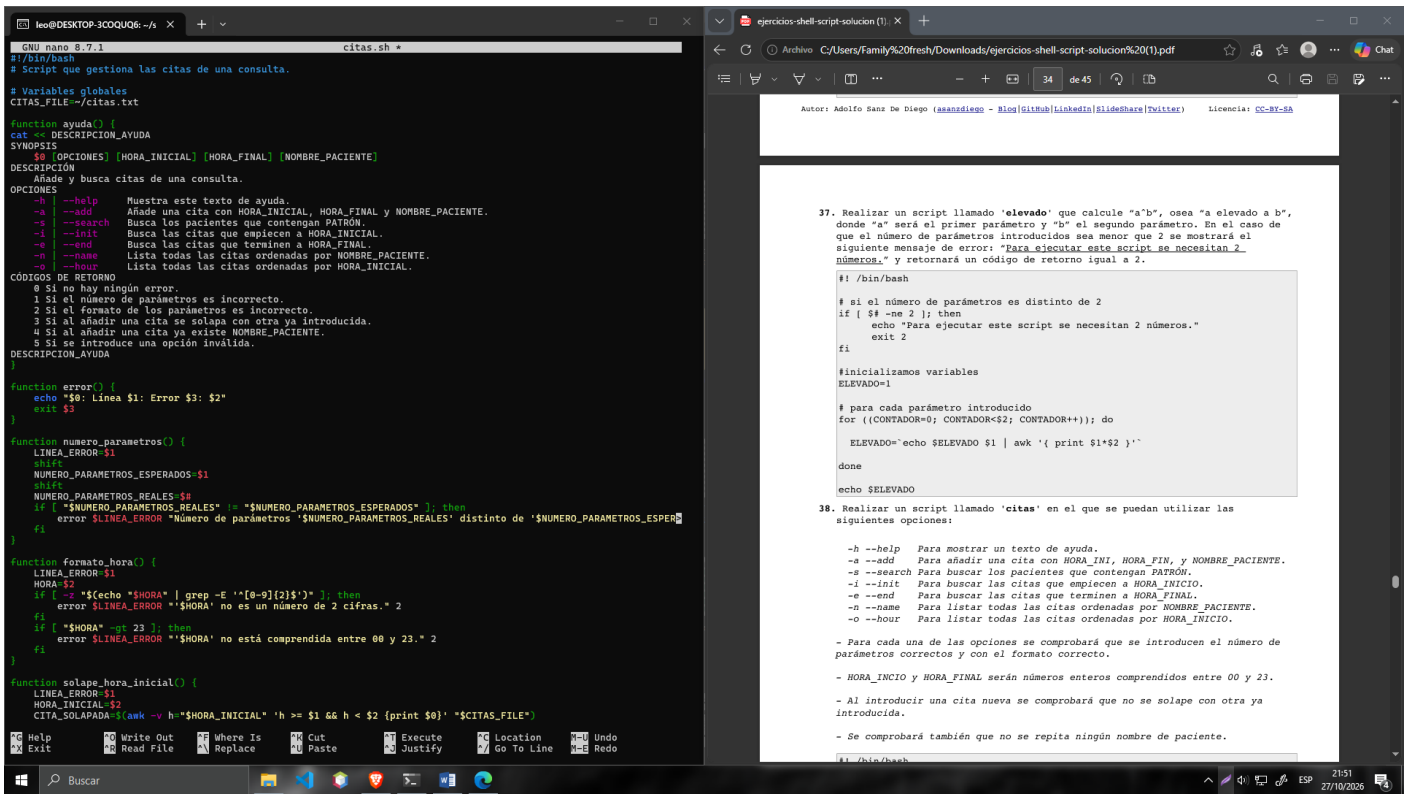


Figura 38.1. Captura original del ejercicio 38 (1 de 4).

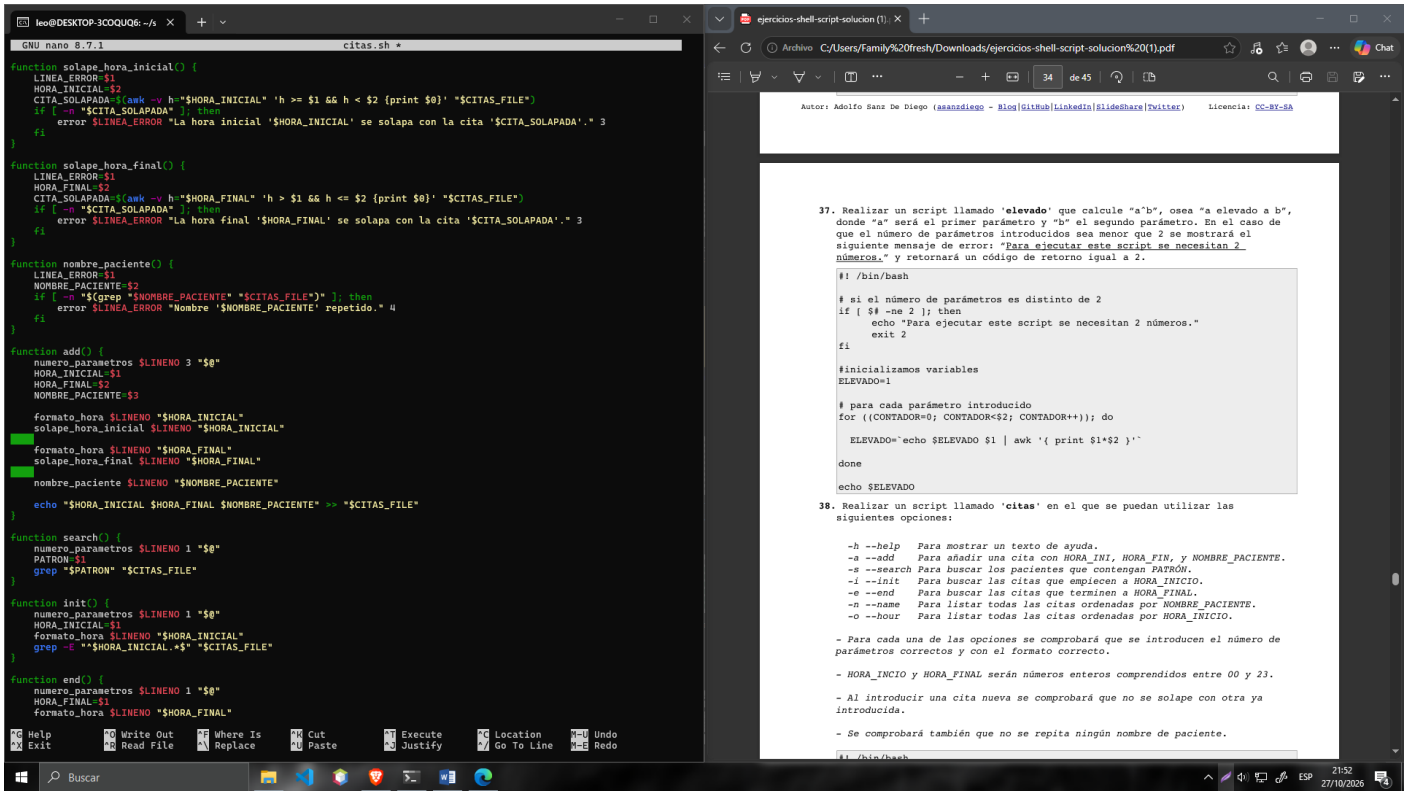


Figura 38.2. Captura original del ejercicio 38 (2 de 4).

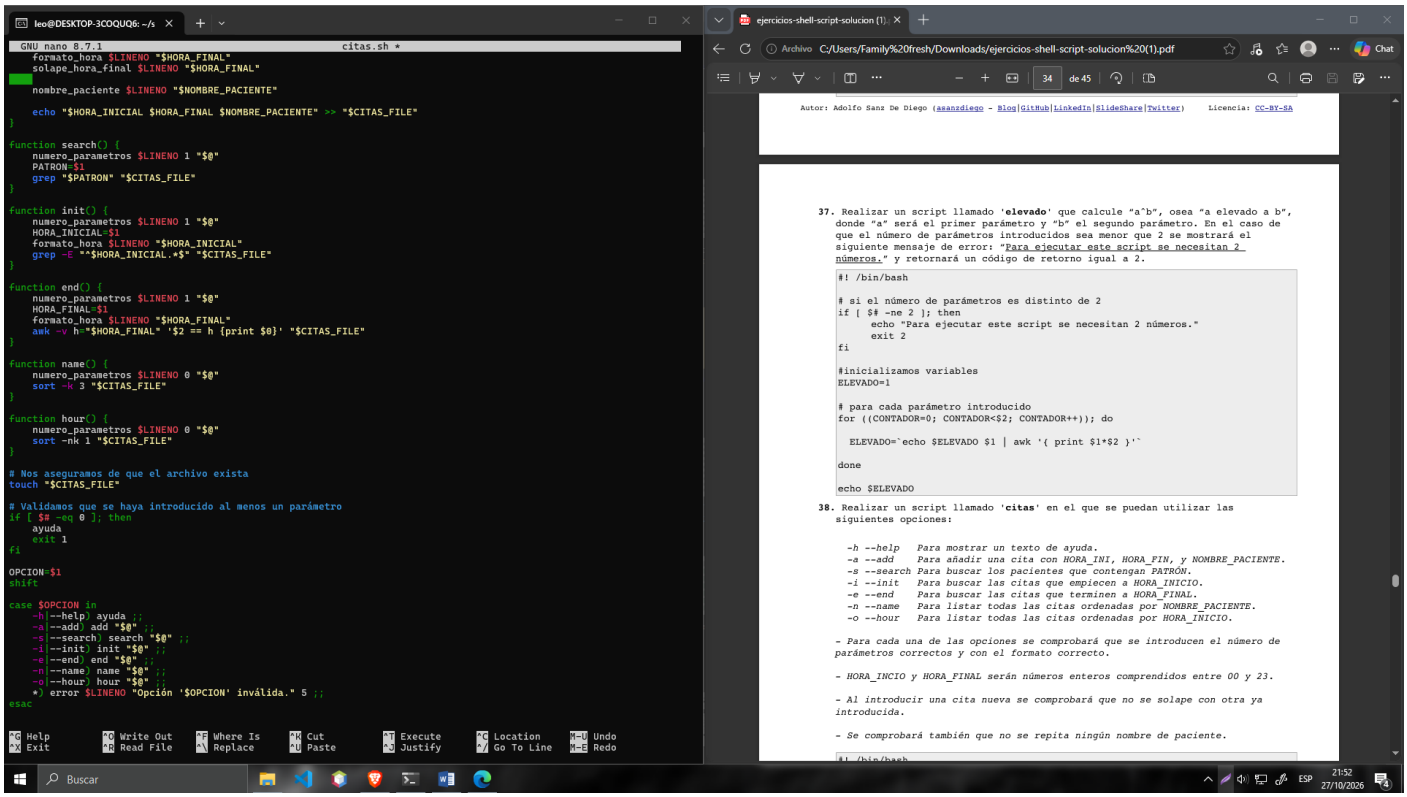


Figura 38.3. Captura original del ejercicio 38 (3 de 4).

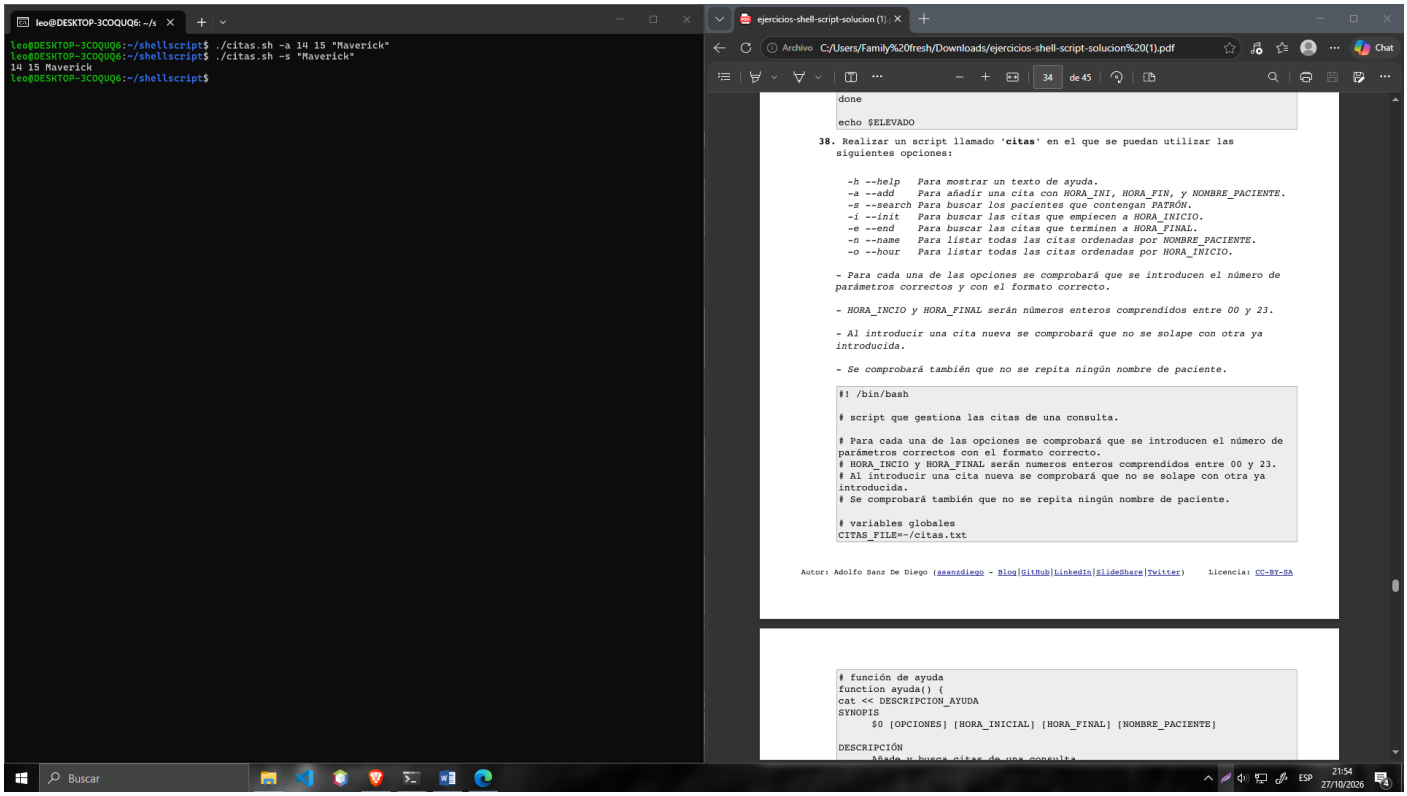


Figura 38.4. Captura original del ejercicio 38 (4 de 4).

Explicación de la salida: Este script permite registrar y consultar citas. La salida depende de la operación solicitada: puede mostrar los datos de las citas, confirmar un registro o indicar que existe un problema con la información proporcionada.

Práctica 39. Menú para gestionar citas

Se preparó una interfaz interactiva que utiliza las funciones del script de citas mediante opciones de un menú.

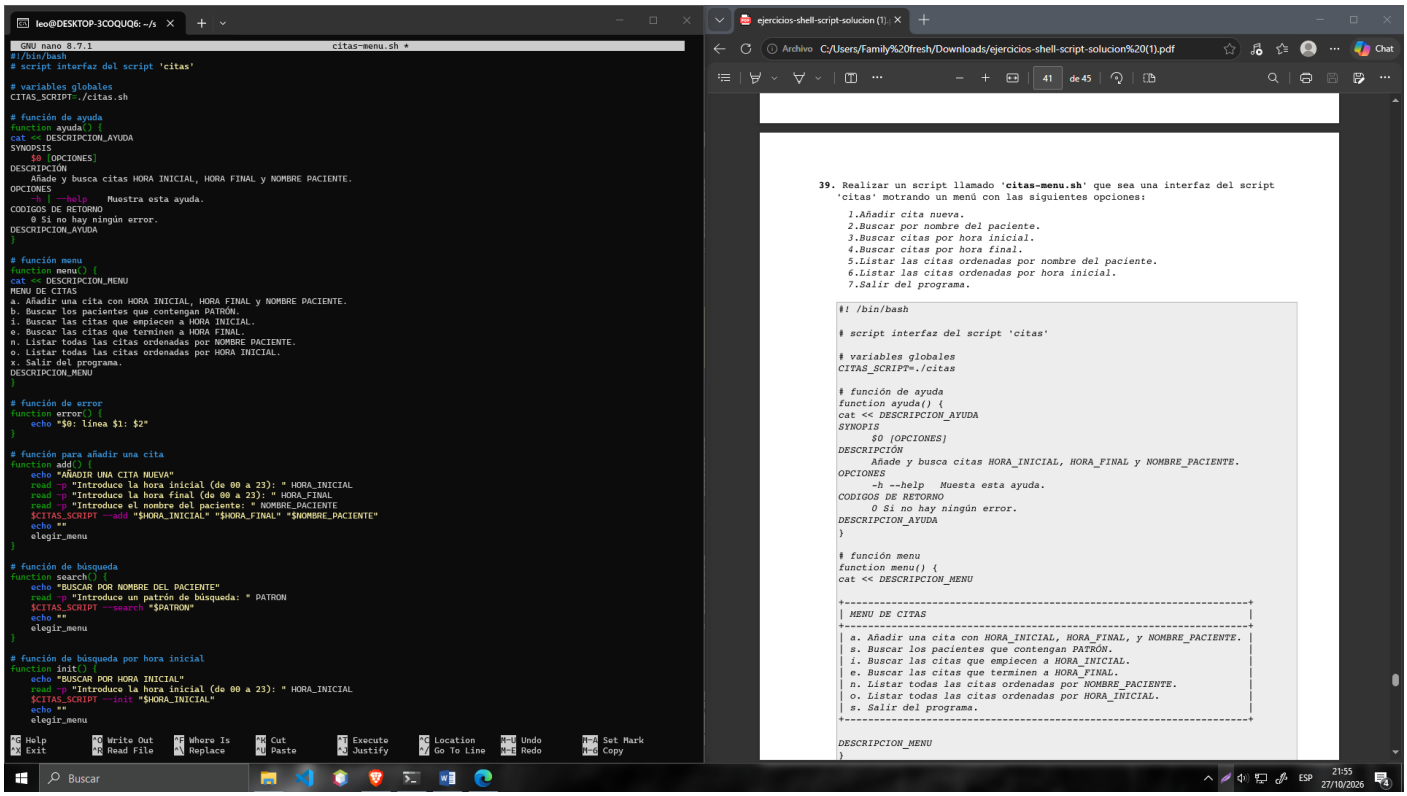


Figura 39.1. Captura original del ejercicio 39 (1 de 9).

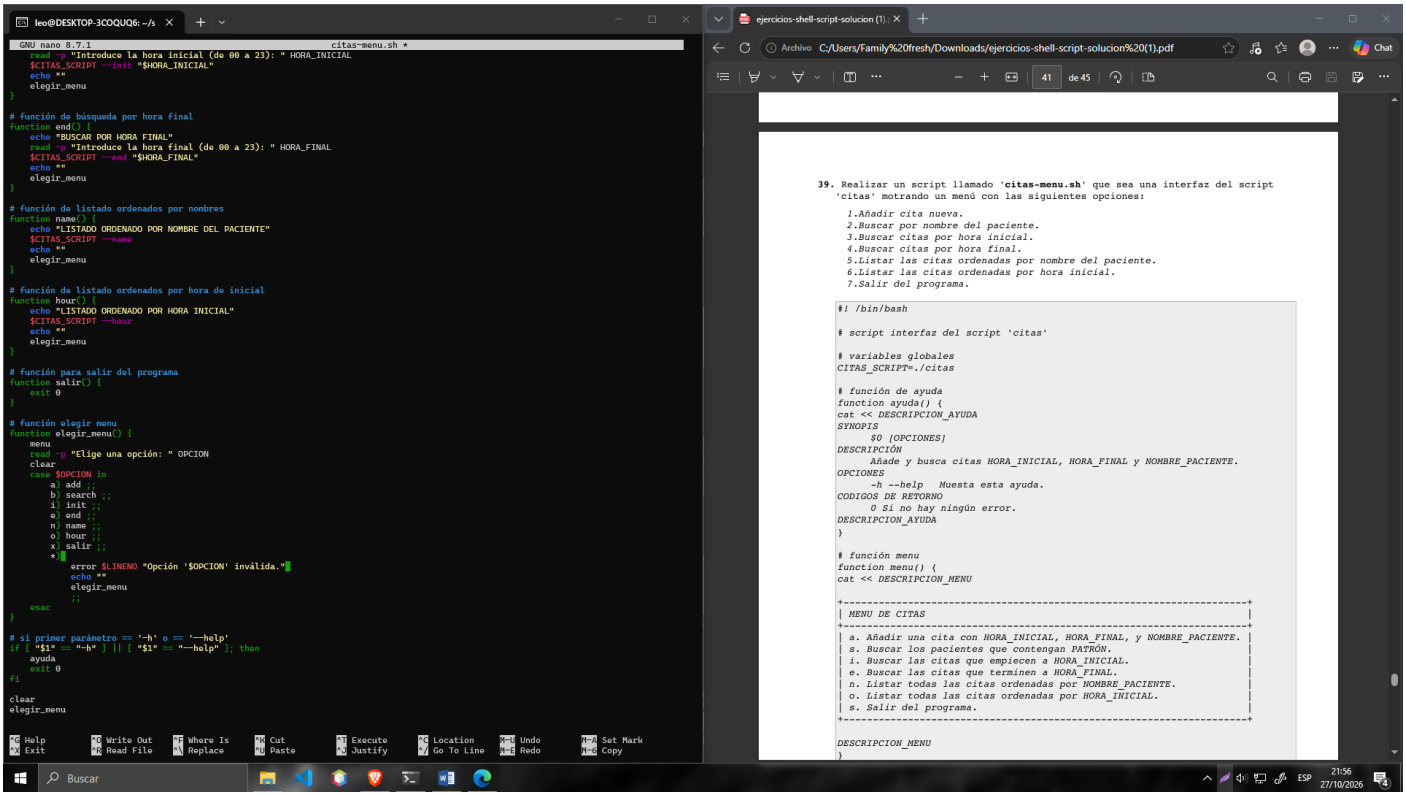


Figura 39.2. Captura original del ejercicio 39 (2 de 9).

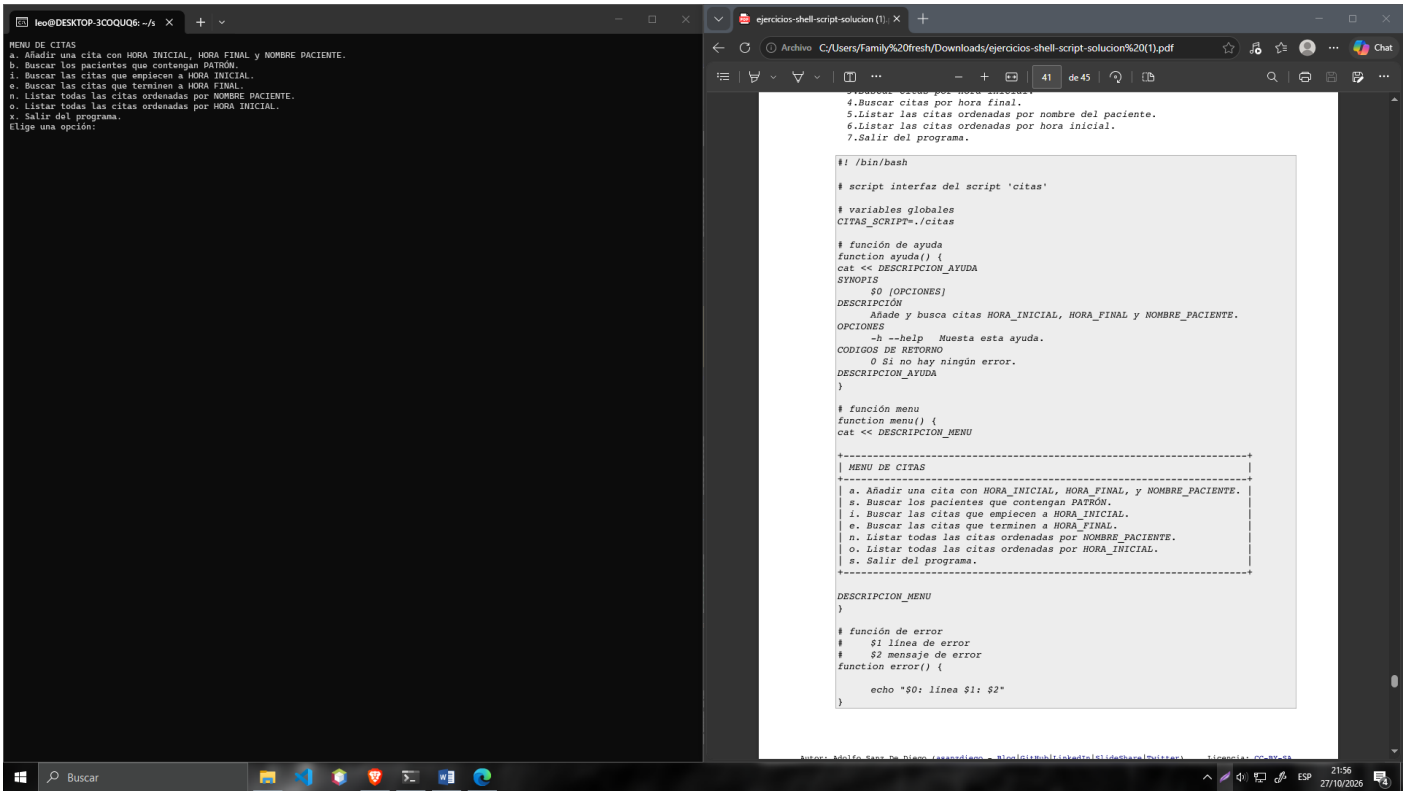


Figura 39.3. Captura original del ejercicio 39 (3 de 9).

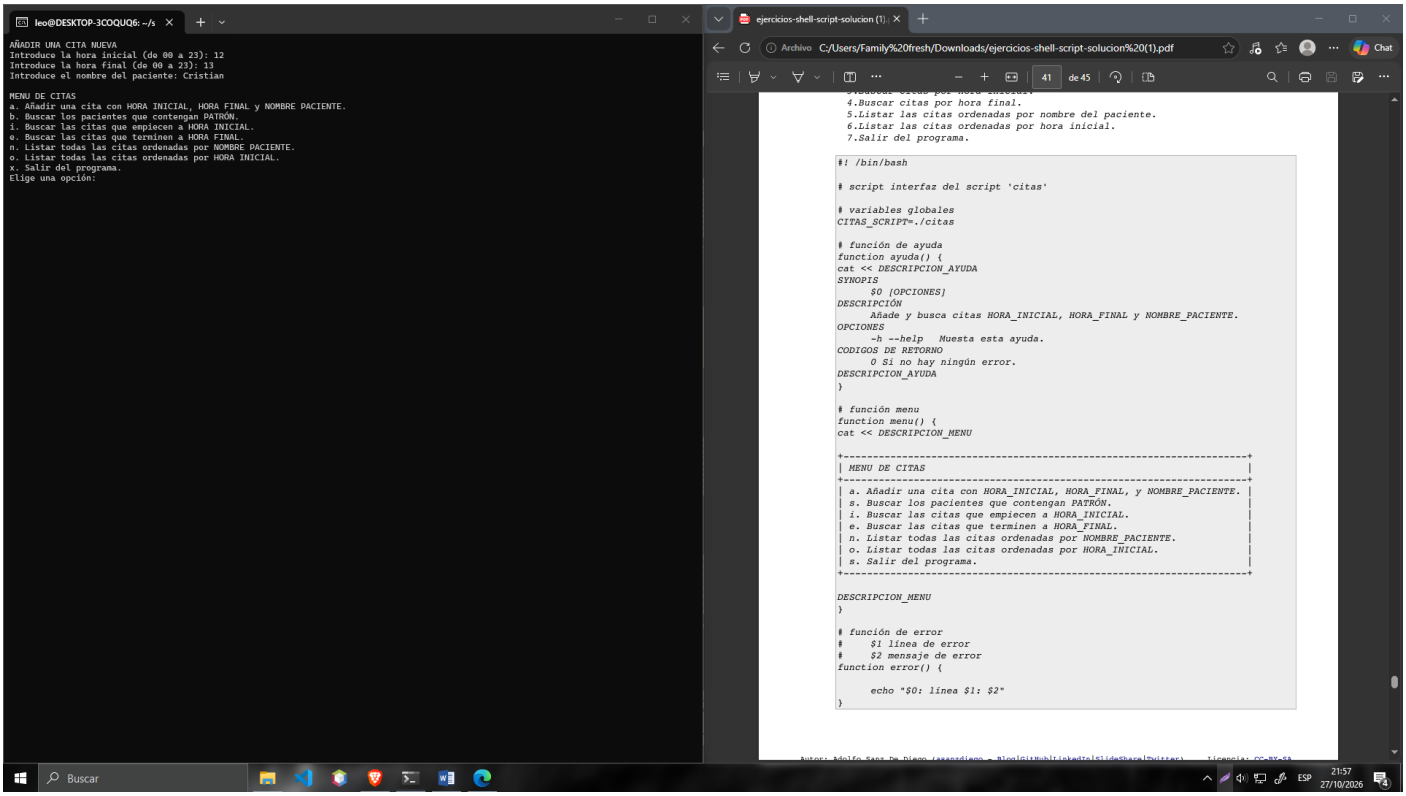


Figura 39.4. Captura original del ejercicio 39 (4 de 9).

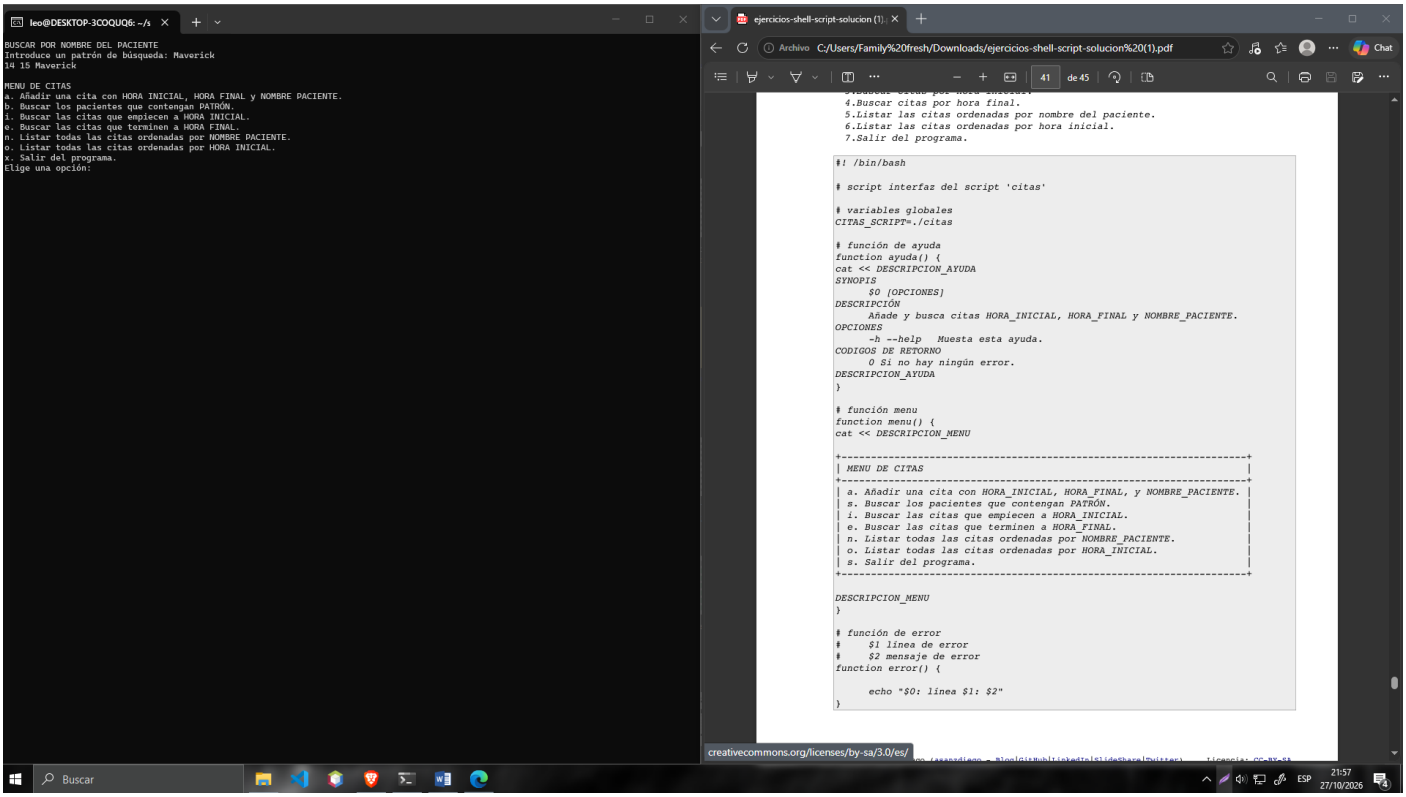


Figura 39.5. Captura original del ejercicio 39 (5 de 9).

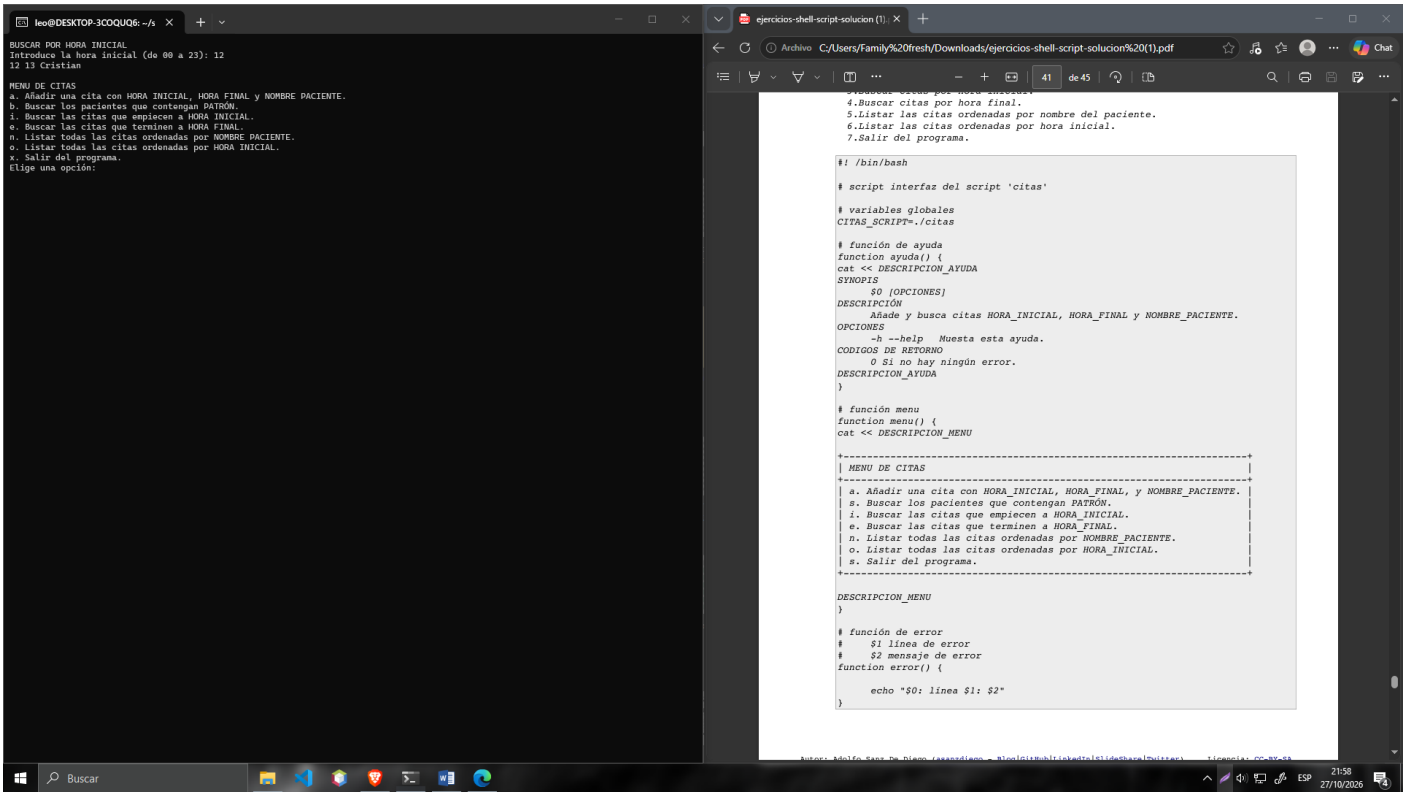


Figura 39.6. Captura original del ejercicio 39 (6 de 9).

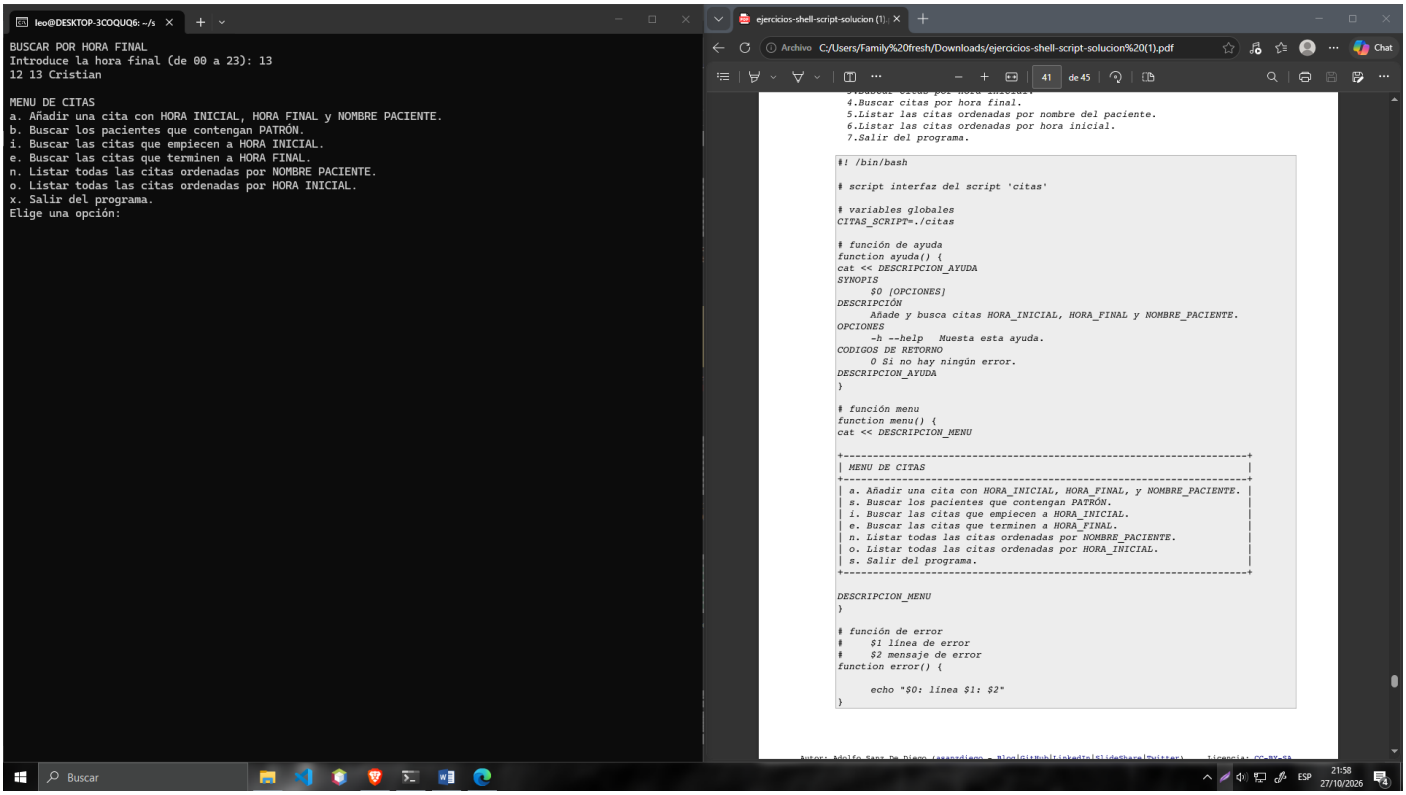


Figura 39.7. Captura original del ejercicio 39 (7 de 9).

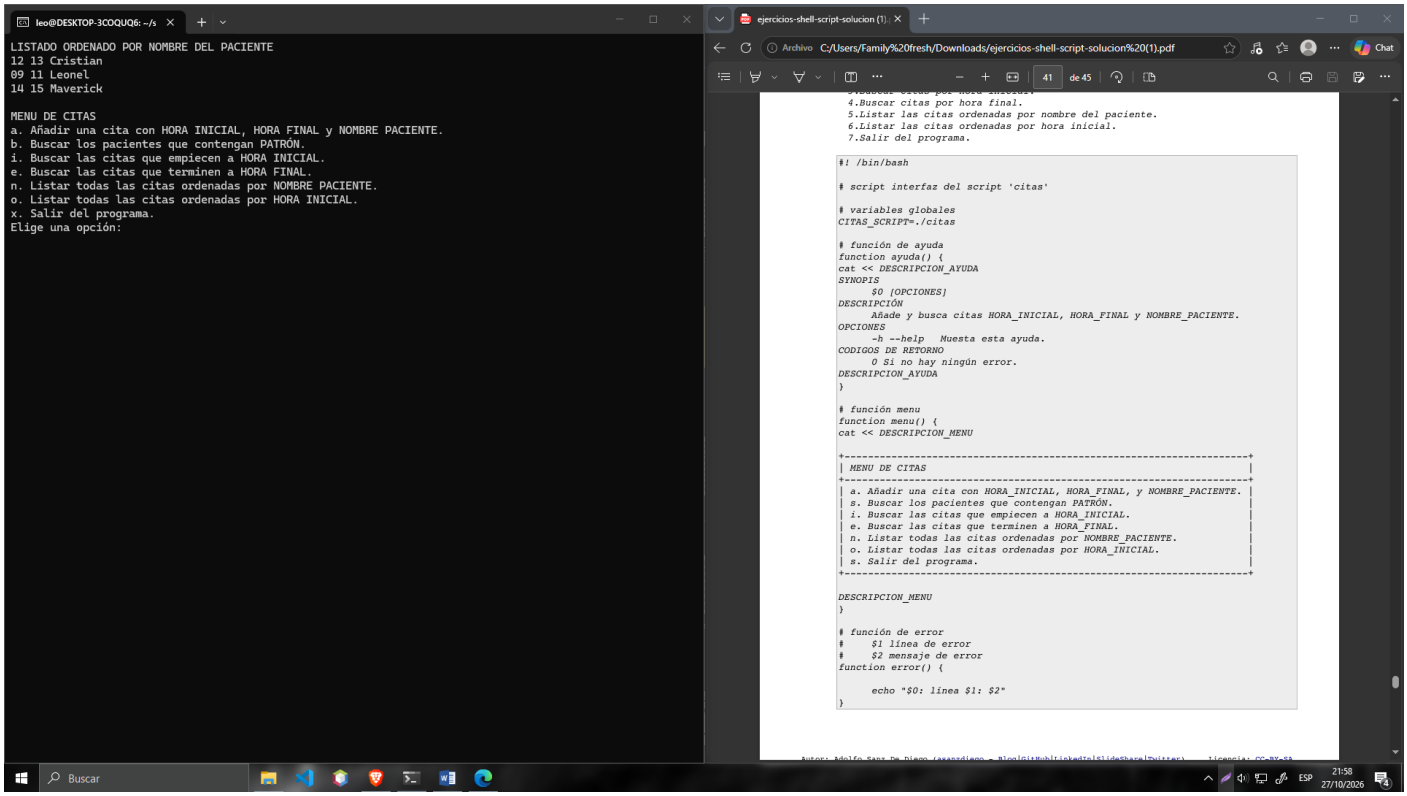


Figura 39.8. Captura original del ejercicio 39 (8 de 9).

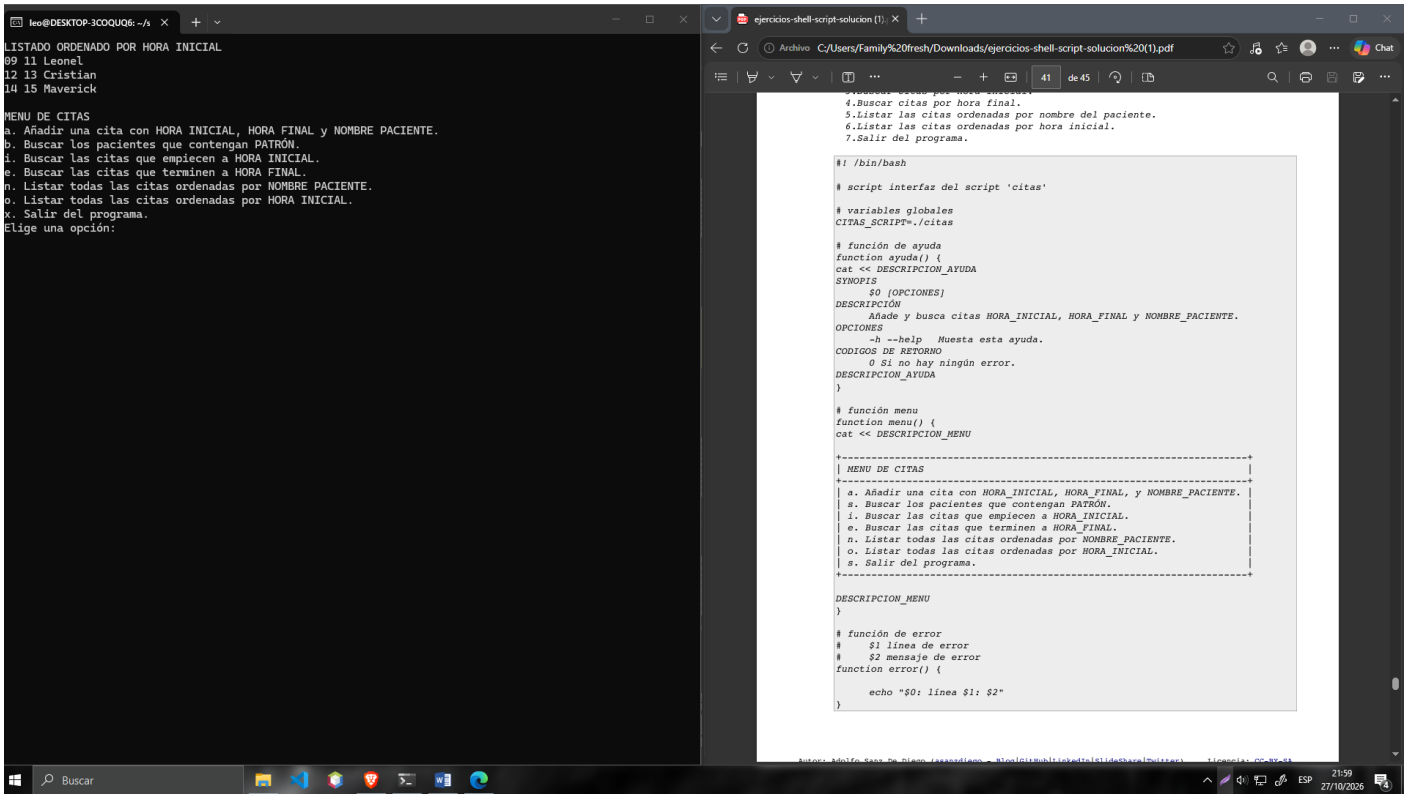


Figura 39.9. Captura original del ejercicio 39 (9 de 9).

Explicación de la salida: La ejecución presenta un menú para administrar las citas. Después de elegir una acción, se muestra la información correspondiente o el mensaje del programa cuando la solicitud no puede completarse.

Práctica 40. Gestión de citas mediante flags

Se implementó una segunda interfaz para el sistema de citas utilizando opciones de línea de comandos.

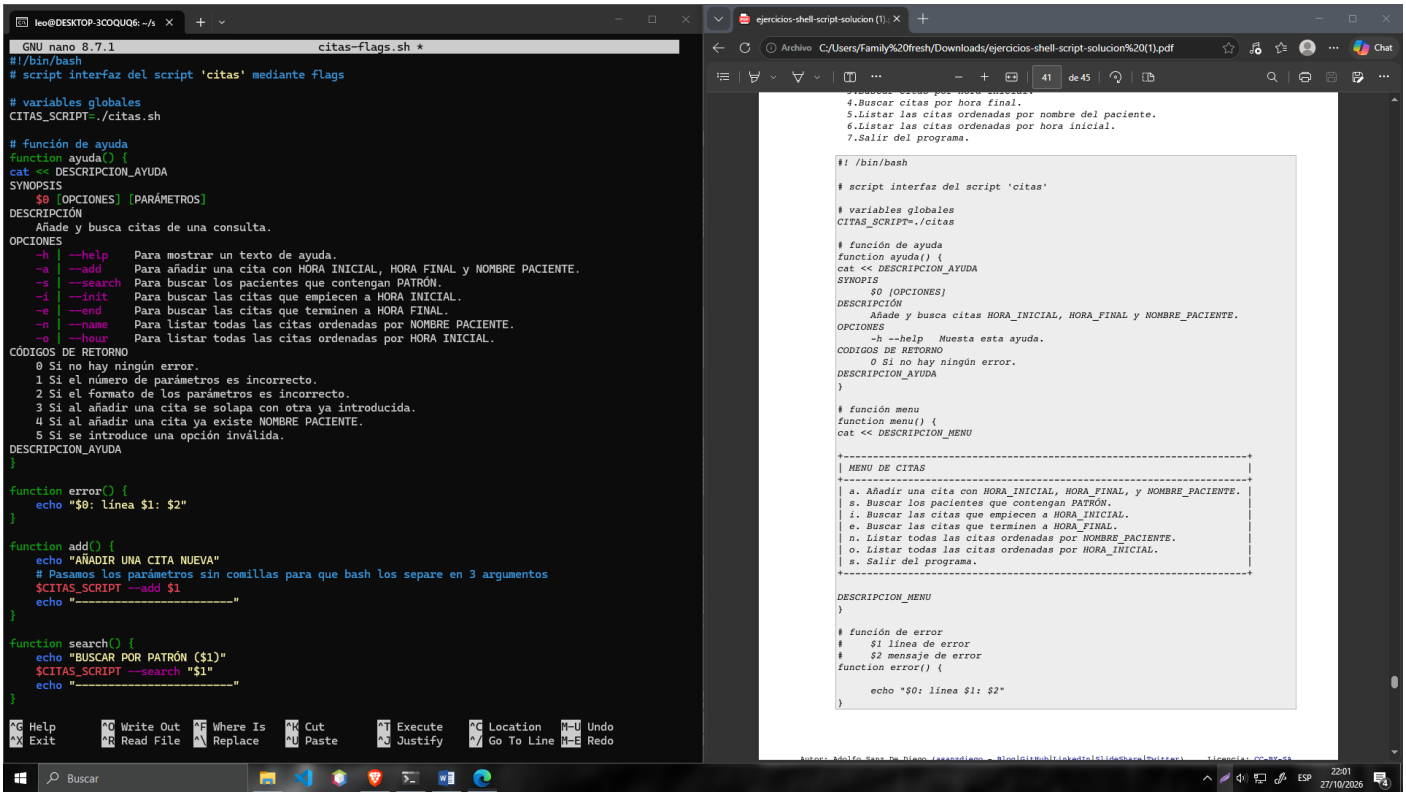


Figura 40.1. Captura original del ejercicio 40 (1 de 3).

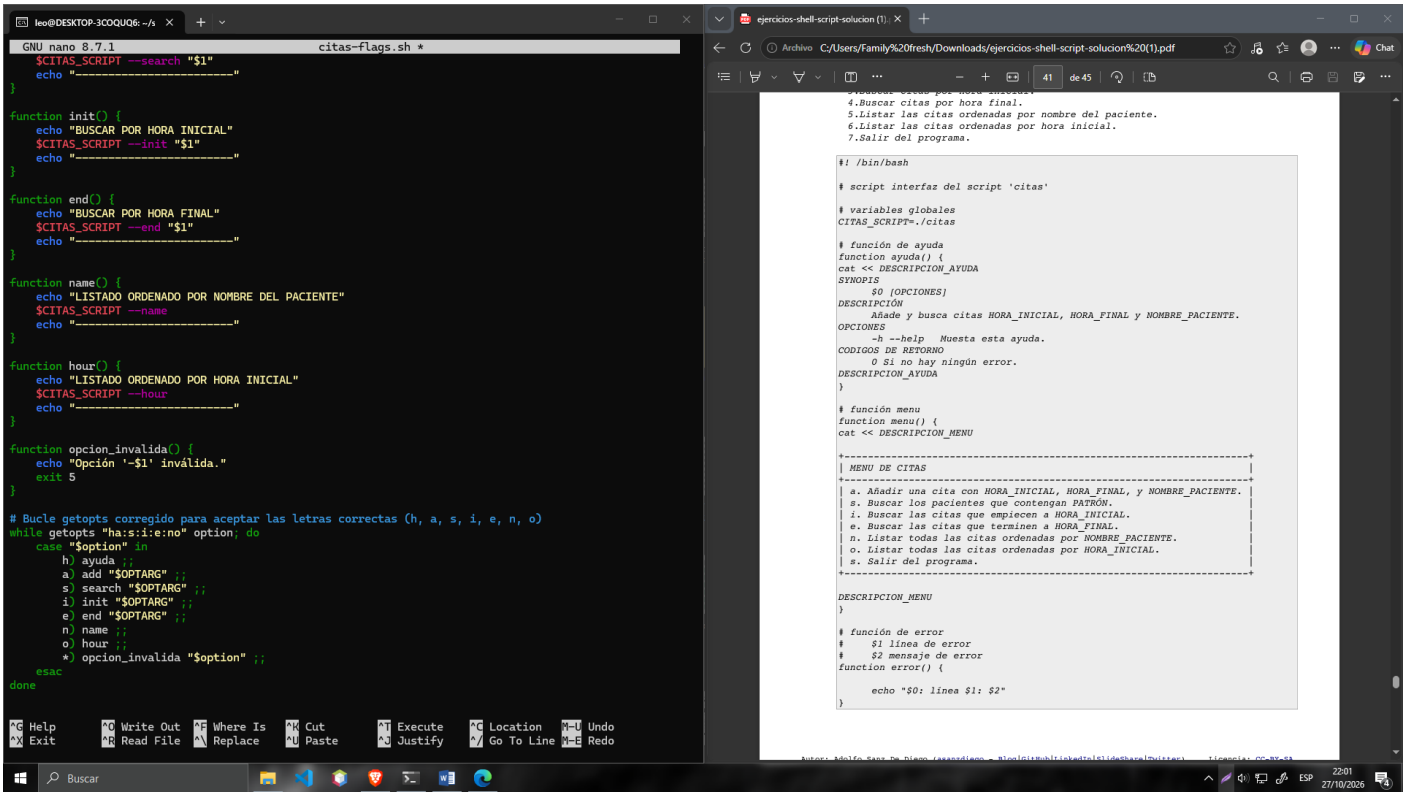


Figura 40.2. Captura original del ejercicio 40 (2 de 3).

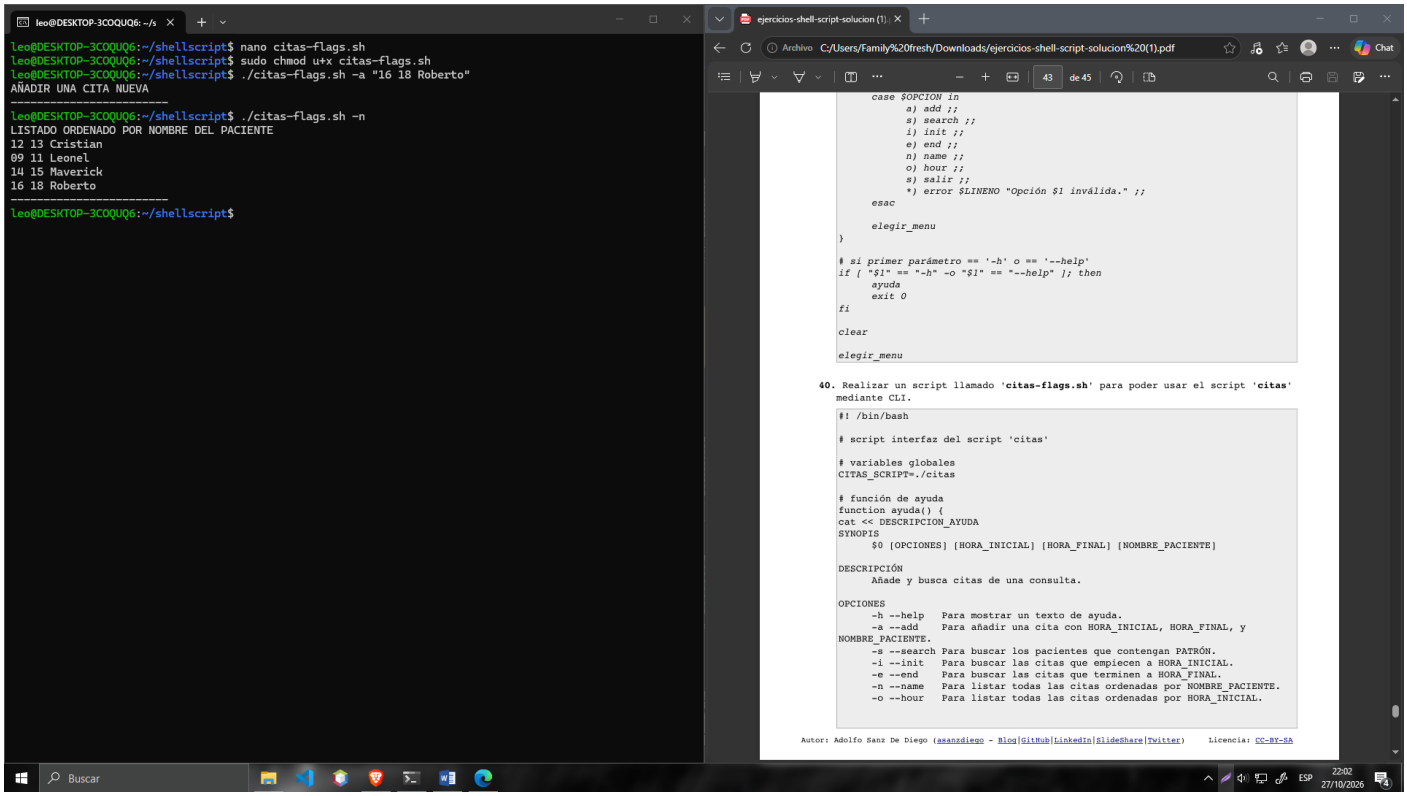


Figura 40.3. Captura original del ejercicio 40 (3 de 3).

Explicación de la salida: El script gestiona las citas mediante opciones escritas junto al comando. La terminal responde con el resultado de la acción solicitada, como una consulta o un registro, o muestra un mensaje si los parámetros son incorrectos.

Resultados generales

Las evidencias muestran las distintas pruebas realizadas con scripts de Bash, desde la validación de parámetros hasta operaciones aritméticas, procesos, copias de seguridad y gestión de archivos. Cada ejercicio conserva todas las capturas originales, incluso cuando incluye varios intentos o pruebas.

Conclusión

Las prácticas permitieron poner en funcionamiento distintos scripts desde la terminal de Linux. En las capturas se observan tanto la escritura y revisión de los programas como las respuestas obtenidas al ejecutarlos. La serie reúne diferentes formas de trabajar con argumentos, condiciones, archivos, procesos y comandos del sistema.